



Arquitecturas y Procesadores

Fundamentos para Sistemas Operativos

Guía de estudio integral

Resumen: Esta guía presenta los fundamentos de arquitectura de computadores necesarios para comprender el funcionamiento interno de los sistemas operativos modernos. Desde el modelo básico de ejecución hasta los procesadores multinúcleo y vectoriales, establece un puente entre el hardware y el software de sistema.

Curso: Sistemas Operativos

German Sánchez Torres, Ph.D.

Versión: 1.1

Índice general

0	Guía de uso de este documento	11
0.1	Objetivo de la guía y alcance	11
0.1.1	Contenido de la guía	11
0.1.2	Contenido no cubierto	11
0.2	Prerrequisitos mínimos	12
0.2.1	Representación de datos	12
0.2.2	Programación en C	12
0.2.3	Noción de memoria	12
0.3	Metodología de estudio	12
0.3.1	Estructura de cada capítulo	12
0.3.2	Convenciones visuales	13
0.3.3	Metodología de estudio recomendada	13
0.3.4	Tiempo estimado de dedicación	14
0.4	Software y herramientas recomendadas	14
0.4.1	Entorno Linux	14
0.4.2	Verificación del entorno	15
0.5	Preguntas de control del capítulo 0	15
1	Fundamentos de arquitectura para sistemas operativos	17
1.1	Información del hardware necesaria para el SO	17
1.1.1	La CPU y sus capacidades	17
1.1.2	La jerarquía de memoria	18
1.1.3	Dispositivos de E/S	18
1.2	Mapa conceptual del sistema	19
1.2.1	El flujo de control	19
1.3	Conexión directa con temas del curso	19
1.3.1	Procesos y planificación	19
1.3.2	Memoria virtual	20
1.3.3	Concurrencia y sincronización	21
1.4	Síntesis del capítulo	22
1.5	Preguntas de control del capítulo 1	22
2	Modelo mínimo de ejecución de un procesador	23
2.1	Componentes básicos	23
2.1.1	Unidad aritmético-lógica (ALU)	23
2.1.2	Registros	24
2.1.3	Unidad de control (CU)	25
2.1.4	Buses	25
2.2	Ciclo de búsqueda-decodificación-ejecución	25
2.2.1	Búsqueda (<i>fetch</i>)	25
2.2.2	Decodificación (<i>decode</i>)	26
2.2.3	Ejecución (<i>execute</i>)	26

2.3	Instrucciones, registros y memoria en el contrato CPU–RAM	27
2.3.1	Espacio de direcciones	27
2.3.2	Modelo de memoria del programador	27
2.4	Costos de acceso a memoria y cómputo	28
2.4.1	Latencias aproximadas	28
2.4.2	Implicaciones para el programador de sistemas	29
2.5	Síntesis del capítulo	30
2.6	Preguntas de control del capítulo 2	30
3	Organización de memoria en las arquitecturas de Von Neumann y Harvard	31
3.1	Arquitectura de Von Neumann	31
3.1.1	La idea fundamental	31
3.1.2	Ventajas de Von Neumann	32
3.1.3	Limitaciones de Von Neumann	32
3.2	El cuello de botella de Von Neumann	33
3.2.1	Análisis del problema	33
3.2.2	Impacto en el rendimiento	33
3.3	Arquitectura Harvard	34
3.3.1	La idea de separación	34
3.3.2	Ventajas de Harvard	34
3.4	Harvard modificada	35
3.4.1	Implementación en CPU modernas	35
3.4.2	Características clave	36
3.5	Implicaciones para el SO	36
3.5.1	Rendimiento y cachés	36
3.5.2	Protección y seguridad	36
3.6	Síntesis del capítulo	37
3.7	Preguntas de control del capítulo 3	37
4	Diferencias entre ISA y microarquitectura	39
4.1	Definición de la ISA y su relación con el SO	39
4.1.1	Componentes de una ISA	39
4.1.2	ISA principales	40
4.2	Convenciones ABI y modos de ejecución	40
4.2.1	Interfaz binaria de aplicaciones	40
4.2.2	Modos de ejecución	41
4.3	Implementación microarquitectónica de la ISA	42
4.3.1	Elementos de la microarquitectura	42
4.3.2	Ejemplos de implementaciones x86-64	42
4.4	Elementos visibles para el software y dependientes de la implementación	42
4.4.1	Visible para el software (ISA)	43
4.4.2	No visible directamente (microarquitectura)	43
4.4.3	Ejemplo de impacto en el rendimiento	44
4.5	Síntesis del capítulo	44
4.6	Preguntas de control del capítulo 4	44
5	Filosofías de diseño CISC y RISC	45
5.1	Motivación histórica de CISC y RISC	45
5.1.1	La era CISC (décadas de 1970 y 1980)	45
5.1.2	El surgimiento de RISC (década de 1980)	46
5.2	Complejidad de las instrucciones frente a simplicidad y regularidad	46
5.2.1	Características CISC	46

5.2.2	Características RISC	47
5.3	Microcódigo, longitud variable y modos de direccionamiento	48
5.3.1	Microcódigo en CISC	48
5.3.2	Longitud variable frente a longitud fija	48
5.4	Impacto en compiladores y consecuencias para el SO	49
5.4.1	Generación de código	49
5.4.2	Implicaciones para el sistema operativo	50
5.5	Convergencia en los procesadores modernos	50
5.5.1	Decodificación x86 moderna	50
5.6	Síntesis del capítulo	50
5.7	Preguntas de control del capítulo 5	51
6	Procesadores no segmentados y segmentados	53
6.1	Diseño no segmentado y utilización de recursos	53
6.1.1	Procesador de un ciclo	54
6.2	Segmentación por etapas	54
6.2.1	Principio de funcionamiento	54
6.3	Etapas típicas y función de cada una	54
6.3.1	Etapas clásicas de una segmentación	55
6.3.2	Mejora en la tasa de procesamiento	55
6.4	Riesgos estructurales, de datos y de control	55
6.4.1	Riesgos estructurales	56
6.4.2	Riesgos de datos	56
6.4.3	Riesgos de control	57
6.5	Técnicas de mitigación mediante esperas, reenvío y predicción	57
6.5.1	Esperas e inserción de burbujas	57
6.5.2	Reenvío de datos	57
6.5.3	Predicción de saltos	58
6.6	Síntesis del capítulo	58
6.7	Preguntas de control del capítulo 6	59
7	Métricas para el análisis del rendimiento	61
7.1	Latencia y tasa de procesamiento	61
7.1.1	Analogía intuitiva	61
7.1.2	En el contexto de los procesadores	62
7.2	Interpretación de CPI e IPC y errores comunes	62
7.2.1	Ecuación del rendimiento	62
7.2.2	Factores que afectan al CPI	63
7.2.3	Errores comunes de interpretación	63
7.3	Amdahl y límites de la aceleración	63
7.3.1	Ejemplo ilustrativo	63
7.3.2	Implicaciones para el diseño de sistemas	64
7.4	Mediciones básicas de rendimiento en Linux	64
7.4.1	Herramientas básicas	65
7.4.2	Métricas clave	65
7.4.3	Ejemplo de medición completa	65
7.5	Síntesis del capítulo	67
7.6	Preguntas de control del capítulo 7	67
8	Procesadores escalares y superescalares	69
8.1	Procesamiento escalar con una emisión por ciclo	69
8.1.1	Características del diseño escalar	70

8.2	Procesamiento superescalar con varias emisiones por ciclo	70
8.2.1	Ejemplo de microarquitectura superescalar fuera de orden	70
8.2.2	Grados de superescalaridad	71
8.3	Paralelismo a nivel de instrucción (ILP)	71
8.3.1	Factores que limitan el ILP	71
8.3.2	Técnicas para explotar el ILP	72
8.4	Especulación y predicción de saltos	72
8.4.1	Tipos de especulación	72
8.4.2	Impacto de la predicción de saltos	73
8.5	Ejecución fuera de orden y su efecto en el rendimiento	73
8.5.1	Ejemplo conceptual de ejecución fuera de orden	73
8.5.2	Estructura de un procesador fuera de orden	74
8.6	Síntesis del capítulo	74
8.7	Preguntas de control del capítulo 8	74
9	Procesamiento vectorial (SIMD)	77
9.1	Condiciones para aplicar el paralelismo de datos	77
9.1.1	Aplicaciones adecuadas para SIMD	77
9.1.2	Limitaciones para el uso de SIMD	78
9.2	Registros vectoriales y operaciones en bloque	78
9.2.1	Evolución de las extensiones SIMD en x86	78
9.2.2	Ejemplo de operación SIMD	79
9.3	Ejemplos conceptuales	79
9.3.1	Suma de vectores	79
9.3.2	Filtro de imagen por convolución	80
9.4	Papel del compilador y de las funciones intrínsecas	80
9.4.1	Autovectorización	81
9.4.2	Funciones intrínsecas y autovectorización	81
9.5	Relación con el SO	81
9.5.1	Detección de capacidades SIMD	81
9.5.2	Estado vectorial y sistema operativo	82
9.6	Síntesis del capítulo	82
9.7	Preguntas de control del capítulo 9	83
10	Procesadores multinúcleo y paralelismo entre hilos	85
10.1	Motivación para el uso de varios núcleos	85
10.1.1	Evolución de los procesadores	85
10.2	Jerarquía de cachés privadas y compartidas	86
10.2.1	Organización típica	86
10.3	Coherencia de caché	86
10.3.1	Protocolo MESI	87
10.3.2	Ejemplo de transiciones MESI	87
10.4	Consistencia de memoria	87
10.4.1	Modelos de consistencia	88
10.4.2	Implicaciones para la programación	88
10.5	NUMA y afinidad	89
10.5.1	Arquitectura NUMA	89
10.5.2	Afinidad de procesos	89
10.6	SMT o multihilo simultáneo	90
10.6.1	Beneficios y limitaciones	90
10.7	Síntesis del capítulo	91
10.8	Preguntas de control del capítulo 10	91

11 Hardware y sistemas operativos	93
11.1 Interrupciones, excepciones y llamadas al sistema	93
11.1.1 Tipos de eventos	94
11.1.2 Manejo de interrupciones	94
11.1.3 Llamadas al sistema	95
11.2 Modos de privilegio y protección	95
11.2.1 Anillos de protección en x86	95
11.2.2 Instrucciones privilegiadas	95
11.3 Cambio de contexto y su costo	96
11.3.1 Estado conservado durante un cambio de contexto	96
11.3.2 Fuentes de costo	97
11.4 Planificación en sistemas multinúcleo	97
11.4.1 Estrategias de planificación	97
11.5 Sincronización mediante primitivas atómicas	98
11.5.1 Operaciones atómicas fundamentales	98
11.6 E/S y DMA	99
11.6.1 Comparación de E/S programada y DMA	99
11.7 Síntesis del capítulo	100
11.8 Preguntas de control del capítulo 11	100
12 Talleres y prácticas sugeridas	101
12.1 Medición de llamadas al sistema y cambios de contexto	101
12.1.1 Objetivo	101
12.1.2 Código de referencia	101
12.1.3 Preguntas de análisis	104
12.2 Efecto del paso de acceso en la caché	104
12.2.1 Objetivo	104
12.2.2 Interpretación	105
12.3 Comparación escalar y vectorial	106
12.3.1 Objetivo	106
12.4 Escalamiento multinúcleo	108
12.4.1 Objetivo	108
12.5 Informe técnico	110
12.5.1 Estructura del informe	110
12.5.2 Ejemplo de tabla de resultados	110
13 Síntesis y cierre	111
13.1 Resumen integrador	111
13.1.1 El puente entre arquitectura y SO	111
13.1.2 Conceptos clave por capítulo	112
13.2 Errores típicos de los estudiantes	112
13.3 Ruta de profundización	112
13.3.1 Para estudiantes interesados en sistemas operativos	112
13.3.2 Para estudiantes interesados en arquitectura	113
13.3.3 Para estudiantes interesados en rendimiento	113
13.3.4 Recursos adicionales	113
13.4 Mensaje final	113
14 Referencias y recursos adicionales	115
14.1 Bibliografía	115
14.2 Recursos en línea adicionales	117

Índice de figuras

1.1	Mapa conceptual de la interacción hardware-SO	19
1.2	Traducción de direcciones virtuales a físicas	21
2.1	Estructura básica de la ALU	24
2.2	Ciclo básico de ejecución de instrucciones	25
2.3	Disposición simplificada del espacio virtual de un proceso de 32 bits	28
2.4	Comparación de latencias (escala logarítmica)	29
3.1	Arquitectura Von Neumann simplificada	32
3.2	Brecha creciente entre velocidad de CPU y memoria	34
3.3	Arquitectura Harvard con memorias y buses separados	34
3.4	Jerarquía de cachés en arquitectura Harvard modificada	35
4.1	Anillos de protección	41
4.2	IPC hipotético de distintas implementaciones x86-64	44
5.1	Ejecución de instrucción CISC mediante microcódigo	48
5.2	Flujo conceptual de una microarquitectura x86 moderna	50
6.1	Ejecución de un ciclo: cada instrucción ocupa un ciclo completo	54
6.2	Segmentación de cinco etapas: varias instrucciones en curso	54
6.3	Comparación de ciclos: ejecución no segmentada y segmentación ideal	55
6.4	Comparación de la ejecución sin reenvío y con reenvío	56
6.5	Máquina de estados de predicción de dos bits	58
7.1	Analogía de latencia y tasa de procesamiento en una autopista	62
7.2	Ley de Amdahl: aceleración según el número de procesadores	64
8.1	Segmentación escalar: como máximo una instrucción emitida por ciclo	70
8.2	Estructura conceptual de un procesador superescalar fuera de orden	70
8.3	Impacto de la tasa de predicción en el CPI	73
8.4	Estructura de un procesador fuera de orden	74
9.1	Operación SIMD: suma de vectores de cuatro elementos	79
10.1	Tendencias ilustrativas de frecuencia y número de núcleos	86
10.2	Ejemplo de jerarquía de cachés en un procesador de cuatro núcleos	86
10.3	Sistema NUMA de dos nodos con latencias ilustrativas	89
11.1	Flujo simplificado de una llamada al sistema	95
11.2	Arquitectura DMA	100
13.1	La cadena de abstracciones del computador	111

Índice de cuadros

1	Convenciones visuales utilizadas en la guía	13
2	Tiempo estimado por capítulo	14
1.1	Aspectos de memoria que el SO debe gestionar	18
1.2	Relación entre arquitectura y gestión de procesos	20
2.1	Operaciones típicas de una ALU	24
2.2	Tipos de registros en una arquitectura típica (x86-64)	24
2.3	Tipos de buses en un sistema	25
2.4	Ejemplos de amplitud de direcciones	27
2.5	Órdenes de magnitud de latencia y capacidad	28
3.1	Ventajas de la arquitectura Von Neumann	32
3.2	Comparación de las arquitecturas de Von Neumann y Harvard	35
3.3	Mecanismos de protección apoyados por el hardware y el SO	36
4.1	Componentes principales de una ISA	40
4.2	ISAs comunes en la actualidad	40
4.3	Modos de ejecución típicos en x86-64	41
4.4	Propiedades de la ISA y de la microarquitectura	42
5.1	Contexto histórico del diseño CISC	46
5.2	Características de arquitecturas CISC	46
5.3	Características de arquitecturas RISC	47
5.4	Comparación de longitud variable y fija	49
5.5	Implicaciones del diseño CISC/RISC para el SO	50
6.1	Características del diseño de un ciclo	54
6.2	Etapas típicas de una segmentación RISC de cinco etapas	55
6.3	Técnicas de predicción de saltos	58
7.1	Latencia y tasa de procesamiento en diferentes contextos	62
7.2	Factores que afectan el CPI	63
7.3	Métricas útiles para estudiantes	65
8.1	Características de procesadores escalares	70
8.2	Anchos relevantes en una microarquitectura superescalar	71
8.3	Técnicas para explotar ILP	72
8.4	Tipos de especulación en procesadores modernos	73
9.1	Evolución de extensiones SIMD en x86	78
9.2	Comparación de autovectorización y funciones intrínsecas	81
10.1	Estados del protocolo MESI	87
10.2	Modelos de consistencia de memoria	88

10.3 SMT: beneficios y limitaciones	90
11.1 Tipos de eventos y sus características	94
11.2 Anillos de protección y su uso	95
11.3 Fuentes de costo de un cambio de contexto	97
11.4 Estrategias de planificación en sistemas multinúcleo	98
11.5 Instrucciones atómicas comunes	99
11.6 Comparación de métodos de E/S	100
12.1 Formato de tabla de resultados	110
13.1 Resumen de conceptos clave por capítulo	112
13.2 Recursos adicionales recomendados	113

Capítulo 0

Guía de uso de este documento

0.1 Objetivo de la guía y alcance

Esta guía tiene como propósito establecer un puente conceptual entre la arquitectura de computadores y los sistemas operativos modernos. A diferencia de los textos tradicionales de arquitectura, que suelen centrarse en el diseño del hardware, este documento adopta una perspectiva orientada al software de sistema.

El objetivo principal es que el estudiante comprenda **cómo las decisiones de diseño del hardware afectan directamente** la implementación y el rendimiento de los sistemas operativos, desde la gestión de procesos hasta la administración de memoria virtual.

0.1.1 Contenido de la guía

El recorrido comienza con el modelo mínimo de un procesador y con la relación entre la CPU, la memoria y los dispositivos de entrada y salida. A partir de esa base se estudian la ISA, la microarquitectura y la jerarquía de memoria, siempre desde la pregunta que interesa en este curso: ¿qué necesita conocer el sistema operativo para administrar el computador?

Los capítulos posteriores introducen la segmentación, las métricas de rendimiento y tres formas de paralelismo: entre instrucciones, entre datos y entre hilos. El propósito no es describir cada detalle de un procesador comercial, sino construir un modelo que permita interpretar fenómenos como un cambio de contexto, un fallo de caché, una llamada al sistema o la pérdida de rendimiento de un programa paralelo.

0.1.2 Contenido no cubierto

Esta guía no pretende enseñar diseño de circuitos digitales, electrónica ni fabricación de semiconductores. Tampoco desarrolla programación avanzada en lenguaje ensamblador, arquitecturas especializadas como FPGA y ASIC, o sistemas embebidos en profundidad. Estos temas requieren cursos y textos especializados; aquí solo aparecerán cuando ayuden a explicar una decisión del sistema operativo.

0.2 Prerrequisitos mínimos

La exposición parte de conocimientos introductorios de representación de datos, programación en C y organización de la memoria. No es necesario dominar arquitectura de computadores, pero sí conviene reconocer las ideas que se describen a continuación.

0.2.1 Representación de datos

El estudiante debe poder interpretar números binarios, hexadecimales y decimales, así como reconocer la representación básica de enteros y números en punto flotante. También se utilizarán operaciones lógicas y aritméticas elementales para explicar instrucciones, máscaras y direcciones.

Si necesita repasar estos conceptos, revise el Apéndice A del libro “Computer Systems: A Programmer’s Perspective” de Bryant y O’Hallaron [4].

0.2.2 Programación en C

El lenguaje C sirve como referencia porque hace visibles muchos aspectos que otros lenguajes abstraen. Se presupone familiaridad con variables, funciones y estructuras de control, además de una comprensión inicial de punteros, aritmética de punteros y gestión dinámica de memoria mediante `malloc/free`. Las prácticas también requieren compilar con GCC y ejecutar comandos básicos desde una terminal; el uso de *Makefiles* es conveniente, aunque no indispensable para la lectura conceptual.

0.2.3 Noción de memoria

Cuando un programa accede a una variable, lo hace mediante una dirección. Por ello, conviene distinguir la pila (*stack*) de la memoria dinámica (*heap*) y tener una idea preliminar de la diferencia entre memoria virtual y memoria física. Estos conceptos se retomarán con mayor precisión cuando se estudie la cooperación entre la MMU y el sistema operativo.

0.3 Metodología de estudio

0.3.1 Estructura de cada capítulo

Cada capítulo abre con una idea guía que sitúa el problema antes de introducir la terminología. El desarrollo avanza desde una explicación intuitiva hasta los detalles técnicos y se apoya en diagramas, fragmentos de código y pequeños casos de estudio. Cuando un mecanismo de hardware tiene consecuencias para el sistema operativo, esa relación se hace explícita en el cuerpo de la explicación, no como una observación aislada. Las preguntas finales permiten comprobar si el estudiante puede explicar el concepto y aplicarlo en una situación nueva.

0.3.2 Convenciones visuales

Tabla 1: Convenciones visuales utilizadas en la guía

Elemento	Significado
Consejo	Recomendaciones de estudio y buenas prácticas
Advertencia	Errores comunes o conceptos que requieren atención
Nota	Información importante para recordar
Concepto clave	Definiciones fundamentales que debe dominar
código	Fragmentos de código fuente
término	Definición de un concepto nuevo

0.3.3 Metodología de estudio recomendada

Protocolo de estudio efectivo

1. **Lectura inicial:** Lea el capítulo completo sin detenerse en los detalles técnicos. Obtenga la “visión general”.
2. **Lectura profunda:** Vuelva a leer, esta vez tomando notas y verificando que entiende cada concepto antes de continuar.
3. **Experimentación:** Ejecute los ejemplos de código en su propia máquina. Modifique parámetros y observe el comportamiento.
4. **Prácticas:** Complete los ejercicios del Capítulo 12 relacionados con el tema.
5. **Autoevaluación:** Responda las preguntas de control del capítulo.
6. **Discusión:** Participe en foros o grupos de estudio para discutir conceptos difíciles.

0.3.4 Tiempo estimado de dedicación

Tabla 2: Tiempo estimado por capítulo

Capítulo	Lectura (horas)	Práctica (horas)
0. Guía de uso	0.5	–
1. Arquitectura y SO	1.0	0.5
2. Modelo de ejecución	2.0	1.0
3. Von Neumann y Harvard	1.5	0.5
4. ISA y microarquitectura	2.0	1.0
5. CISC y RISC	1.5	0.5
6. Segmentación	3.0	2.0
7. Métricas	2.0	1.5
8. Escalar/Superscalar	2.0	1.0
9. SIMD	2.5	2.0
10. Multinúcleo	3.0	2.5
11. Implicaciones para el SO	2.0	1.5
12. Talleres	–	8.0
13. Cierre	1.0	–
Total	24.0	22.5

Ruta esencial para la introducción al curso: capítulos 0–7 y las secciones sobre interrupciones, privilegios y llamadas al sistema del Capítulo 11.

Ruta de profundización: capítulos 8–10, talleres completos y temas avanzados del Capítulo 11. La estimación de 46,5 horas corresponde al recorrido integral, no a una sola unidad introductoria.

0.4 Software y herramientas recomendadas

Para seguir las prácticas de esta guía, necesitará:

0.4.1 Entorno Linux

```
# Actualizar repositorios
sudo apt update

# Compilador, depurador y utilidades básicas
sudo apt install build-essential gdb valgrind

# perf depende de la distribución y de la versión del kernel
sudo apt install linux-tools-common linux-tools-generic
sudo apt install linux-tools-$(uname -r)

# Herramientas de desarrollo
sudo apt install git vim nano

# Python en un entorno virtual
```

```
sudo apt install python3 python3-venv
python3 -m venv .venv
source .venv/bin/activate
python3 -m pip install matplotlib numpy pandas
```

Código 1: Instalación de herramientas en Ubuntu/Debian

0.4.2 Verificación del entorno

```
# Verificar GCC
gcc --version

# Verificar perf (Linux Performance Counters)
perf --version

# Verificar información del CPU
cat /proc/cpuinfo | grep "model name" | head -1
lscpu

# Verificar memoria disponible
free -h
```

Código 2: Verificación de la instalación

0.5 Preguntas de control del capítulo 0

1. ¿Cuál es el objetivo principal de esta guía y cómo difiere de un texto tradicional de arquitectura de computadores?
2. Enumere tres prerequisites mínimos que debe tener un estudiante antes de comenzar esta guía.
3. Describa las seis etapas del protocolo de estudio recomendado.
4. ¿Qué tiempo total estimado (lectura + práctica) requiere completar esta guía?
5. ¿Qué herramientas de software son indispensables para realizar las prácticas?

Capítulo 1

Fundamentos de arquitectura para sistemas operativos

Un sistema operativo administra los recursos del computador y presta servicios a los programas. Para hacerlo con eficiencia y seguridad, necesita comprender el hardware que controla.

— Adaptado de [27]

Al finalizar este capítulo, el estudiante podrá explicar por qué el sistema operativo necesita conocer las capacidades del procesador, relacionar la CPU, la memoria y la E/S dentro de un mismo flujo de ejecución, e identificar la base arquitectónica de los procesos, la memoria virtual y la sincronización.

Cuando una aplicación se ejecuta, parece disponer de un procesador y de una memoria propios. Esa impresión es una abstracción construida por el sistema operativo. Debajo de ella existen recursos compartidos, dispositivos que producen eventos y mecanismos de protección que impiden que un programa tome el control del computador. Comprender la arquitectura permite ver cómo se construye esa abstracción y cuáles son sus límites.

1.1 Información del hardware necesaria para el SO

Un sistema operativo actúa como intermediario entre las aplicaciones y el hardware, pero no es una capa pasiva. Configura el procesador, decide qué código puede ejecutarse y responde a los eventos que alteran el flujo normal. Para cumplir esas tareas debe conocer tanto las operaciones visibles de la arquitectura como ciertos detalles de la plataforma donde se ejecuta.

1.1.1 La CPU y sus capacidades

La CPU debe ofrecer, en primer lugar, al menos un modo restringido para las aplicaciones y otro privilegiado para el núcleo. El sistema operativo necesita saber cómo pasar de uno a otro, qué instrucciones están reservadas al modo privilegiado y qué registros de control modifican la gestión de memoria, las interrupciones o el estado general del procesador. También necesita un mecanismo por el cual una interrupción, una excepción o una llamada al sistema le transfiera el control de manera predecible [15, 27].

El **modo dual de operación** (usuario/núcleo) es fundamental para la protección y seguridad del sistema. El hardware debe proporcionar mecanismos para: ejecutar las aplicaciones con permisos restringidos, detectar cualquier intento de realizar una operación no autorizada y transferir el control al núcleo sin exponer su estado. Esta separación no evita por sí sola todos los errores, pero establece el límite mínimo de confianza entre una aplicación y el sistema.

1.1.2 La jerarquía de memoria

La memoria que observa una aplicación tampoco corresponde de forma directa con los módulos físicos instalados. El sistema operativo descubre qué regiones pueden utilizarse, crea espacios de direcciones virtuales y coordina con la MMU la traducción de cada acceso. Las cachés y la TLB aceleran esta ruta, mientras que DMA permite a ciertos dispositivos transferir datos sin exigir que la CPU copie cada byte. La Tabla 1.1 resume estas responsabilidades.

Tabla 1.1: Aspectos de memoria que el SO debe gestionar

Aspecto	Responsabilidad del SO
Memoria física	Detectar cantidad disponible, marcar regiones reservadas
Memoria virtual	Configurar tablas de páginas, manejar fallos de página
Cachés	Confiar en la coherencia del hardware y ejecutar mantenimiento explícito cuando la plataforma lo requiera
TLB	Construir tablas de páginas e invalidar traducciones obsoletas
DMA	Preparar búferes y mapeos sin que la CPU copie cada dato

La distribución exacta de estas tareas depende de la arquitectura: la MMU y la TLB realizan la traducción en hardware, mientras el núcleo mantiene las tablas que describen los mapeos y ordena las invalidaciones necesarias [21, 22, 27].

1.1.3 Dispositivos de E/S

Los dispositivos se presentan al sistema mediante controladoras y registros. Un controlador puede exponer memoria mapeada, puertos de E/S, interrupciones y capacidades de DMA. El núcleo identifica esos recursos durante el arranque y los administra a través de controladores de dispositivo. En Linux, los siguientes comandos permiten observar parte de esa información.

```
# Listar dispositivos PCI
lspci

# Listar dispositivos USB
lsusb

# Información de dispositivos de bloque
```

```
lsblk  
  
# Interrupciones actuales  
cat /proc/interrupts
```

Código 1.1: Visualización de dispositivos en Linux

1.2 Mapa conceptual del sistema

La Figura 1.1 muestra la interconexión de los componentes fundamentales desde la perspectiva del SO.

El diagrama no representa una conexión física exacta, sino las relaciones que interesan para razonar sobre la ejecución. La CPU obtiene instrucciones y datos de la memoria; las controladoras intercambian información con el sistema; y el núcleo recupera el control mediante eventos definidos por la arquitectura.

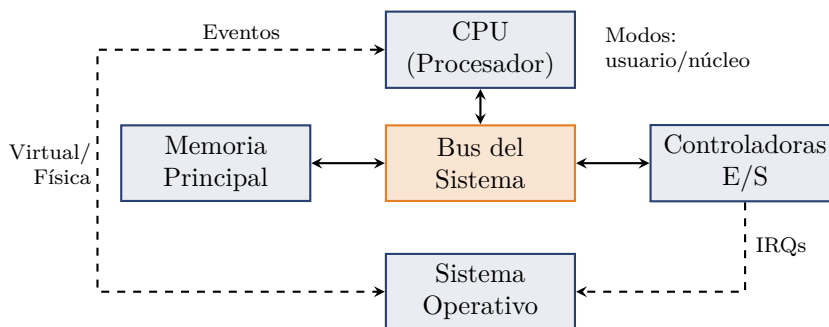


Figura 1.1: Mapa conceptual de la interacción hardware-SO

1.2.1 El flujo de control

Durante la ejecución ordinaria, la CPU procesa instrucciones de la aplicación en modo usuario. Si el programa necesita abrir un archivo, reservar una página o comunicarse con un dispositivo, invoca una llamada al sistema. Una instrucción especial provoca entonces una transición controlada: el procesador conserva el estado mínimo requerido, cambia al modo privilegiado y comienza a ejecutar un punto de entrada del núcleo. El sistema operativo valida la solicitud, realiza el servicio y prepara el valor de retorno. Finalmente, restaura el contexto necesario y la CPU continúa en modo usuario justo después de la llamada.

Este recorrido es breve en apariencia, pero concentra varias ideas fundamentales: protección, cambio de contexto, convenciones ABI y manejo de excepciones. En capítulos posteriores se estudiará cada una por separado.

1.3 Conexión directa con temas del curso

1.3.1 Procesos y planificación

Un proceso necesita algo más que memoria para conservar su identidad. El sistema operativo debe recordar el contador de programa, el puntero de pila, los registros y otros datos que

permiten suspender la ejecución y reanudarla más tarde. El temporizador de hardware evita que una aplicación conserve la CPU indefinidamente y ofrece al planificador oportunidades periódicas para elegir otro hilo. La Tabla 1.2 conecta estos mecanismos con las decisiones de planificación.

Tabla 1.2: Relación entre arquitectura y gestión de procesos

Tema SO	Concepto de arquitectura	Impacto
Cambio de contexto	Registros, PC, SP	Latencia del cambio
Cuanto de tiempo (<i>time slice</i>)	Temporizador de hardware	Granularidad de la planificación
Preempción	Temporizador e interrupciones	Oportunidades de planificación
Planificación multi-núcleo	Topología de cachés	Afinidad de procesos

```

1 // Estructura del contexto de un proceso (simplificado)
2 struct context {
3     uint64_t regs[32]; // Registros generales
4     uint64_t pc; // Program counter
5     uint64_t sp; // Stack pointer
6     uint64_t status; // Registro de estado
7     uint64_t page_table; // Identificador de las tablas de páginas
8     // ... más registros de control
9 };
10
11 // Cambio de contexto: guardar uno, restaurar otro
12 void context_switch(struct context *old, struct context *new) {
13     // 1. Guardar registros del proceso actual
14     save_registers(old);
15
16     // 2. Actualizar estructuras del SO
17     update_process_state();
18
19     // 3. Cambiar espacio de direcciones (MMU)
20     switch_page_table(new->page_table);
21
22     // 4. Restaurar registros del nuevo proceso
23     restore_registers(new);
24
25     // 5. Retornar (al nuevo proceso)
26 }

```

Código 1.2: Costo del cambio de contexto (ejemplo conceptual)

1.3.2 Memoria virtual

La memoria virtual separa las direcciones utilizadas por un proceso de las ubicaciones físicas donde residen sus datos. Esa separación permite aislar programas, compartir bibliotecas y mover páginas entre memoria y almacenamiento secundario sin cambiar el código de la aplicación.

La **memoria virtual** combina mecanismos de hardware, como la MMU, con la gestión del SO. Cada proceso observa un espacio de direcciones propio y aislado, aunque la memoria física sea compartida. [4, 27]

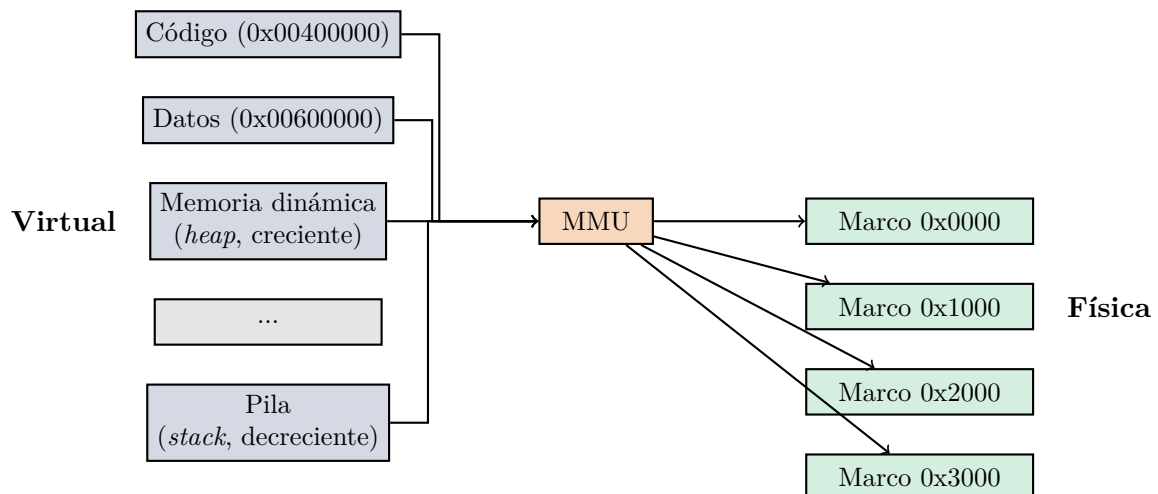


Figura 1.2: Traducción de direcciones virtuales a físicas

1.3.3 Concurrencia y sincronización

Cuando varios hilos acceden a una estructura compartida, una lectura y una escritura ordinarias pueden entrelazarse de forma incorrecta. Por eso los mecanismos de sincronización del SO se apoyan en operaciones atómicas proporcionadas por el hardware. El siguiente cerrojo ilustra la idea, aunque un núcleo real emplea primitivas más elaboradas para evitar esperas innecesarias.

```

1  #include <stdatomic.h>
2
3  typedef atomic_flag spinlock_t;
4
5  void spinlock_acquire(spinlock_t *lock) {
6      while (atomic_flag_test_and_set_explicit(
7          lock, memory_order_acquire)) {
8          // Espera activa. En x86 puede añadirse PAUSE.
9      }
10 }
11
12 void spinlock_release(spinlock_t *lock) {
13     atomic_flag_clear_explicit(lock, memory_order_release);
14 }

```

Código 1.3: Spinlock conceptual con operaciones atómicas de C11

Las instrucciones atómicas como CAS (Compare-And-Swap), TAS (Test-And-Set) y LL/SC (Load-Link/Store-Conditional) son primitivas fundamentales que el hardware debe proporcionar para implementar sincronización de alto nivel (mutexes, semáforos, etc.). [12, 23]

1.4 Síntesis del capítulo

El sistema operativo puede ofrecer procesos, memoria virtual y servicios de E/S porque la arquitectura proporciona mecanismos más elementales: modos de privilegio, registros de control, traducción de direcciones e interrupciones. El núcleo organiza esos mecanismos y los convierte en abstracciones que las aplicaciones pueden utilizar sin controlar directamente el hardware.

Esta relación también explica por qué el rendimiento de un sistema no depende solo del código del núcleo. Un cambio de contexto altera registros y puede afectar la localidad; una operación de E/S involucra controladores e interrupciones; y una primitiva de sincronización depende del modelo de memoria. Estudiar arquitectura, por tanto, permite razonar con mayor precisión sobre el comportamiento del sistema operativo.

1.5 Preguntas de control del capítulo 1

1. Enumere cinco aspectos de la CPU que un SO necesita conocer.
2. Explique el flujo de control cuando un programa en modo usuario solicita un servicio del SO.
3. ¿Qué es la MMU y qué papel juega en la implementación de memoria virtual?
4. ¿Por qué las instrucciones atómicas son fundamentales para la sincronización?
5. Dibuje un diagrama que muestre cómo se relacionan la CPU, la memoria, las controladoras de E/S y el SO.

Capítulo 2

Modelo mínimo de ejecución de un procesador

El procesador ejecuta instrucciones siguiendo reglas definidas por la arquitectura. Comprender esas reglas permite explicar qué puede hacer el sistema operativo y cuánto cuesta hacerlo.

Al finalizar este capítulo, el estudiante podrá identificar los componentes básicos de un procesador, describir el ciclo de búsqueda, decodificación y ejecución, y comparar el costo de acceder a registros, cachés y memoria principal.

Un procesador real contiene decodificadores, colas, predictores, múltiples unidades de ejecución y varios niveles de caché. Comenzar por todos esos detalles ocultaría la idea esencial. Para esta introducción basta con un modelo mínimo: el procesador mantiene un estado, obtiene una instrucción, interpreta su significado y transforma ese estado. El modelo es deliberadamente sencillo, pero permite seguir la ejecución de un programa y entender dónde interviene el sistema operativo.

2.1 Componentes básicos

Aunque su implementación sea compleja, el procesador puede describirse inicialmente mediante cuatro elementos: una unidad aritmético-lógica, un conjunto de registros, una unidad de control y caminos de comunicación. No son piezas completamente independientes; funcionan como partes de un ciclo en el que los datos se seleccionan, se transforman y se almacenan [24].

2.1.1 Unidad aritmético-lógica (ALU)

La ALU realiza transformaciones aritméticas y lógicas sobre los operandos que recibe. Una suma puede producir no solo un resultado, sino también indicadores como cero, acarreo o desbordamiento. Esos indicadores permiten tomar decisiones posteriores, por ejemplo en una instrucción de salto condicional. La Tabla 2.1 muestra operaciones representativas.

Tabla 2.1: Operaciones típicas de una ALU

Categoría	Operaciones	Ejemplos
Aritméticas	Suma, resta, multiplicación, división	ADD, SUB, MUL, DIV
Lógicas	AND, OR, NOT, XOR	AND, OR, NOT, XOR
Comparación	Igual, menor, mayor	CMP, TEST
Desplazamiento	Shift left/right, rotación	SHL, SHR, ROL

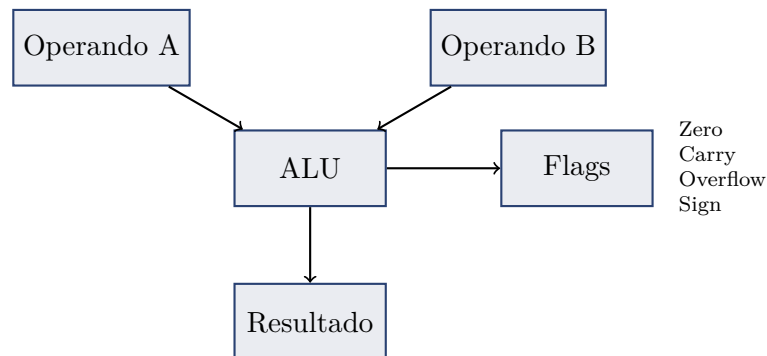


Figura 2.1: Estructura básica de la ALU

2.1.2 Registros

Los registros son pequeñas ubicaciones de almacenamiento dentro del procesador. Allí se mantienen los operandos inmediatos, las direcciones y el estado que guía la ejecución. La CPU puede acceder a ellos con una latencia muy baja, pero su número es limitado; por eso el compilador y el procesador deben decidir qué valores permanecen cerca de las unidades de ejecución y cuáles se envían a memoria.

Los **registros** almacenan operandos, direcciones y estados que la CPU necesita de inmediato. Muchas operaciones simples sobre registros tienen latencias cercanas a un ciclo, mientras que un acceso a memoria principal puede requerir cientos de ciclos.

Tabla 2.2: Tipos de registros en una arquitectura típica (x86-64)

Tipo	Ejemplos	Propósito
Propósito general	RAX, RBX, RCX, RDX	Datos temporales, argumentos
Índice	RSI, RDI	Punteros a cadenas o búferes
Puntero de pila	RSP	Tope de la pila (<i>stack</i>)
Puntero de base	RBP	Base del marco de activación actual
Contador de programa	RIP	Dirección de la siguiente instrucción
Indicadores	RFLAGS	Estado de operaciones (acarreo, cero, etc.)
Segmento	CS, DS, SS, ES	Segmentación de memoria
Control	CR0–CR8	Configuración del procesador

2.1.3 Unidad de control (CU)

La unidad de control coordina el recorrido de cada instrucción. Primero utiliza el contador de programa para localizarla; luego interpreta su codificación y selecciona los operandos. A continuación activa la unidad capaz de realizar la operación y, cuando existe un resultado, dirige su escritura al destino correspondiente. En procesadores modernos estas actividades se solapan y pueden distribuirse entre muchas etapas, pero el orden lógico sigue siendo útil para comprender la ejecución.

2.1.4 Buses

Los buses y las interconexiones transportan direcciones, datos y señales de control. La metáfora de una carretera resulta útil siempre que se recuerde que los procesadores actuales no utilizan un único camino uniforme: poseen redes internas y enlaces especializados. Para el modelo mínimo, sin embargo, basta distinguir qué información se transfiere, como resume la Tabla 2.3.

Tabla 2.3: Tipos de buses en un sistema

Bus	Función	Ancho típico
Datos	Transferir datos entre componentes	32, 64, 128 bits
Direcciones	Especificar ubicaciones de memoria	32, 64 bits
Control	Señales de coordinación	Variable

2.2 Ciclo de búsqueda–decodificación–ejecución

El ciclo de la Figura 2.2 describe, de forma simplificada, cómo el procesador obtiene y ejecuta una instrucción. Las etapas no deben interpretarse como compartimentos rígidos presentes en toda CPU, sino como operaciones lógicas que permiten seguir el cambio de estado.

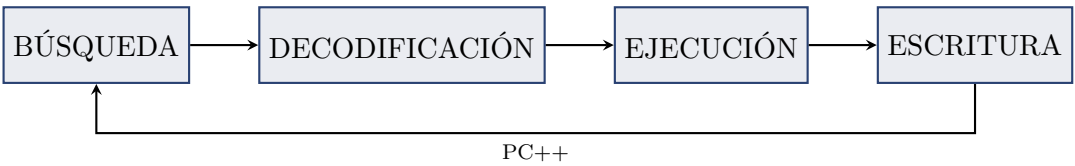


Figura 2.2: Ciclo básico de ejecución de instrucciones

2.2.1 Búsqueda (*fetch*)

La búsqueda comienza con el contador de programa, que contiene la dirección de la próxima instrucción. Esa dirección se presenta a la jerarquía de memoria y los bytes recuperados se colocan en un registro o una cola interna. Al mismo tiempo se calcula una dirección candidata para continuar. Un salto puede cambiarla más adelante, razón por la cual los procesadores modernos intentan predecir el flujo antes de conocerlo con certeza.

```
1 ; El PC (Program Counter) apunta a la siguiente instrucción
2 ; Supongamos PC = 0x1000
3
4 ; En memoria en 0x1000:
5 ; 0x1000: 0x48 0x89 0xC3 (mov %rax, %rbx en x86-64)
```

```

6
7 ; La CU:
8 ; 1. Coloca 0x1000 en el bus de direcciones
9 ; 2. Activa señal de lectura de memoria
10 ; 3. Espera que los datos lleguen por el bus de datos
11 ; 4. Incrementa PC para apuntar a la siguiente instrucción

```

Código 2.1: Ejemplo de fetch (conceptual)

2.2.2 Decodificación (*decode*)

Los bytes obtenidos no expresan directamente una frase como “mover RAX a RBX”. Su significado depende de la ISA y de la posición de campos como el código de operación, los registros y el modo de direccionamiento. Decodificar consiste en reconocer esos campos y preparar la operación interna correspondiente.

```

1 ; Instrucción binaria: 0x48 0x89 0xC3
2 ;
3 ; 0x48 = REX prefix (operando de 64 bits, registro extendido)
4 ; 0x89 = Opcode MOV r/m64, r64
5 ; 0xC3 = ModR/M byte (mod=11, reg=000, r/m=011)
6 ;
7 ; Resultado de decodificación:
8 ; - Operación: MOV
9 ; - Fuente: RAX (registro 000)
10 ; - Destino: RBX (registro 011)
11 ; - Tamaño: 64 bits

```

Código 2.2: Decodificación de instrucción

2.2.3 Ejecución (*execute*)

Durante la ejecución, la unidad seleccionada recibe los operandos y produce el resultado. Una instrucción aritmética utiliza la ALU; una carga calcula una dirección y consulta la memoria; un salto puede modificar el contador de programa. Si la instrucción produce un valor arquitectónico, este debe quedar visible en el registro o la ubicación definidos por la ISA.

```

1 // Pseudocódigo de la ejecución
2 void execute_mov(Register dest, Register src) {
3     // Leer valor del registro fuente
4     uint64_t value = registers[src];
5
6     // Escribir en registro destino
7     registers[dest] = value;
8
9     // Actualizar flags si es necesario
10    // (MOV no afecta flags en x86)
11 }

```

Código 2.3: Ejecución de instrucción MOV

2.3 Instrucciones, registros y memoria en el contrato CPU–RAM

El **contrato CPU-RAM** define cómo se comunican el procesador y la memoria principal. Este contrato incluye: el espacio de direcciones disponible, el tamaño de los operandos, la semántica de lectura y escritura y las reglas que determinan el orden y la visibilidad de los accesos. Entre la CPU y la DRAM existen cachés, traducciones y controladores, de modo que el “contrato” describe el comportamiento observable y no necesariamente un intercambio físico directo.

2.3.1 Espacio de direcciones

Una dirección es un identificador numérico de una ubicación dentro de un espacio. El número de bits de la ISA fija un límite teórico, pero un procesador puede implementar menos bits virtuales o físicos. Además, las direcciones que genera una aplicación suelen ser virtuales y deben traducirse antes de acceder a la memoria principal. Por ello, afirmar que una CPU es “de 64 bits” no significa que cada sistema instalado pueda utilizar los 2^{64} bytes.

Tabla 2.4: Ejemplos de amplitud de direcciones

Caso	Bits implementados	Espacio direccionable
Arquitectura de 32 bits	32 virtuales	2^{32} bytes = 4 GiB
x86-64, espacio virtual común	48 o 57 virtuales	256 TiB o 128 PiB
x86-64, dirección física	Hasta 52 físicos	Hasta 4 PiB
Arquitectura de 64 bits, límite teórico	64	16 EiB

Los límites de 48 o 57 bits virtuales y de hasta 52 bits físicos corresponden a capacidades arquitectónicas de x86-64; cada procesador puede implementar un subconjunto y debe consultarse el modelo concreto [15].

No deben confundirse las direcciones **virtuales** que observa un proceso con las direcciones **físicas** que llegan a la memoria. Una ISA de 64 bits puede implementar subconjuntos diferentes para cada caso.

2.3.2 Modelo de memoria del programador

Desde la perspectiva de un proceso, el espacio virtual se organiza en regiones con propósitos distintos. El código ejecutable, los datos globales, la memoria dinámica y la pila ocupan intervalos que el enlazador, el cargador y el sistema operativo coordinan. La Figura 2.3 muestra una disposición simplificada; las direcciones concretas cambian entre sistemas y ejecuciones.

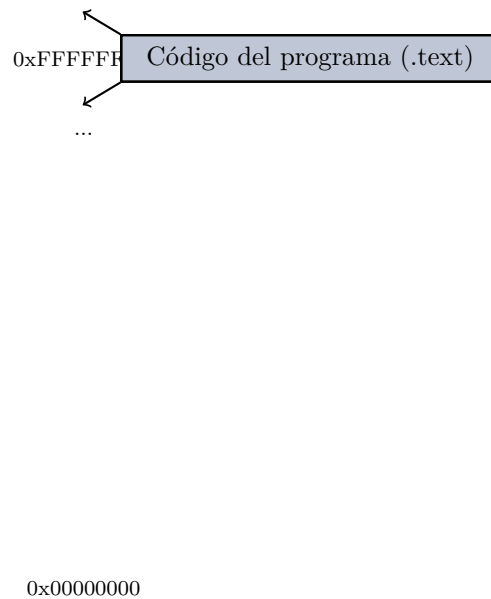


Figura 2.3: Disposición simplificada del espacio virtual de un proceso de 32 bits

2.4 Costos de acceso a memoria y cómputo

La **jerarquía de memoria** [9] existe por la gran diferencia de velocidad entre el procesador y la memoria principal. Esta limitación suele denominarse *memory wall*.

2.4.1 Latencias aproximadas

Acercar toda la memoria a la velocidad de los registros sería demasiado costoso. La jerarquía combina niveles pequeños y rápidos con otros más grandes y lentos. Cuando un dato está en la caché L1, la CPU puede recuperarlo en pocos ciclos; si debe llegar hasta DRAM, muchas instrucciones posteriores pueden quedar esperando. La Tabla 2.5 compara órdenes de magnitud y no valores universales.

Tabla 2.5: Órdenes de magnitud de latencia y capacidad

Nivel	Latencia	Tamaño típico
Registro	~1 ciclo	8 bytes (escalar); hasta 64 bytes (vectorial)
Caché L1	3–4 ciclos (~1 ns)	32–64 KB
Caché L2	10–12 ciclos (~4 ns)	256 KB–1 MB
Caché L3	30–40 ciclos (~12 ns)	8–64 MB (compartida)
Memoria DRAM	200–300 ciclos (~80 ns)	8–512 GB
SSD (NVMe)	10^5 – 10^6 ciclos (~25 μ s)	256 GB–8 TB
Disco HDD	10^7 – 10^8 ciclos (~10 ms)	1–20 TB

Estos valores son aproximaciones para comparar órdenes de magnitud. Cambian según la microarquitectura, la frecuencia, el patrón de acceso y el dispositivo. Para cifras de un procesador concreto deben consultarse sus manuales y realizarse mediciones controladas [8, 14].

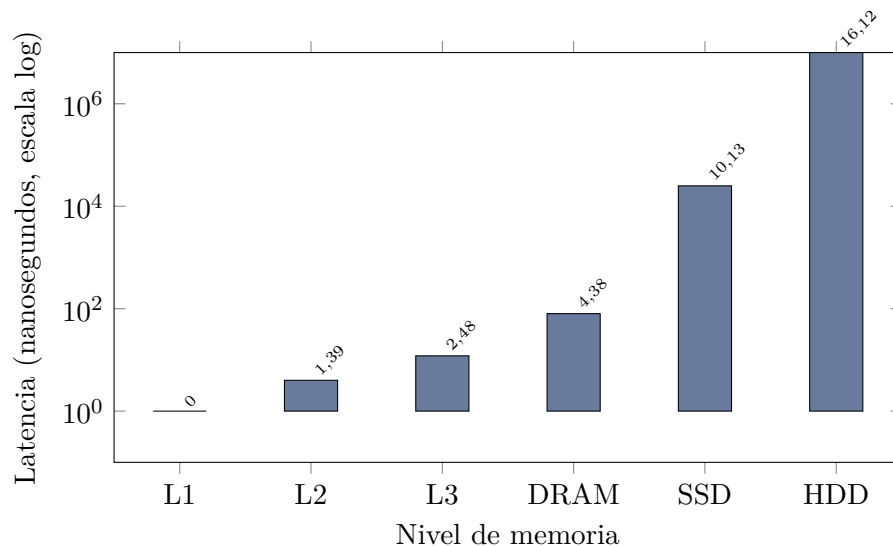


Figura 2.4: Comparación de latencias (escala logarítmica)

2.4.2 Implicaciones para el programador de sistemas

Principio de localidad. Los programas tienden a reutilizar datos recientes y a recorrer ubicaciones próximas. La primera tendencia se denomina localidad temporal; la segunda, localidad espacial. Las cachés aprovechan ambas propiedades al conservar información usada recientemente y transferir bloques completos, no bytes aislados.

La localidad conecta una propiedad del programa con una decisión de hardware. Dos algoritmos que realizan el mismo número de sumas pueden tardar tiempos muy distintos si uno recorre memoria contigua y el otro salta entre posiciones impredecibles. El siguiente ejemplo hace visible esa diferencia.

```

1 // ACCESO SECUENCIAL (buena localidad espacial)
2 // Los elementos se acceden en orden contiguo
3 int suma_secuencial(int *arr, int n) {
4     int sum = 0;
5     for (int i = 0; i < n; i++) {
6         sum += arr[i]; // Acceso a arr[0], arr[1], arr[2]...
7     }
8     return sum;
9 }
10
11 // ACCESO ALEATORIO (mala localidad)
12 // Saltos impredecibles en memoria
13 int suma_aleatoria(int *arr, int *indices, int n) {
14     int sum = 0;

```

```
15  for (int i = 0; i < n; i++) {  
16      sum += arr[indices[i]]; // Acceso impredecible  
17  }  
18  return sum;  
19 }
```

Código 2.4: Ejemplo de buen uso de localidad

2.5 Síntesis del capítulo

El modelo mínimo presentado en este capítulo describe la CPU como una máquina que conserva estado y ejecuta transformaciones definidas por instrucciones. La ALU realiza operaciones, los registros mantienen valores inmediatos y la unidad de control coordina la búsqueda, la decodificación, la ejecución y la escritura de resultados. Este modelo será suficiente para introducir segmentación, interrupciones y cambios de contexto.

La ejecución no ocurre aislada de la memoria. El espacio de direcciones y las reglas de acceso forman parte del contrato visible, mientras que la jerarquía de cachés intenta ocultar la diferencia entre la velocidad de la CPU y la DRAM. Cuando un programa exhibe localidad temporal y espacial, esa jerarquía funciona bien; cuando no lo hace, el acceso a datos puede dominar el tiempo total.

2.6 Preguntas de control del capítulo 2

1. Describa las cuatro etapas del ciclo de búsqueda, decodificación, ejecución y escritura.
2. ¿Qué tipos de registros existen y cuál es la función de cada uno?
3. Explique el “contrato” entre CPU y memoria RAM.
4. Compare las latencias de acceso a L1, L2, L3 y DRAM. ¿Por qué existen estas diferencias?
5. Dé un ejemplo de código con buena localidad espacial y otro con mala localidad.

Capítulo 3

Organización de memoria en las arquitecturas de Von Neumann y Harvard

Separar o compartir los caminos de instrucciones y datos cambia el rendimiento, la complejidad y las decisiones que debe tomar el sistema operativo.

Al finalizar este capítulo, el estudiante podrá diferenciar las organizaciones Von Neumann y Harvard, explicar la limitación asociada con una ruta compartida y reconocer la organización Harvard modificada de muchos procesadores modernos.

Un programa necesita dos clases de información: las instrucciones que indican qué hacer y los datos sobre los que se opera. La forma de almacenar y transportar ambas clases influye en el ancho de banda disponible, la complejidad del hardware y los mecanismos de protección. Las denominaciones Von Neumann y Harvard describen dos modelos de organización útiles para entender esas decisiones. Los procesadores actuales rara vez corresponden por completo con uno solo de ellos.

3.1 Arquitectura de Von Neumann

3.1.1 La idea fundamental

La organización asociada con el modelo de programa almacenado reúne instrucciones y datos en una memoria direccionable común. Esta idea, descrita en los primeros trabajos sobre computadores electrónicos y conocida como arquitectura Von Neumann, permite cargar un programa como información y cambiarlo sin reconstruir la máquina [24, 29].

En el modelo Von Neumann, instrucciones y datos comparten un espacio de memoria y una ruta de comunicación. El contador de programa selecciona la instrucción siguiente y los saltos modifican esa secuencia. Como el programa también está almacenado, el sistema puede cargar distintos ejecutables e incluso generar código, siempre que respete las políticas de protección.

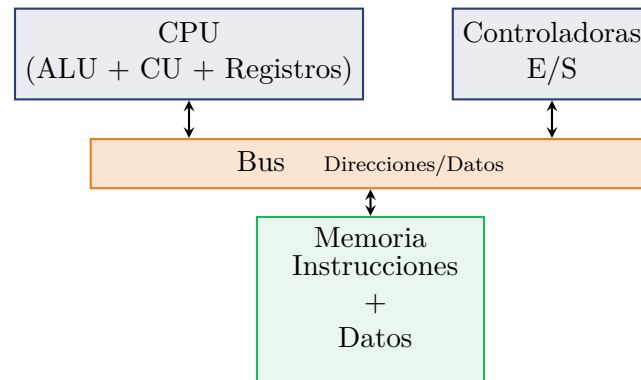


Figura 3.1: Arquitectura Von Neumann simplificada

3.1.2 Ventajas de Von Neumann

La unificación simplifica el modelo visible para el software: una dirección identifica una posición y la memoria puede distribuirse entre código y datos según las necesidades del programa. Históricamente, compartir los caminos también redujo componentes y costos. La Tabla 3.1 resume estas ventajas, aunque su importancia relativa ha cambiado con la tecnología.

Tabla 3.1: Ventajas de la arquitectura Von Neumann

Ventaja	Descripción
Simplicidad	Un solo bus simplifica el diseño del hardware
Flexibilidad	Los programas pueden modificarse a sí mismos
Eficiencia de memoria	No hay memoria desperdiciada si hay más datos que código
Costo	Menor costo al requerir menos componentes
Programabilidad	Fácil cargar y ejecutar diferentes programas

3.1.3 Limitaciones de Von Neumann

Una ruta compartida obliga a competir por ancho de banda. Si la CPU necesita obtener una instrucción y leer un dato al mismo tiempo, alguno de los accesos debe esperar o resolverse mediante una memoria intermedia. En un diseño básico, esa contención limita la velocidad a la que pueden alimentarse las unidades de ejecución.

El problema se conoce como *cuello de botella de Von Neumann*: el rendimiento deja de estar determinado únicamente por la rapidez de cálculo y pasa a depender de la capacidad para transportar instrucciones y datos. La unificación también obliga a introducir permisos explícitos. Si una región de datos pudiera ejecutarse sin restricciones, un error de memoria podría convertirse en ejecución de código no autorizado. Los sistemas modernos reducen ese riesgo mediante permisos de página, como escritura y ejecución.

3.2 El cuello de botella de Von Neumann

El **cuello de botella de Von Neumann** es la limitación de rendimiento causada por el bus único compartido entre CPU y memoria. El procesador no puede acceder simultáneamente a instrucciones y datos.

3.2.1 Análisis del problema

Considérese un bucle que acumula los elementos de un arreglo. Cada iteración requiere obtener instrucciones, leer un elemento y actualizar un acumulador. En el modelo más sencillo, todos esos accesos compiten por el mismo camino. El siguiente fragmento no pretende representar los ciclos de una CPU actual; hace visible la contención que motiva el concepto.

```
1  // Supongamos este código simple:
2  int sum = 0;
3  for (int i = 0; i < N; i++) {
4      sum += array[i]; // En cada iteración:
5  }
6
7  // En arquitectura de Von Neumann básica:
8  // Ciclo 1-3: Fetch instrucción (LOAD)
9  // Ciclo 4-6: Fetch instrucción (ADD)
10 // Ciclo 7-9: Fetch instrucción (STORE)
11 // En cada instrucción que accede memoria:
12 // - Se envía dirección por el bus
13 // - Se espera respuesta (latencia DRAM)
14 // - No se puede fetch la siguiente instrucción simultáneamente
```

Código 3.1: Ejemplo conceptual de la contención del bus

3.2.2 Impacto en el rendimiento

La limitación se vuelve más visible cuando el procesador mejora a mayor ritmo que la latencia de la memoria. Una CPU capaz de iniciar muchas operaciones por ciclo no obtiene beneficio si pasa la mayor parte del tiempo esperando datos. La Figura 3.2 muestra una tendencia conceptual; no constituye una serie histórica para comparar productos concretos.

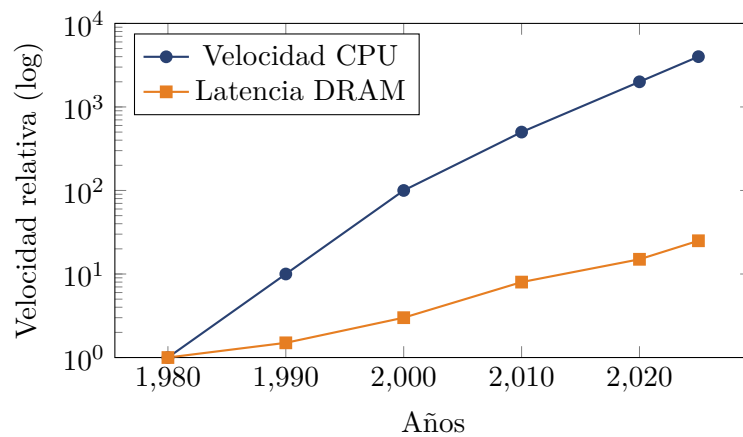


Figura 3.2: Brecha creciente entre velocidad de CPU y memoria

3.3 Arquitectura Harvard

3.3.1 La idea de separación

El modelo Harvard propone una separación física entre la memoria de instrucciones y la memoria de datos. Su nombre procede de máquinas tempranas en las que el programa y los datos utilizaban medios diferentes; aquí interesa como modelo de organización, no como descripción de un único computador histórico [24].

En una organización Harvard, las instrucciones y los datos residen en memorias separadas y utilizan caminos independientes. El procesador puede, por tanto, obtener una instrucción mientras realiza un acceso a datos. Esta simultaneidad aumenta el ancho de banda disponible, aunque exige recursos adicionales y puede complicar la transferencia de información entre ambos espacios.

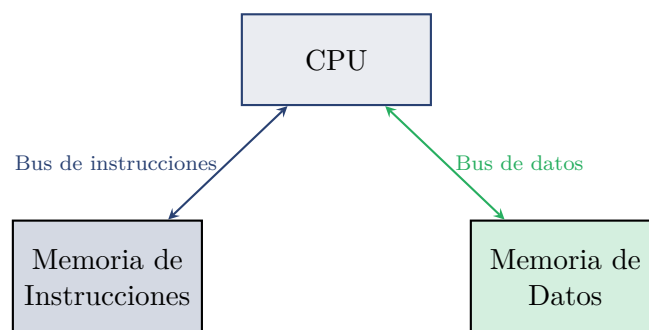


Figura 3.3: Arquitectura Harvard con memorias y buses separados

3.3.2 Ventajas de Harvard

La separación permite adaptar cada memoria a su propósito. Las instrucciones pueden tener un ancho o una política de acceso diferentes de los datos, y los dos caminos pueden trabajar en paralelo. A cambio, el espacio asignado a una clase no se reutiliza siempre con facilidad para la

otra. La Tabla 3.2 presenta la comparación como un modelo pedagógico; una implementación real puede combinar propiedades de ambas columnas.

Tabla 3.2: Comparación de las arquitecturas de Von Neumann y Harvard

Característica	Von Neumann	Harvard
Memoria	Unificada	Separada
Buses	Compartido	Independientes
Acceso simultáneo	No	Sí
Ancho de banda	Limitado	Doble (teórico)
Seguridad	Menor	Mayor
Complejidad	Menor	Mayor
Costo	Menor	Mayor

3.4 Harvard modificada

La **arquitectura Harvard modificada** combina una separación interna de cachés para mejorar el rendimiento con un espacio de direcciones unificado para el software. [24]

3.4.1 Implementación en CPU modernas

Muchos procesadores modernos adoptan una solución intermedia. Cerca del núcleo mantienen una caché L1 de instrucciones y otra de datos, lo cual permite alimentar simultáneamente la búsqueda y las operaciones de carga o almacenamiento. Los niveles inferiores suelen ser unificados y desembocan en una memoria principal común. Para el software existe un único espacio de direcciones, pero dentro de la jerarquía aparecen caminos separados.

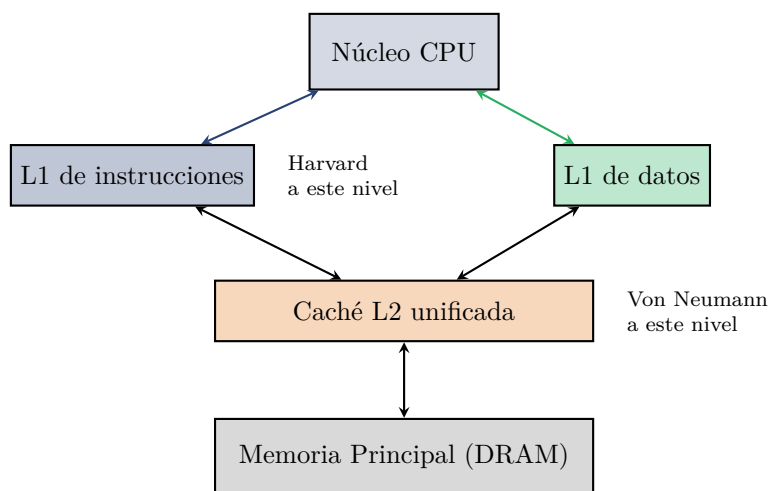


Figura 3.4: Jerarquía de cachés en arquitectura Harvard modificada

3.4.2 Características clave

La separación de L1 favorece accesos simultáneos, mientras que la unificación de L2 o L3 permite utilizar la capacidad según cambie la proporción entre código y datos. Esta combinación requiere mecanismos de coherencia. Si el sistema modifica una región que después se ejecutará —como ocurre en un compilador JIT—, debe asegurarse de que la caché de instrucciones observe la versión actualizada. La operación concreta depende de la arquitectura y del sistema operativo [2, 15, 21].

3.5 Implicaciones para el SO

3.5.1 Rendimiento y cachés

Para el sistema operativo, la distinción importa en tareas como cargar un ejecutable, modificar permisos o sincronizar código generado dinámicamente. Separar las páginas de código y datos también permite aplicar permisos diferentes y favorece patrones de acceso más previsibles.

```

1 // El SO coordina permisos y mantenimiento de código:
2
3 // 1. Separar código y datos en páginas diferentes
4 // para aplicar permisos de escritura y ejecución
5
6 // 2. Sincronizar el código generado o modificado
7 // antes de permitir que el procesador lo ejecute
8
9 // 3. Solicitar mantenimiento de la caché de instrucciones
10 // cuando la ISA y la plataforma lo requieran
11
12 // La precarga y la predicción de saltos son funciones
13 // principalmente microarquitectónicas, no del SO.
```

Código 3.2: Impacto de la separación de cachés

3.5.2 Protección y seguridad

Tabla 3.3: Mecanismos de protección apoyados por el hardware y el SO

Mecanismo	Descripción	Uso en SO
Bit NX	Páginas de datos no ejecutables	Dificultar la ejecución de datos
WAX	Escritura o ejecución, no ambas	Protección de código
ASLR	Randomización de direcciones	Dificultar exploits
DEP	Data Execution Prevention	Seguridad de memoria

Los sistemas operativos modernos utilizan el **bit NX** (*no execute*) para marcar como no ejecutables ciertas páginas de datos. Esto dificulta ataques que intentan ejecutar código inyectado en la pila o en la memoria dinámica. [15, 27]

3.6 Síntesis del capítulo

Los modelos Von Neumann y Harvard permiten razonar sobre una decisión básica: compartir o separar el almacenamiento y los caminos de instrucciones y datos. La unificación facilita la programación y el uso flexible de la memoria, pero crea contención; la separación habilita accesos simultáneos, a costa de mayor complejidad.

La mayoría de los procesadores de propósito general utiliza una organización Harvard modificada: cachés L1 separadas y niveles posteriores unificados. El sistema operativo debe conocer esta organización cuando gestiona permisos, mantiene la coherencia de código generado o interpreta el rendimiento de una carga. No se trata de elegir una etiqueta para la máquina, sino de entender dónde se comparten los recursos.

3.7 Preguntas de control del capítulo 3

1. Explique el principio fundamental de la arquitectura Von Neumann.
2. ¿Qué es el cuello de botella de Von Neumann y cómo afecta el rendimiento?
3. Compare las ventajas y desventajas de las organizaciones Von Neumann y Harvard.
4. ¿Qué es la arquitectura Harvard modificada y por qué se usa en CPU modernas?
5. Explique cómo el SO utiliza el bit NX para seguridad.

Capítulo 4

Diferencias entre ISA y microarquitectura

La ISA define lo que el software puede pedir; la microarquitectura determina cómo el procesador responde a esa petición.

Al finalizar este capítulo, el estudiante podrá distinguir la ISA de su implementación microarquitectónica, explicar el papel de la ABI en la ejecución de programas y clasificar qué propiedades son visibles para el software.

Dos computadores pueden ejecutar el mismo programa binario y, sin embargo, tener organizaciones internas y rendimientos muy diferentes. Esta posibilidad existe porque el software depende de un contrato estable, no de cada circuito interno. Para comprender ese contrato conviene separar tres niveles: la ISA define las operaciones de la máquina; la ABI establece convenciones para que los programas binarios cooperen con el sistema; y la microarquitectura decide cómo implementar esas operaciones.

4.1 Definición de la ISA y su relación con el SO

La **arquitectura del conjunto de instrucciones** (ISA, *Instruction Set Architecture*) es la interfaz abstracta entre el hardware y el software de bajo nivel [9, 15, 26]. Define las instrucciones y su codificación, los registros visibles, los tipos de datos, los modos de direccionamiento y las reglas observables de memoria, excepciones y privilegios. Una ISA no indica necesariamente cuántas etapas tiene la segmentación ni el tamaño de las cachés.

4.1.1 Componentes de una ISA

La ISA debe ser suficientemente precisa para que un compilador genere código y para que un sistema operativo controle la máquina. Si una división por cero produce una excepción, por ejemplo, la arquitectura define qué estado queda visible y dónde continúa el manejo. Del mismo modo, las instrucciones privilegiadas delimitan qué operaciones están reservadas al núcleo. La Tabla 4.1 organiza estas obligaciones.

Tabla 4.1: Componentes principales de una ISA

Componente	Descripción
Instrucciones	Operaciones que el procesador puede ejecutar
Registros	Almacenamiento de propósito general y especial
Modos de direccionamiento	Formas de especificar operandos
Tipos de datos	Enteros, punto flotante, vectores, etc.
Modelo de memoria	Organización y acceso a memoria
Manejo de excepciones	Respuesta a eventos especiales
Modos de operación	Usuario, supervisor, hypervisor

4.1.2 ISA principales

Una misma idea puede expresarse mediante codificaciones distintas. x86-64, Arm64 y RISC-V poseen repertorios de registros, formatos de instrucción y modelos de privilegio propios; por eso un ejecutable compilado para una de ellas no funciona directamente en otra. La Tabla 4.2 muestra usos frecuentes, no fronteras exclusivas.

Tabla 4.2: ISAs comunes en la actualidad

ISA	Tipo	Uso principal	Ejemplos
x86-64	CISC	PCs, servidores	Intel Core, AMD Ryzen
ARM64	RISC	Móviles, servidores	Apple M-series, Snapdragon
RISC-V	RISC	Sistemas embebidos, investigación	SiFive, núcleos propios
MIPS	RISC	Sistemas embebidos, routers	Cavium, MediaTek
Power	RISC	Servidores, mainframes	IBM Power

4.2 Convenciones ABI y modos de ejecución

4.2.1 Interfaz binaria de aplicaciones

La **ABI** (*Application Binary Interface*, interfaz binaria de aplicaciones) define las convenciones que permiten combinar objetos compilados y comunicarlos con el sistema operativo. Especifica, entre otros aspectos, el tamaño y la alineación de los tipos, el uso de registros para argumentos y retornos, la organización de la pila y el formato de los objetos ejecutables [30].

La ISA permite ejecutar una instrucción de suma, pero no determina por sí sola en qué registro debe recibir una función su primer argumento. Esa decisión pertenece a la ABI. Gracias a una convención compartida, una función compilada por separado puede llamar a otra sin conocer su código fuente, y el cargador puede preparar un proceso de manera compatible con las bibliotecas.

La convención de llamada entre funciones y la convención utilizada para entrar al núcleo son contratos relacionados, pero distintos. En Linux x86-64, por ejemplo, la instrucción `syscall` sigue una ABI específica para el número de servicio, los argumentos y el retorno [19, 30].


```
1 // En Linux x86-64, los primeros 6 argumentos enteros se pasan en:
2 // RDI, RSI, RDX, RCX, R8, R9
3 // El valor de retorno va en RAX
4
5 // Ejemplo: int funcion(int a, int b, int c, int d, int e, int f)
6 // (El identificador se escribe sin tilde por portabilidad.)
7 // a -> RDI
8 // b -> RSI
9 // c -> RDX
10 // d -> RCX
11 // e -> R8
12 // f -> R9
13 // Retorno en RAX
14
15 // Esta ABI interviene en:
16 // 1. Configurar correctamente la pila inicial
17 // 2. Enlazar funciones y bibliotecas
18 // 3. Construir y restaurar marcos de señales
19 // Las llamadas al sistema usan una convención específica.
```

Código 4.1: Convención de llamada System V AMD64 ABI

4.2.2 Modos de ejecución

La ISA también define niveles de privilegio y transiciones controladas. En x86-64 existen cuatro anillos, aunque los sistemas operativos de propósito general suelen concentrar el núcleo en el anillo 0 y las aplicaciones en el anillo 3. Los modos históricos de la Tabla 4.3 describen estados de la arquitectura x86 y no deben confundirse con esos niveles de privilegio [15].

Tabla 4.3: Modos de ejecución típicos en x86-64

Modo	Anillo	Privilegios	Uso
Real	–	Total	Modo heredado (arranque)
Protegido	0–3	Variable	Sistema operativo
Long Mode	0–3	Variable	x86-64 nativo
Virtual 8086	–	Restringido	Emulación DOS

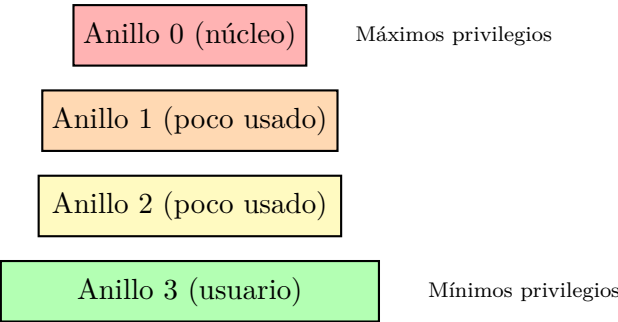


Figura 4.1: Anillos de protección

4.3 Implementación microarquitectónica de la ISA

La **microarquitectura** es la implementación concreta de una ISA. Dos procesadores pueden ejecutar el mismo conjunto de instrucciones mediante organizaciones internas diferentes y, por ello, ofrecer rendimientos distintos.

Para mantener el comportamiento exigido por la ISA, una CPU puede transformar una instrucción en varias microoperaciones, renombrar registros o ejecutar operaciones en un orden interno diferente. Estos mecanismos no cambian el resultado arquitectónico permitido, pero sí modifican la latencia, el consumo y la cantidad de trabajo completado por ciclo.

4.3.1 Elementos de la microarquitectura

Tabla 4.4: Propiedades de la ISA y de la microarquitectura

Aspecto	ISA: contrato visible	Microarquitectura: implementación
Instrucciones	Operaciones y codificación	Decodificación y microoperaciones
Registros	Registros arquitectónicos	Registros físicos y renombrado
Modelo de memoria	Orden observable permitido	Mecanismos internos de ordenamiento
Segmentación	No define su profundidad	Etapas, ancho y latencias
Cachés	Generalmente no define tamaños	Niveles, tamaños y políticas
Unidades funcionales	No define su cantidad	Número, capacidad y latencia

4.3.2 Ejemplos de implementaciones x86-64

La compatibilidad binaria explica por qué familias separadas por muchos años pueden ejecutar el mismo software básico. Sin embargo, cada generación cambia el ancho de decodificación, la predicción, las cachés y las unidades funcionales. Por ello, conocer la ISA permite decidir si un programa es compatible; conocer la microarquitectura ayuda a explicar cómo se comportará [9, 15].

Intel Core, AMD Zen y otras familias implementan x86-64 sin compartir la misma organización interna. Incluso dentro de una familia comercial cambian la cantidad de núcleos y las extensiones disponibles. Ninguna de esas propiedades forma parte de la definición mínima de x86-64, por lo que debe consultarse el modelo concreto antes de seleccionar una ruta optimizada.

4.4 Elementos visibles para el software y dependientes de la implementación

4.4.1 Visible para el software (ISA)

El programador de bajo nivel puede nombrar registros arquitectónicos, elegir instrucciones y utilizar modos de direccionamiento definidos. El sistema operativo, además, emplea instrucciones privilegiadas para configurar la memoria y responder a eventos. El siguiente fragmento muestra elementos que forman parte de ese contrato visible.

```
1 // Registros accesibles
2 mov %rax, %rbx // Registros explícitos
3 push %rbp // Manipulación de la pila
4
5 // Instrucciones explícitas
6 add $5, %rax // Suma
7 mul %rbx // Multiplicación
8
9 // Modos de direccionamiento
10 mov 8(%rsp), %rax // Direccionamiento con desplazamiento
11 mov (%rdi,%rsi,4), %eax // Direccionamiento indexado
12
13 // Llamadas al sistema
14 syscall // Transición controlada al núcleo
```

Código 4.2: Elementos visibles al programador

4.4.2 No visible directamente (microarquitectura)

En cambio, el binario no especifica qué predictor de saltos debe utilizarse ni cuántos registros físicos existen. El procesador puede cambiar esas decisiones entre modelos siempre que mantenga el comportamiento arquitectónico. Aunque el software no controle directamente estos mecanismos, sí percibe sus efectos en el tiempo de ejecución.

```
1 // El programador NO ve directamente:
2
3 // 1. Pipeline y sus etapas
4 // - Búsqueda, decodificación, ejecución y escritura
5 // - Todas ocurren "detrás" de cada instrucción
6
7 // 2. Cachés y su operación
8 // - Cachés L1, L2 y L3
9 // - Políticas de reemplazo y escritura
10
11 // 3. Ejecución fuera de orden
12 // - Las instrucciones pueden ejecutarse en orden diferente
13 // - El hardware preserva el comportamiento permitido por la ISA
14
15 // 4. Renombramiento de registros
16 // - Más registros físicos que arquitectónicos
17 // - Resuelve dependencias WAR/WAW
18
19 // 5. Predicción de saltos
20 // - El hardware predice el resultado de los saltos
21 // - Mantiene el pipeline lleno
```

Código 4.3: Elementos invisibles (microarquitectura)

4.4.3 Ejemplo de impacto en el rendimiento

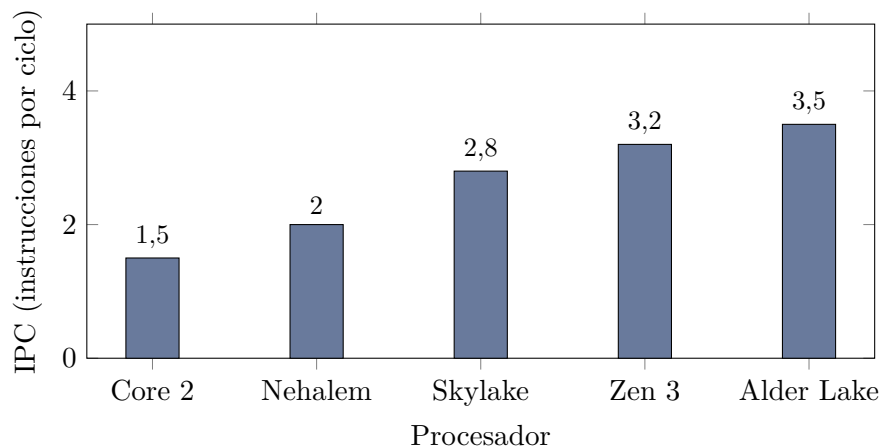


Figura 4.2: IPC hipotético de distintas implementaciones x86-64

Aunque todos estos procesadores ejecutan código x86-64, su rendimiento puede variar debido a la microarquitectura y a la carga de trabajo. Los valores de la figura son ilustrativos: el IPC no es una propiedad fija del procesador.

4.5 Síntesis del capítulo

La ISA establece qué operaciones, registros y eventos puede observar el software. Sobre esa base, la ABI fija convenciones que hacen interoperables los programas, las bibliotecas y el sistema operativo. Ambas interfaces deben permanecer estables para conservar la compatibilidad binaria.

La microarquitectura resuelve un problema distinto: cómo ejecutar eficientemente ese contrato. Sus decisiones sobre segmentación, cachés, renombrado y especulación pueden cambiar el rendimiento sin alterar el significado del programa. En consecuencia, la compatibilidad se razona desde la ISA y la ABI; el comportamiento temporal se analiza también desde la microarquitectura.

4.6 Preguntas de control del capítulo 4

1. Defina ISA y enumere sus componentes principales.
2. ¿Qué es el ABI y por qué es importante para el SO?
3. Explique los anillos de protección (rings) y su propósito.
4. ¿Cuál es la diferencia entre ISA y microarquitectura?
5. Dé tres ejemplos de elementos visibles al programador y tres invisibles.

Capítulo 5

Filosofías de diseño CISC y RISC

CISC y RISC describen decisiones de diseño, no una división absoluta entre procesadores “complejos” y “simples”.

Al finalizar este capítulo, el estudiante podrá explicar las motivaciones históricas de CISC y RISC, comparar la regularidad, la codificación y el acceso a memoria en ambas familias, y reconocer por qué los procesadores actuales combinan ideas de los dos enfoques.

CISC y RISC suelen presentarse como categorías opuestas, pero esa simplificación puede resultar engañosa. Las etiquetas surgieron en un contexto histórico concreto y describen tendencias de diseño, no una medida absoluta de complejidad. Una ISA puede tener instrucciones de longitud variable y, al mismo tiempo, ejecutarlas internamente mediante operaciones regulares. Lo útil para el estudiante es identificar qué decisiones están en la interfaz y cuáles pertenecen a la implementación.

5.1 Motivación histórica de CISC y RISC

5.1.1 La era CISC (décadas de 1970 y 1980)

CISC (*Complex Instruction Set Computer*) describe una tendencia a ofrecer instrucciones capaces de combinar varias operaciones y modos de direccionamiento. En una época de memoria costosa y compiladores menos sofisticados, esa expresividad permitía reducir la distancia entre las operaciones de alto nivel y el código máquina, además de producir programas compactos [5, 9].

Tabla 5.1: Contexto histórico del diseño CISC

Factor	Situación en la década de 1970
Costo de la memoria	Elevado en comparación con el costo actual
Velocidad CPU	Mucho menor que hoy
Compiladores	Simples, poca optimización
Lenguajes	Uso frecuente del ensamblador

5.1.2 El surgimiento de RISC (década de 1980)

RISC (*Reduced Instruction Set Computer*) se consolidó a partir de proyectos de investigación en Berkeley y Stanford [25]. Sus diseñadores observaron que: los programas utilizaban con mayor frecuencia un subconjunto relativamente pequeño de operaciones y que una codificación regular facilitaba la segmentación. El abaratamiento de la memoria y la mejora de los compiladores hicieron viable delegar en el software la combinación de operaciones simples. El objetivo no era reducir el número de instrucciones de un programa a cualquier costo, sino simplificar la ruta común de ejecución.

5.2 Complejidad de las instrucciones frente a simplicidad y regularidad

5.2.1 Características CISC

En una ISA asociada con la tradición CISC, la longitud de las instrucciones puede variar y una operación puede incluir acceso a memoria, cálculo y actualización de estado. Esto favorece la densidad de código, pero exige decodificadores capaces de localizar campos irregulares. La Tabla 5.2 reúne rasgos frecuentes; no todos deben aparecer en cada ISA clasificada como CISC.

Tabla 5.2: Características de arquitecturas CISC

Característica	Descripción
Repertorio amplio	Numerosas operaciones y extensiones
Instrucciones complejas	Una instrucción puede combinar varias acciones
Modos de direccionamiento	Diversas formas de localizar operandos
Longitud variable	En x86, instrucciones de 1 a 15 bytes
Formato irregular	Campos en posiciones variables
Operaciones con memoria	Algunas instrucciones combinan cálculo y acceso a memoria
Microcódigo	Instrucciones implementadas en firmware

La longitud máxima de 15 bytes es una restricción arquitectónica de x86; la forma concreta de decodificar esas instrucciones pertenece a la microarquitectura [14, 15].

```

1  ; Instrucción REP MOVSB en x86
2  ; Copia N bytes de una ubicación a otra.
3  ; Este esquema conceptual supone DF = 0.
4  ; Realiza implícitamente:
5  ; while (RCX != 0) {
6  ;   *RDI = *RSI;
7  ;   RSI++; RDI++; RCX--;
8  ; }
9
10 rep movsb ; Una sola instrucción que ejecuta un bucle!
11
12 ; Equivalente conceptual:
13 memcpy(dest, src, count); // Llamada a función

```

Código 5.1: Ejemplo de instrucción CISC compleja (x86)

5.2.2 Características RISC

Las ISA asociadas con RISC suelen separar el acceso a memoria de las operaciones entre registros mediante un modelo de carga y almacenamiento. Sus formatos más regulares facilitan la búsqueda y la decodificación, aunque las extensiones comprimidas y vectoriales introducen variantes. Por eso “reducido” no significa que una ISA moderna tenga pocas capacidades.

Tabla 5.3: Características de arquitecturas RISC

Característica	Descripción
Conjunto regular	Operaciones y formatos diseñados de manera sistemática
Operaciones simples	La mayoría realiza una transformación acotada
Modos de direccionamiento	Pocos modos, normalmente sencillos
Codificación	Suele ser fija; puede incluir extensiones comprimidas
Formato regular	Campos en posiciones previsibles
Carga/almacenamiento	El cálculo opera en registros; instrucciones específicas acceden a memoria
Control	La lógica cableada es común, aunque puede coexistir con microcódigo

```

1  // En ARM64, memcpy puede expresarse explícitamente.
2  // Ejemplo simplificado, sin optimizaciones:
3
4  memcpy_loop:
5      CBZ X2, done // Si count == 0, terminar
6      LDRB W3, [X1] // Cargar byte de src
7      STRB W3, [X0] // Almacenar en dest
8      ADD X0, X0, #1 // Incrementar dest
9      ADD X1, X1, #1 // Incrementar src
10     SUB X2, X2, #1 // Decrementar count
11     B memcpy_loop // Repetir
12
13 done:
14     RET // Retornar

```

Código 5.2: Equivalente RISC de memcpy (ARM64)

5.3 Microcódigo, longitud variable y modos de direccionamiento

5.3.1 Microcódigo en CISC

El **microcódigo** es un mecanismo de control utilizado para ejecutar determinadas instrucciones como secuencias de operaciones internas. En una CPU x86 moderna, muchas instrucciones comunes pasan por decodificadores ordinarios; las rutas de microcódigo se reservan para casos que requieren una secuencia más compleja [9, 14].

Desde el punto de vista del programa, una instrucción conserva un significado único. Internamente, sin embargo, el procesador puede descomponerla y coordinar varias acciones. Esta separación permite mantener compatibilidad mientras se modifica la implementación, y también facilita corregir ciertos comportamientos mediante actualizaciones de microcódigo.

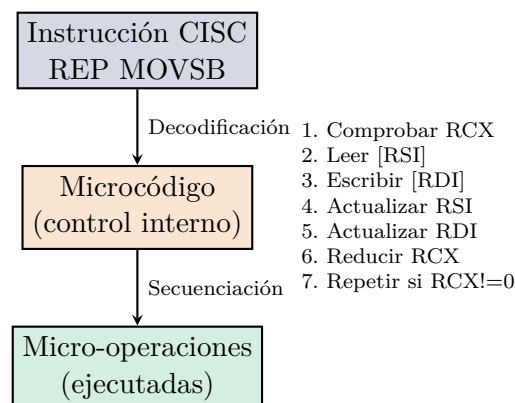


Figura 5.1: Ejecución de instrucción CISC mediante microcódigo

5.3.2 Longitud variable frente a longitud fija

La longitud influye en dos objetivos que suelen competir. Una codificación variable puede representar operaciones comunes con pocos bytes y aprovechar mejor la caché de instrucciones. Una longitud fija hace previsible los límites de cada instrucción y simplifica la decodificación paralela. Los procesadores reales equilibran densidad, ancho de búsqueda y costo de decodificación, como resume la Tabla 5.4.

Tabla 5.4: Comparación de longitud variable y fija

Aspecto	Variable (CISC)	Fija (RISC)
Densidad de código	Mayor (menos bytes)	Menor (más bytes)
Decodificación	Compleja (múltiples bytes)	Simple (tamaño fijo)
Segmentación	Decodificación más exigente	Decodificación más regular
Búsqueda	Límites de instrucción variables	Límites normalmente previsibles

5.4 Impacto en compiladores y consecuencias para el SO

5.4.1 Generación de código

El compilador traduce una misma expresión del lenguaje fuente a secuencias adaptadas a la ISA y a la ABI de destino. La cantidad de instrucciones o de bytes no basta por sí sola para anticipar el rendimiento: también importan las dependencias, las unidades disponibles y los accesos a memoria. El siguiente ejemplo solo muestra que distintas ISA expresan la suma mediante codificaciones diferentes.

```
1 // Código fuente:
2 int suma(int a, int b) {
3     return a + b;
4 }
5
6 // x86-64 (CISC) - 4 bytes, incluido ret:
7 // lea eax, [rdi + rsi]
8 // ret
9
10 // ARM64 (RISC) - 8 bytes:
11 // add w0, w0, w1
12 // ret
13
14 // RISC-V (RISC) - 8 bytes:
15 // add a0, a0, a1
16 // ret
```

Código 5.3: Diferencias en generación de código

5.4.2 Implicaciones para el sistema operativo

Tabla 5.5: Implicaciones del diseño CISC/RISC para el SO

Aspecto	CISC (x86)	RISC (ARM)
Decodificación	Codificación variable y más irregular	Codificación generalmente más regular
Manejo de excepciones	Definido por la ISA y el modelo de privilegios	Definido por la ISA y el modelo de privilegios
Cambio de contexto	Depende del estado arquitectónico habilitado	Depende del estado arquitectónico habilitado
Portabilidad	Requiere adaptar código dependiente de la ISA	Requiere adaptar código dependiente de la ISA
Energía y rendimiento	Dependen de la implementación y la carga	Dependen de la implementación y la carga

5.5 Convergencia en los procesadores modernos

La distinción se ha difuminado en los procesadores modernos. Muchas implementaciones x86 traducen instrucciones a microoperaciones internas, mientras que las ISA RISC incorporan extensiones especializadas y codificaciones compactas. Por ello, la etiqueta de la ISA no determina por sí sola el rendimiento ni el consumo energético; ambos dependen de la microarquitectura, el proceso de fabricación y la carga de trabajo [9, 14].

5.5.1 Decodificación x86 moderna

La Figura 5.2 ilustra esta convergencia. El frente del procesador reconoce instrucciones x86 y las convierte en operaciones internas que pueden planificarse entre varias unidades. La aplicación sigue observando la semántica x86; la organización posterior a la decodificación pertenece a la microarquitectura.

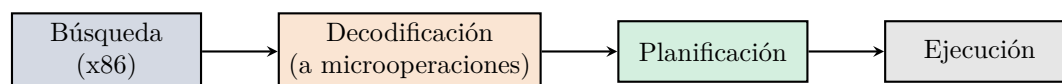


Figura 5.2: Flujo conceptual de una microarquitectura x86 moderna

5.6 Síntesis del capítulo

CISC y RISC representan respuestas históricas diferentes a las restricciones de memoria, compilación y hardware. La primera tradición favoreció instrucciones expresivas y codificaciones compactas; la segunda priorizó formatos regulares y operaciones fáciles de segmentar. Ninguna descripción permite inferir por sí sola el rendimiento de una CPU actual.

Los procesadores contemporáneos combinan ideas de ambos enfoques. El sistema operativo se relaciona con la ISA concreta —sus instrucciones privilegiadas, registros y excepciones—, no

con una etiqueta general. La microarquitectura y la carga determinan después cuánto cuestan esas operaciones.

5.7 Preguntas de control del capítulo 5

1. Explique las motivaciones históricas detrás de CISC y RISC.
2. Compare tres características clave de CISC y RISC.
3. ¿Qué es el microcódigo y por qué se usa en CISC?
4. ¿Por qué la longitud fija de instrucciones facilita la segmentación?
5. Explique cómo los procesadores x86 modernos combinan ambas filosofías.

Capítulo 6

Procesadores no segmentados y segmentados

La segmentación no acorta necesariamente una instrucción; aumenta la cantidad de instrucciones que el procesador puede atender por unidad de tiempo.

Al finalizar este capítulo, el estudiante podrá diferenciar la latencia de la tasa de procesamiento en una CPU segmentada, reconocer riesgos estructurales, de datos y de control, y explicar cómo las esperas, el reenvío y la predicción reducen su impacto.

Una instrucción atraviesa varias actividades antes de terminar: debe obtenerse, decodificarse, ejecutarse y, en algunos casos, acceder a memoria. Si la CPU esperara a completar todo el recorrido antes de comenzar la siguiente, muchos recursos permanecerían inactivos. La segmentación aprovecha esa estructura al solapar instrucciones, del mismo modo que una línea de producción atiende varios objetos en estaciones diferentes. La ganancia aparece en la cantidad de instrucciones terminadas por unidad de tiempo, no necesariamente en el tiempo de una instrucción aislada.

6.1 Diseño no segmentado y utilización de recursos

Un procesador **no segmentado** (de un ciclo o multiciclo sin segmentación) completa una instrucción antes de comenzar la siguiente. En determinados diseños, esto deja recursos inactivos durante parte del ciclo.

En un procesador de un ciclo, el periodo del reloj debe permitir que finalice la instrucción más lenta. Una suma sencilla queda sometida al mismo periodo que una carga que recorre más componentes. En un diseño multiciclo, las actividades pueden distribuirse entre ciclos, pero si no se solapan instrucciones sigue habiendo una sola en curso. Ambos modelos son útiles para identificar la oportunidad que aprovecha la segmentación.

6.1.1 Procesador de un ciclo



Figura 6.1: Ejecución de un ciclo: cada instrucción ocupa un ciclo completo

Tabla 6.1: Características del diseño de un ciclo

Característica	Valor
Ciclos por instrucción (CPI)	1
Frecuencia de reloj	Limitada por instrucción más lenta
Utilización de hardware	Baja (componentes ociosos)
Complejidad	Menor

6.2 Segmentación por etapas

La **segmentación** (*pipeline*) divide el procesamiento de una instrucción en etapas. Así, varias instrucciones pueden encontrarse al mismo tiempo en etapas diferentes, como en una línea de ensamblaje [10, 24].

Cada etapa realiza una parte del trabajo y entrega resultados intermedios a la siguiente. Cuando la segmentación se llena, una instrucción puede terminar en cada ciclo idealmente, aunque cada una necesite varios ciclos para recorrer todas las etapas. La profundidad y la función exacta de las etapas dependen de la microarquitectura; el modelo de cinco etapas se utiliza porque muestra con claridad el principio.

6.2.1 Principio de funcionamiento

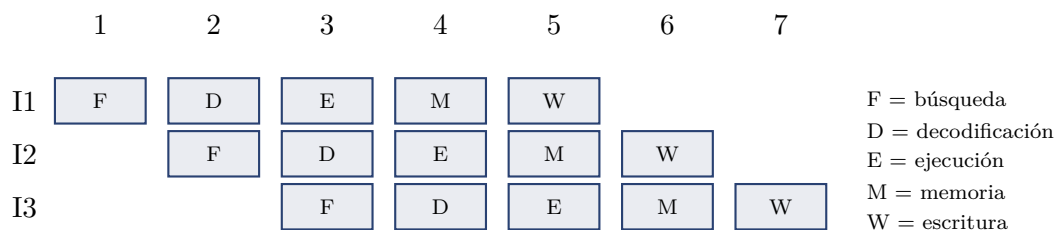


Figura 6.2: Segmentación de cinco etapas: varias instrucciones en curso

6.3 Etapas típicas y función de cada una

6.3.1 Etapas clásicas de una segmentación

El modelo clásico separa la búsqueda, la decodificación, la ejecución, el acceso a memoria y la escritura. Esta división presupone una ISA de carga/almacenamiento y sirve como punto de partida, no como descripción literal de una CPU comercial. En una microarquitectura actual, la búsqueda y la decodificación pueden ocupar varias etapas y la ejecución puede distribuirse entre unidades con latencias diferentes.

Tabla 6.2: Etapas típicas de una segmentación RISC de cinco etapas

Etapa	Nombre	Función
IF	Búsqueda	Obtener la instrucción de memoria
ID	Decodificación	Decodificar la instrucción y leer registros
EX	Ejecución	Ejecutar la operación en la ALU
MEM	Acceso a memoria	Realizar una carga o un almacenamiento
WB	Escritura	Escribir el resultado en los registros

6.3.2 Mejora en la tasa de procesamiento

Para n instrucciones y una segmentación ideal de k etapas, completar el conjunto requiere aproximadamente $k + n - 1$ ciclos: k ciclos para que termine la primera y uno adicional por cada instrucción posterior. Sin segmentación, un modelo que emplee k ciclos completos por instrucción necesitaría kn ciclos. Esta comparación supone etapas equilibradas y ausencia de conflictos; los riesgos explican por qué el resultado real se aparta del ideal.

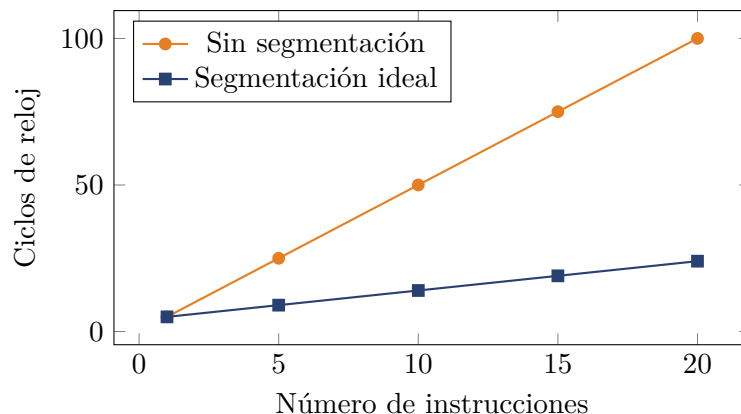


Figura 6.3: Comparación de ciclos: ejecución no segmentada y segmentación ideal

6.4 Riesgos estructurales, de datos y de control

Los **riesgos** (*hazards*) [10, 24] son situaciones que impiden que una instrucción avance en el ciclo previsto y reducen el rendimiento de la segmentación.

6.4.1 Riesgos estructurales

Los riesgos estructurales aparecen cuando dos instrucciones necesitan el mismo recurso en el mismo ciclo. Si una sola unidad debe calcular una operación aritmética y una dirección simultáneamente, una de las instrucciones tendrá que esperar. El diseñador puede añadir recursos, cambiar la organización o aceptar la espera; la elección compara costo de hardware y frecuencia del conflicto.

```

1 // Ciclo 1: I1 en EX (necesita ALU)
2 // Ciclo 1: I2 en MEM (necesita calcular dirección - ALU)
3 // CONFLICTO: Ambas necesitan la ALU!
4
5 // Solución: duplicar recursos o introducir una espera.
```

Código 6.1: Ejemplo de riesgo estructural

6.4.2 Riesgos de datos

Un riesgo de datos surge cuando una instrucción necesita un resultado que aún no está disponible. La dependencia real más directa es lectura después de escritura (RAW): una instrucción intenta leer un registro que otra anterior todavía no ha actualizado. Las dependencias no son errores del programa; son relaciones que el hardware debe respetar mientras intenta solapar la ejecución.

```

1 // RAW: Read After Write
2 ADD R1, R2, R3 // R1 = R2 + R3
3 SUB R4, R1, R5 // R4 = R1 - R5 (depende de R1!)
4 AND R6, R1, R7 // R6 = R1 & R7 (depende de R1!)
5
6 // Sin reenvío:
7 // ADD completa en ciclo 5 (WB)
8 // SUB debe esperar hasta ciclo 6 para leer R1
9 // -> 2 ciclos de espera (burbujas)
10
11 // Con reenvío:
12 // El resultado de ADD está disponible al final de EX (ciclo 3)
13 // Se puede reenviar a EX de SUB (ciclo 4)
14 // -> Sin esperas.
```

Código 6.2: Ejemplo de riesgo de datos (RAW)

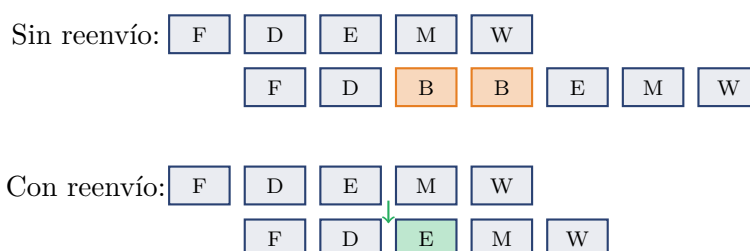


Figura 6.4: Comparación de la ejecución sin reenvío y con reenvío

6.4.3 Riesgos de control

Los riesgos de control aparecen cuando todavía no se conoce la próxima dirección de instrucción. Un salto condicional puede continuar de forma secuencial o dirigirse a otro punto. Si la CPU esperara a resolver cada salto, la etapa de búsqueda quedaría inactiva; si avanza por un camino supuesto y se equivoca, deberá descartar el trabajo especulativo.

```
1  // El procesador no sabe si el salto se tomará
2  // hasta que se evalúa la condición.
3
4  CMP R1, R2 // Comparar
5  BEQ etiqueta // Salto condicional si son iguales
6  ADD R3, R4, R5 // Instrucción siguiente (si no salta)
7  SUB R6, R7, R8 // Otra instrucción
8
9  // etiqueta:
10 // MUL R9, R10, R11 // Destino del salto
11
12 // El pipeline debe:
13 // 1. Predecir si el salto se toma o no
14 // 2. Si predice mal, descartar instrucciones incorrectas
15 // 3. Cargar instrucciones correctas (penalidad)
```

Código 6.3: Ejemplo de riesgo de control

6.5 Técnicas de mitigación mediante esperas, reenvío y predicción

6.5.1 Esperas e inserción de burbujas

La respuesta más simple ante un riesgo es detener temporalmente una o varias etapas. La espera introduce una “burbuja”, es decir, una posición que avanza por la segmentación sin completar una instrucción útil. Esta técnica conserva la corrección, pero reduce la tasa de procesamiento.

```
1  // Cuando hay dependencia que no se puede resolver:
2  // Insertar NOP (burbujas) hasta que el dato esté listo.
3
4  ADD R1, R2, R3 // Escribe R1 en ciclo 5
5  NOP // Espera
6  NOP // Espera
7  SUB R4, R1, R5 // Lee R1 en ciclo 6 (ahora válido)
```

Código 6.4: Espera en una segmentación

6.5.2 Reenvío de datos

No siempre es necesario esperar a que un resultado se escriba en el banco de registros. Si la ALU ya lo produjo, un camino de reenvío puede llevarlo directamente a la entrada de otra unidad. El reenvío acorta determinadas esperas, aunque no elimina todas; una carga seguida

inmediatamente por su uso, por ejemplo, puede requerir tiempo adicional si el dato llega tarde desde la caché.

```

1 // El resultado se envía directamente desde una etapa posterior
2 // a una etapa anterior que lo necesita
3
4 // Caminos de reenvío típicos:
5 // EX -> EX: Resultado ALU a siguiente instrucción
6 // MEM -> EX: Resultado de una carga cuando ya está disponible
7 // WB -> EX: Resultado disponible tras la escritura
8
9 // En el modelo clásico, una carga seguida inmediatamente
10 // por su uso todavía puede requerir un ciclo de espera.
11
12 // En hardware: multiplexores que seleccionan la fuente del dato

```

Código 6.5: Reenvío de datos

6.5.3 Predicción de saltos

La predicción permite elegir una dirección antes de resolver el salto. Un predictor utiliza reglas estáticas o el comportamiento previo para estimar el resultado y, en diseños modernos, también el destino. Una predicción correcta mantiene ocupado el frente del procesador. Una predicción incorrecta obliga a anular instrucciones y reanudar la búsqueda desde la dirección correcta, por lo que la penalización crece con la profundidad de la ruta especulativa.

Tabla 6.3: Técnicas de predicción de saltos

Técnica	Descripción
Estática siempre-tomado	Siempre predice que el salto se toma
Estática nunca-tomado	Siempre predice que no se toma
Estática hacia atrás	Predice como tomados los saltos hacia atrás
Dinámica 1-bit	Tabla con último resultado del salto
Dinámica 2-bit	Máquina de estados con histéresis
Correlacional	Considera historia de otros saltos
Combinada	Combina múltiples predictores

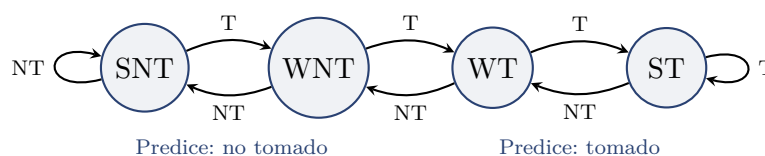


Figura 6.5: Máquina de estados de predicción de dos bits

6.6 Síntesis del capítulo

La segmentación divide el trabajo de una instrucción y solapa el recorrido de varias. En condiciones ideales, después del llenado aumenta la tasa de procesamiento sin reducir nece-

sariamente la latencia de cada instrucción. El modelo de cinco etapas permite visualizar el principio, aunque las CPU actuales emplean organizaciones más profundas y anchas.

El solapamiento introduce conflictos. Los riesgos estructurales representan competencia por recursos; los de datos, dependencias entre resultados; y los de control, incertidumbre sobre la siguiente instrucción. Las esperas conservan la corrección, el reenvío aprovecha resultados antes de su escritura y la predicción mantiene activa la búsqueda. El rendimiento efectivo depende del equilibrio entre estas técnicas y el comportamiento del programa.

6.7 Preguntas de control del capítulo 6

1. Compare un procesador de un ciclo con uno segmentado.
2. Enumere las cinco etapas típicas de una segmentación y describa cada una.
3. Explique los tres tipos de riesgos con ejemplos.
4. ¿Qué es el reenvío y cómo resuelve riesgos de datos?
5. Describa la máquina de estados de dos bits para predicción de saltos.

Capítulo 7

Métricas para el análisis del rendimiento

Una medición solo es útil cuando la métrica, la carga de trabajo y el método están claramente definidos.

Al finalizar este capítulo, el estudiante podrá diferenciar latencia y tasa de procesamiento, relacionar tiempo, frecuencia, CPI e IPC sin comparar de manera engañosa ISA diferentes, y aplicar la ley de Amdahl en mediciones reproducibles realizadas en Linux.

Decir que un procesador o un programa es “rápido” no constituye una explicación. Puede significar que una tarea individual responde pronto, que el sistema completa muchas tareas por segundo o que una versión supera a otra bajo una carga concreta. El análisis de rendimiento comienza al definir qué trabajo se mide, qué métrica lo representa y qué condiciones se mantienen constantes.

7.1 Latencia y tasa de procesamiento

Latencia es el tiempo requerido para completar una tarea individual. La **tasa de procesamiento** (*throughput*) es la cantidad de trabajo completado por unidad de tiempo. [8, 9]

7.1.1 Analogía intuitiva

Una autopista ayuda a separar dos preguntas. El tiempo que tarda un vehículo en recorrerla representa la latencia; la cantidad de vehículos que atraviesan un punto por hora representa la tasa de procesamiento. Añadir carriles puede aumentar el flujo sin reducir el tiempo de viaje de un automóvil. De forma análoga, la segmentación y el paralelismo pueden aumentar el trabajo completado sin reducir la latencia de una operación individual.

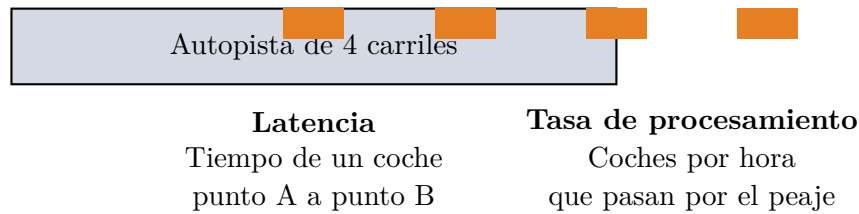


Figura 7.1: Analogía de latencia y tasa de procesamiento en una autopista

7.1.2 En el contexto de los procesadores

La distinción se aplica en varios niveles. Una memoria puede tardar cierto número de nanosegundos en responder y, gracias al solapamiento, atender muchos accesos por segundo. Un sistema operativo puede reducir el tiempo de una llamada o aumentar la cantidad de llamadas concurrentes. La métrica adecuada depende de la pregunta.

Tabla 7.1: Latencia y tasa de procesamiento en diferentes contextos

Contexto	Latencia	Tasa de procesamiento
Acceso a memoria	Ciclos para obtener dato	Accesos por segundo
Instrucción	Ciclos para completar	Instrucciones por ciclo (IPC)
Llamada al sistema	Tiempo de una llamada	Llamadas por segundo
Cambio de contexto	Tiempo de un cambio	Cambios por segundo

7.2 Interpretación de CPI e IPC y errores comunes

CPI (*Cycles Per Instruction*) es el promedio de ciclos de reloj por instrucción. **IPC** (*Instructions Per Cycle*) es su inverso para una misma ejecución.

$$\text{CPI} = \frac{1}{\text{IPC}} \quad \text{y} \quad \text{IPC} = \frac{1}{\text{CPI}}$$

7.2.1 Ecuación del rendimiento

La ecuación fundamental del rendimiento del procesador:

$$\text{Tiempo de ejecución} = \frac{\text{Instrucciones} \times \text{CPI}}{\text{Frecuencia de reloj}} \quad (7.1)$$

O equivalentemente:

$$\text{Tiempo de ejecución} = \frac{\text{Instrucciones}}{\text{IPC} \times \text{Frecuencia de reloj}} \quad (7.2)$$

La ecuación obliga a separar tres factores. El programa y el compilador determinan cuántas instrucciones se ejecutan; la interacción entre el código y la microarquitectura influye en el CPI; y el reloj establece la duración de cada ciclo. Mejorar uno de ellos no garantiza reducir el tiempo total si otro empeora. Un compilador puede, por ejemplo, emitir más instrucciones pero exponer mayor paralelismo y obtener un tiempo menor.

7.2.2 Factores que afectan al CPI

Tabla 7.2: Factores que afectan el CPI

Factor	Impacto	Ejemplo
Esperas en la segmentación	+CPI	Riesgos de datos o de control
Fallos de caché	+CPI, a menudo elevado	Acceso a memoria principal
Predicción incorrecta	+CPI	Descarte de instrucciones especulativas
Conflictos de recursos	+CPI	Riesgos estructurales

7.2.3 Errores comunes de interpretación

CPI e IPC necesitan contexto. Comparar IPC entre ISA diferentes puede ser engañoso porque una instrucción no representa la misma cantidad de trabajo en cada arquitectura. Incluso dentro de una misma ISA, el valor cambia con la carga: un programa limitado por memoria no se comporta como uno con abundantes operaciones independientes. El IPC pico expresa una capacidad teórica y no un resultado garantizado para código real.

7.3 Amdahl y límites de la aceleración

La **ley de Amdahl** [1] establece que la mejora del rendimiento de un sistema está limitada por la fracción de tiempo que no puede acelerarse.

$$S_{\text{total}} = \frac{1}{(1 - P) + \frac{P}{S}}$$

Donde: P representa la fracción del tiempo original que puede mejorarse y S el factor de aceleración aplicado a esa fracción. El término $1 - P$ permanece sin cambio y acaba dominando el resultado cuando S crece.

7.3.1 Ejemplo ilustrativo

Supóngase que un programa dedica el 80 % de su tiempo a una parte paralelizable y el 20 % a una parte que permanece secuencial. Acelerar cuatro veces la primera fracción no

acelera cuatro veces el programa completo: la parte secuencial conserva su duración. El cálculo siguiente hace explícita esa limitación.

```

1 // Supongamos un programa que:
2 // - 80% del tiempo: cómputo paralelizable
3 // - 20% del tiempo: secuencial (inicialización, etc.)
4
5 // Si paralelizamos el 80% en 4 procesadores:
6 //  $P = 0.8, S = 4$ 
7 //  $S_{total} = 1 / ((1 - 0.8) + 0.8/4)$ 
8 //  $= 1 / (0.2 + 0.2)$ 
9 //  $= 1 / 0.4$ 
10 //  $= 2.5x$  de aceleración
11
12 // Con infinitos procesadores:
13 //  $S_{total\_max} = 1 / (1 - P) = 1 / 0.2 = 5x$  de aceleración máxima

```

Código 7.1: Ejemplo de aplicación de la Ley de Amdahl

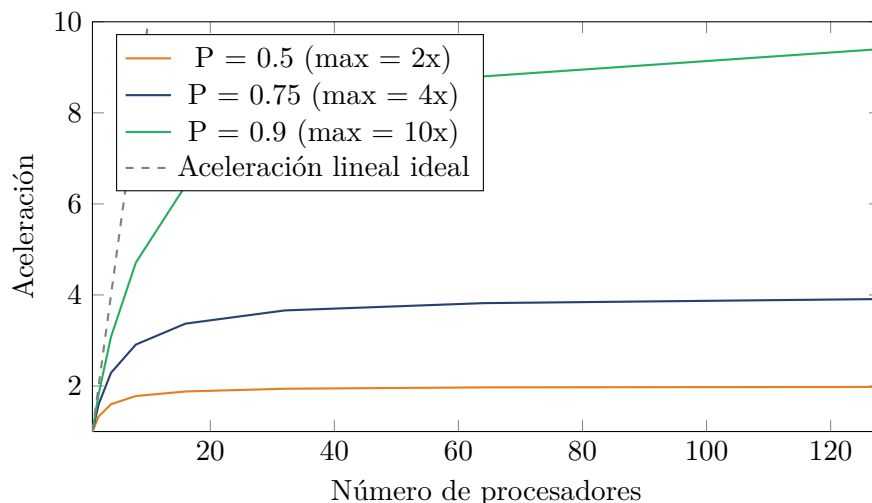


Figura 7.2: Ley de Amdahl: aceleración según el número de procesadores

7.3.2 Implicaciones para el diseño de sistemas

La ley de Amdahl también se ha analizado en el contexto de los procesadores multinúcleo [11].

Lectura práctica de la ley de Amdahl. Una optimización produce mayor impacto cuando actúa sobre la parte que domina el tiempo total. Paralelizar no elimina la fracción secuencial ni los costos de coordinación, de modo que añadir procesadores termina ofreciendo rendimientos decrecientes. Antes de optimizar, conviene medir dónde se consume el tiempo.

7.4 Mediciones básicas de rendimiento en Linux

7.4.1 Herramientas básicas

Linux ofrece temporizadores y contadores de hardware que permiten observar el comportamiento de una ejecución. `time` separa el tiempo de reloj de pared del tiempo de CPU en usuario y núcleo. `perf stat` solicita contadores como ciclos, instrucciones o fallos, sujetos a la disponibilidad y a las políticas de acceso de la plataforma [20]. Una sola ejecución no basta: la carga del sistema, la frecuencia y el calentamiento de cachés introducen variación.

```
# 1. time - Medición básica de tiempo
$ time ./mi_programa
real 0m2.345s # Tiempo total (reloj de pared)
user 0m2.100s # Tiempo en modo usuario
sys 0m0.200s # Tiempo en modo kernel

# 2. perf - Contadores de rendimiento de la CPU
$ perf stat ./mi_programa
Performance counter stats for './mi_programa':
    2,341.42 msec task-clock
    8,234,567 instructions
    1,234,890 cache-references
    45,678 cache-misses

# 3. perf record/report - Perfilado detallado
$ perf record ./mi_programa
$ perf report

# 4. valgrind/cachegrind - Simulación de caché
$ valgrind --tool=cachegrind ./mi_programa
```

Código 7.2: Herramientas de medición en Linux

7.4.2 Métricas clave

Los contadores deben interpretarse como evidencia de una hipótesis, no como un diagnóstico automático. Un aumento de fallos de caché puede explicar una regresión si el trabajo es equivalente y el resto de condiciones se controla. Sin esa comparación, el número aislado dice poco. La Tabla 7.3 reúne mediciones accesibles para las prácticas.

Tabla 7.3: Métricas útiles para estudiantes

Métrica	Herramienta	Comando
Tiempo de ejecución	time	time ./prog
Ciclos de CPU	perf	perf stat -e cycles
Instrucciones	perf	perf stat -e instructions
CPI	perf	perf stat -e cycles,instructions
Fallos de caché	perf	perf stat -e cache-misses
Fallos de página	perf	perf stat -e page-faults
Cambios de contexto	perf	perf stat -e context-switches

7.4.3 Ejemplo de medición completa

```
#!/bin/bash
set -u

if [ "$#" -lt 1 ]; then
    echo "Uso: $0 PROGRAMA [ARGUMENTOS...]" >&2
    exit 1
fi

PROGRAMA=$1
shift
ITERACIONES=${ITERACIONES:-10}
TIEMPOS=$(mktemp)
MUESTRA=$(mktemp)
trap 'rm -f "$TIEMPOS" "$MUESTRA"' EXIT

printf "Midiendo: %s\n" "$PROGRAMA"
printf "Iteraciones: %s\n\n" "$ITERACIONES"

for i in $(seq 1 "$ITERACIONES"); do
    # -o separa la salida de time de stdout y stderr del programa.
    /usr/bin/time -f "%e" -o "$MUESTRA" \
        -- "$PROGRAMA" "$@"
    t=$(<"$MUESTRA")
    printf "Iteración %d: %ss\n" "$i" "$t"
    printf "%s\n" "$t" >> "$TIEMPOS"
done

# Ordenar permite calcular una mediana sin depender de bc.
sort -n "$TIEMPOS" -o "$TIEMPOS"
awk '
{
    valores[NR] = $1
    suma += $1
    suma2 += $1 * $1
}
END {
    if (NR % 2)
        mediana = valores[(NR + 1) / 2]
    else
        mediana = (valores[NR / 2] + valores[NR / 2 + 1]) / 2
    media = suma / NR
    varianza = suma2 / NR - media * media
    if (varianza < 0) varianza = 0
    printf "\nMedia: %.4f s\n", media
    printf "Mediana: %.4f s\n", mediana
    printf "Mínimo: %.4f s\n", valores[1]
    printf "Máximo: %.4f s\n", valores[NR]
    printf "Desviación: %.4f s\n", sqrt(varianza)
}' "$TIEMPOS"

printf "\nEstadísticas de perf:\n"
perf stat -r "$ITERACIONES" \
    -e cycles,instructions,cache-misses \
    -- "$PROGRAMA" "$@"
```

Código 7.3: Script de medición completo

El script conserva cada observación y separa la salida del temporizador de la salida del programa. La mediana y la dispersión permiten detectar ejecuciones atípicas; `perf stat -r` añade la variabilidad de los contadores cuando la plataforma los expone [8, 20].

7.5 Síntesis del capítulo

La latencia describe cuánto tarda una tarea; la tasa de procesamiento indica cuánto trabajo se completa en un intervalo. CPI, IPC, frecuencia y número de instrucciones ayudan a descomponer el tiempo de CPU, pero solo son comparables cuando la carga, la ISA y el método están claramente definidos. Ninguna métrica aislada sustituye el tiempo observado por el usuario.

La ley de Amdahl muestra que la fracción no mejorada limita la aceleración global. Este principio conduce a una disciplina experimental: medir primero, repetir, controlar variables y documentar el entorno. Herramientas como `time` y `perf` proporcionan datos; la interpretación exige relacionarlos con el trabajo ejecutado y con una hipótesis concreta.

7.6 Preguntas de control del capítulo 7

1. Explique la diferencia entre latencia y tasa de procesamiento con un ejemplo.
2. Derive la ecuación del rendimiento del procesador.
3. Enumere tres trampas comunes al interpretar IPC.
4. Aplique la ley de Amdahl: si un programa tiene 60 % paralelizable y se usan ocho procesadores, ¿cuál es la aceleración máxima?
5. Escriba un comando para medir CPI usando `perf`.

Capítulo 8

Procesadores escalares y superescalares

El ancho del procesador solo se aprovecha cuando existen instrucciones independientes y recursos disponibles.

Al finalizar este capítulo, el estudiante podrá distinguir un procesador escalar de uno superescalar, identificar dependencias que limitan el paralelismo entre instrucciones y explicar el papel de la ejecución especulativa y fuera de orden.

La segmentación permite mantener varias instrucciones en curso. El siguiente paso consiste en iniciar más de una durante el mismo ciclo. Para conseguirlo no basta con duplicar unidades: el procesador debe encontrar instrucciones independientes, disponer de operandos y preservar el comportamiento exigido por la ISA. El paralelismo a nivel de instrucción describe esa oportunidad y las técnicas superescalares intentan aprovecharla.

8.1 Procesamiento escalar con una emisión por ciclo

Un procesador **escalar** puede emitir como máximo una instrucción por ciclo de reloj. La latencia de esa instrucción puede ser mayor que un ciclo.

La palabra “escalar” se refiere aquí al ancho de emisión, no al tipo de dato. Una CPU escalar segmentada puede tener varias instrucciones en etapas diferentes, pero incorpora como máximo una nueva instrucción por ciclo. En ausencia de esperas, su IPC se aproxima a uno; las dependencias y los fallos de memoria lo reducen.

8.1.1 Características del diseño escalar

Tabla 8.1: Características de procesadores escalares

Característica	Valor típico
Instrucciones por ciclo (IPC)	≤ 1
CPI	≥ 1
Pipeline	Simple, una instrucción por etapa
Unidades funcionales	Una de cada tipo
Complejidad de control	Relativamente simple

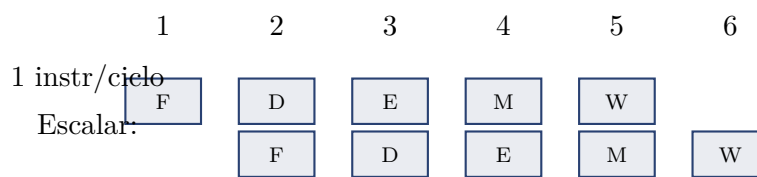


Figura 8.1: Segmentación escalar: como máximo una instrucción emitida por ciclo

8.2 Procesamiento superescalar con varias emisiones por ciclo

Un procesador **superescalar** puede emitir varias instrucciones independientes en un ciclo y explotar el paralelismo a nivel de instrucción (ILP).

Para emitir varias instrucciones, el frente del procesador debe obtenerlas y decodificarlas con suficiente rapidez. Después necesita distribuirlas entre unidades funcionales y conservar sus resultados hasta que puedan hacerse visibles en un orden compatible con el programa. El ancho anunciado de una etapa no garantiza el mismo ancho en toda la máquina.

8.2.1 Ejemplo de microarquitectura superescalar fuera de orden

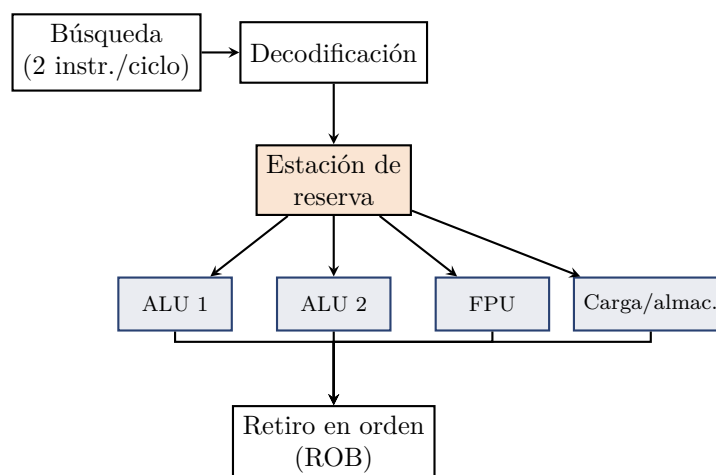


Figura 8.2: Estructura conceptual de un procesador superescalar fuera de orden

8.2.2 Grados de superescalaridad

Tabla 8.2: Anchos relevantes en una microarquitectura superescalar

Ancho	Pregunta que responde
Búsqueda	¿Cuántos bytes o instrucciones puede obtener el frente por ciclo?
Decodificación	¿Cuántas instrucciones puede convertir a la representación interna?
Despacho o emisión	¿Cuántas operaciones puede enviar hacia las unidades?
Ejecución	¿Qué combinación de operaciones aceptan las unidades disponibles?
Retiro	¿Cuántos resultados pueden hacerse arquitectónicamente visibles?

El ancho de decodificación, emisión y retiro no es necesariamente el mismo. Además, cada etapa puede tener restricciones por tipo de instrucción. Por eso ningún ancho aislado equivale al IPC que alcanzará una carga real [9].

8.3 Paralelismo a nivel de instrucción (ILP)

El **paralelismo a nivel de instrucción** (ILP) [9] es la cantidad de instrucciones independientes que pueden ejecutarse de forma solapada.

8.3.1 Factores que limitan el ILP

El flujo de instrucciones contiene dependencias verdaderas que no pueden eliminarse: si una resta necesita el resultado de una suma, debe esperar. También puede haber competencia por unidades, incertidumbre en los saltos y esperas por memoria. El procesador busca trabajo independiente dentro de una ventana limitada; si no lo encuentra, parte de su ancho queda sin utilizar.

```

1 // Dependencia de datos (RAW)
2 ADD R1, R2, R3 // I1: R1 = R2 + R3
3 SUB R4, R1, R5 // I2: R4 = R1 - R5 (depende de I1)
4 MUL R6, R1, R7 // I3: R6 = R1 * R7 (depende de I1)
5
6 // Aunque el procesador tenga tres vías, primero debe producirse R1.
7 // I2 e I3 deben esperar ese resultado.
8
9 // ILP disponible: bajo

```

Código 8.1: Ejemplo de limitaciones de ILP

```

1 // Instrucciones independientes

```

```

2 ADD R1, R2, R3 // I1
3 SUB R4, R5, R6 // I2 (independiente)
4 MUL R7, R8, R9 // I3 (independiente)
5 AND R10, R11, R12 // I4 (independiente)
6
7 // En un procesador de cuatro vías, podrían emitirse juntas
8 // si existen unidades funcionales suficientes.
9 // ILP disponible: alto

```

Código 8.2: Ejemplo de alto ILP

8.3.2 Técnicas para explotar el ILP

La microarquitectura combina varias técnicas. El renombrado elimina dependencias falsas causadas por reutilizar nombres de registros; la ejecución fuera de orden permite adelantar operaciones listas; y la especulación mantiene el flujo mientras se resuelven saltos o dependencias. El compilador también puede reorganizar y desenrollar bucles para exponer instrucciones independientes.

Tabla 8.3: Técnicas para explotar ILP

Técnica	Descripción
Segmentación superescalar	Emitir varias instrucciones por ciclo
Ejecución fuera de orden	Ejecutar instrucciones cuando estén listas
Ejecución especulativa	Ejecutar antes de confirmar el camino de control
Renombrado de registros	Eliminar dependencias falsas (WAR y WAW)
Desenrollado de bucles	Exponer más instrucciones independientes

8.4 Especulación y predicción de saltos

La **ejecución especulativa** permite al procesador ejecutar instrucciones antes de saber si realmente se necesitarán, basándose en predicciones. Si la predicción es incorrecta, los resultados se descartan [9].

8.4.1 Tipos de especulación

Especular significa utilizar una suposición antes de confirmarla. La forma más visible predice el camino de un salto, pero algunas CPU también predicen si dos accesos a memoria son independientes. El trabajo especulativo no puede modificar de forma irreversible el estado arquitectónico hasta comprobarse la suposición.

Tabla 8.4: Tipos de especulación en procesadores modernos

Tipo	Qué se predice	Penalidad si falla
Flujo de control	Resultado y destino de saltos	Descarte de instrucciones
Dependencias de memoria	Independencia entre accesos	Reejecución
Valores (menos común)	Valores futuros	Reejecución

8.4.2 Impacto de la predicción de saltos

El costo de un fallo depende de cuántas instrucciones deban descartarse y de cuánto tarde en reanudarse la búsqueda. Por eso una mejora pequeña en la tasa de aciertos puede ser importante en una segmentación profunda. La Figura 8.3 ilustra la relación de manera conceptual; no representa una CPU particular.

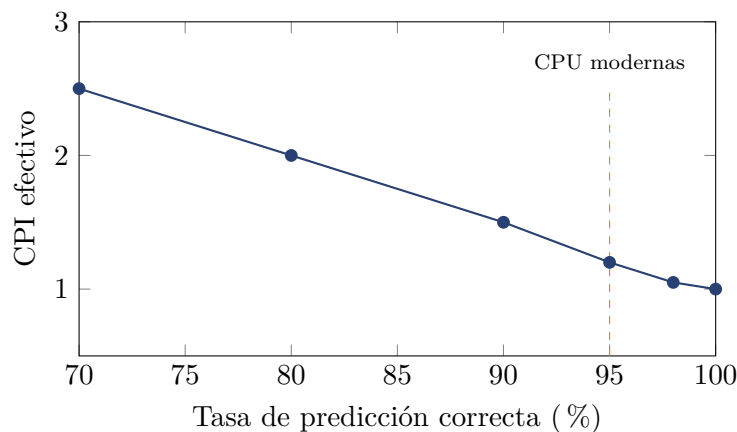


Figura 8.3: Impacto de la tasa de predicción en el CPI

La figura es conceptual: el CPI efectivo también depende de la frecuencia de saltos, la penalización por fallo y el resto de la carga de trabajo.

8.5 Ejecución fuera de orden y su efecto en el rendimiento

La **ejecución fuera de orden** (*out-of-order*, OoO) permite que instrucciones posteriores se ejecuten antes que otras anteriores cuando sus operandos y recursos están disponibles. Los resultados se retiran en orden para conservar un estado arquitectónico preciso [9].

8.5.1 Ejemplo conceptual de ejecución fuera de orden

Ejecutar fuera de orden no significa que el programa observe resultados arbitrarios. La CPU puede adelantar una operación independiente mientras otra espera, pero conserva información suficiente para retirar los resultados en un orden preciso y recuperar el estado ante una excepción o una predicción incorrecta.

```

1 // Código original:
2 ADD R1, R2, R3 // I1: produce R1
3 MUL R4, R5, R6 // I2: independiente de I1
4 SUB R7, R1, R8 // I3: R7 = R1 - R8 (depende de I1)
5 AND R9, R10, R11 // I4: R9 = R10 & R11 (independiente)
6
7 // En orden: I3 puede detener el avance hasta recibir R1.
8 // Fuera de orden: I2 e I4 pueden aprovechar unidades disponibles
9 // mientras I3 espera. I3 se ejecuta cuando R1 está listo.
10 // La ganancia exacta depende de las latencias y del ancho.

```

Código 8.3: Ejemplo de ejecución fuera de orden

8.5.2 Estructura de un procesador fuera de orden

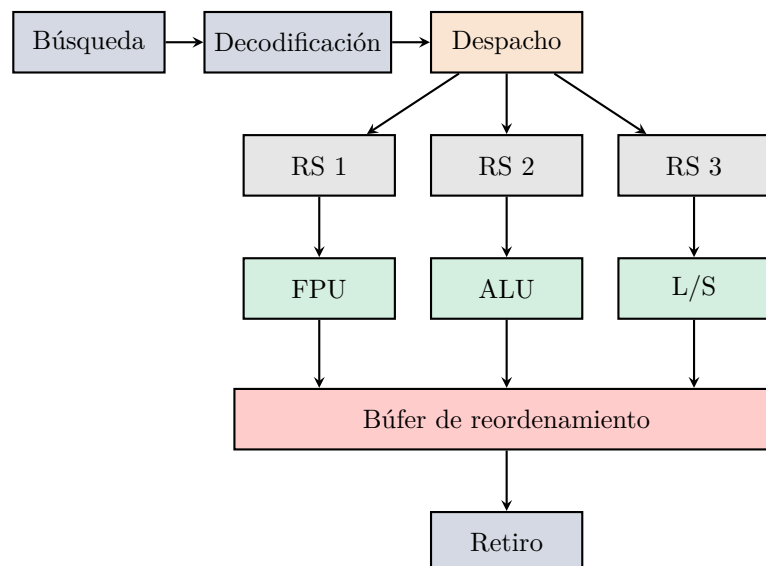


Figura 8.4: Estructura de un procesador fuera de orden

8.6 Síntesis del capítulo

Un procesador escalar incorpora como máximo una instrucción por ciclo, mientras que uno superescalar intenta emitir varias. La ganancia depende del paralelismo disponible y de los recursos para explotarlo; el ancho máximo no equivale al IPC que obtendrá cualquier programa.

La ejecución fuera de orden aprovecha operaciones listas mientras otras esperan, y la especulación evita detener el frente ante cada incertidumbre. El renombrado, las estaciones de reserva y el búfer de reordenamiento ayudan a mantener un estado arquitectónico preciso. Estas técnicas mejoran la utilización, pero no eliminan las dependencias verdaderas ni la latencia de la memoria.

8.7 Preguntas de control del capítulo 8

1. ¿Cuál es la diferencia fundamental entre un procesador escalar y uno superescalar?

2. Defina ILP y explique qué factores lo limitan.
3. Explique cómo la ejecución especulativa mejora el rendimiento.
4. ¿Qué es la ejecución fuera de orden y cuál es su beneficio principal?
5. Compare el CPI teórico de un procesador escalar, uno superescalar en orden y uno superescalar fuera de orden.

Capítulo 9

Procesamiento vectorial (SIMD)

SIMD acelera una operación repetida sobre datos independientes, siempre que la memoria y el flujo de control permitan agruparlos.

Al finalizar este capítulo, el estudiante podrá reconocer bucles adecuados e inadecuados para SIMD, comparar la autovectorización con el uso de funciones intrínsecas y comprobar correctamente tanto el soporte de AVX como el estado habilitado por el sistema operativo.

Hasta ahora se ha estudiado cómo solapar instrucciones distintas. SIMD explota otra forma de paralelismo: aplicar una misma operación a varios datos. La idea resulta natural en arreglos, imágenes y álgebra lineal, donde muchos elementos siguen el mismo cálculo. Sin embargo, un registro más ancho no garantiza una aceleración proporcional. Las dependencias, los accesos irregulares y el ancho de banda de memoria pueden impedir que todas las vías trabajen a la vez.

9.1 Condiciones para aplicar el paralelismo de datos

El **paralelismo de datos** ocurre cuando la misma operación se aplica a múltiples elementos independientes. Es la base del modelo SIMD (*Single Instruction, Multiple Data*).

9.1.1 Aplicaciones adecuadas para SIMD

SIMD resulta especialmente útil cuando las iteraciones realizan la misma operación sobre posiciones contiguas e independientes. Una suma de vectores es el ejemplo más directo: cada resultado depende únicamente de los elementos correspondientes de dos arreglos. El procesamiento de imágenes, los bloques de álgebra lineal, ciertas transformaciones criptográficas y numerosos cálculos numéricos presentan patrones semejantes.

```
1 for (int i = 0; i < N; i++) {  
2     c[i] = a[i] + b[i];  
3 }
```

Código 9.1: Bucle con paralelismo de datos

El compilador puede agrupar varias iteraciones y emitir una instrucción vectorial si demuestra que los rangos no se solapan de forma peligrosa y que existe una estrategia rentable para la cola del bucle.

9.1.2 Limitaciones para el uso de SIMD

La vectorización se dificulta cuando una iteración necesita el resultado de la anterior, como en una recurrencia. También pierde eficacia si cada elemento toma un camino de control distinto o si las direcciones se obtienen mediante índices impredecibles. Algunas ISA ofrecen máscaras y operaciones de recolección, pero su existencia no elimina el costo de un patrón irregular.

```
1 for (int i = 1; i < N; i++) {
2     a[i] = a[i-1] + b[i];
3 }
```

Código 9.2: Dependencia que impide una vectorización directa

En este caso, calcular `a[i]` requiere conocer `a[i-1]`; las iteraciones no pueden tratarse como operaciones independientes sin transformar el algoritmo.

9.2 Registros vectoriales y operaciones en bloque

Las extensiones SIMD proporcionan **registros vectoriales** e **instrucciones vectoriales** que interpretan un registro como varios elementos. En x86, las familias SSE, AVX y AVX-512 introdujeron anchos de 128, 256 y 512 bits, respectivamente; la disponibilidad concreta debe comprobarse por característica y por modelo [13, 15, 16].

9.2.1 Evolución de las extensiones SIMD en x86

Tabla 9.1: Evolución de extensiones SIMD en x86

Extensión	Año	Ancho	Registros
MMX	1997	64 bits	8 (MM0–MM7)
SSE	1999	128 bits	8/16 (XMM)
SSE2	2001	128 bits	8/16 (XMM)
AVX	2011	256 bits	16 (YMM)
AVX2	2013	256 bits	16 (YMM)
AVX-512	2016	512 bits	32 (ZMM)

9.2.2 Ejemplo de operación SIMD

El ancho del registro determina cuántos elementos caben en una operación. Un registro de 128 bits puede contener cuatro valores de 32 bits; uno de 256 bits, ocho. La Figura 9.1 representa cuatro sumas independientes dentro de una sola operación vectorial.



Figura 9.1: Operación SIMD: suma de vectores de cuatro elementos

9.3 Ejemplos conceptuales

9.3.1 Suma de vectores

La versión escalar procesa un elemento por iteración. La versión con AVX carga ocho números de punto flotante de precisión simple, los suma y almacena ocho resultados. El bucle adicional procesa la cola cuando el tamaño no es múltiplo del ancho vectorial. En un programa portable conviene disponer también de una ruta escalar y seleccionar la implementación compatible durante la ejecución.

```

1  // Version escalar
2  void suma_escalar(float *a, float *b, float *c, int n) {
3      for (int i = 0; i < n; i++) {
4          c[i] = a[i] + b[i];
5      }
6  }
7
8  // Version SIMD con AVX (256 bits = 8 floats)
9  #include <immintrin.h>
10
11 void suma_simd(float *a, float *b, float *c, int n) {
12     int i = 0;
13     for (; i + 7 < n; i += 8) {
14         // Cargar 8 floats desde memoria
15         __m256 va = _mm256_loadu_ps(&a[i]);
16         __m256 vb = _mm256_loadu_ps(&b[i]);
17
18         // Sumar 8 elementos simultáneamente
19         __m256 vc = _mm256_add_ps(va, vb);
20
21         // Almacenar resultado
22         _mm256_storeu_ps(&c[i], vc);
23     }
24     // Procesar la cola cuando n no sea múltiplo de 8.
25     for (; i < n; i++) {
26         c[i] = a[i] + b[i];
27     }
28 }

```

Código 9.3: Suma de vectores: escalar vs SIMD

9.3.2 Filtro de imagen por convolución

Una convolución reutiliza coeficientes y aplica la misma combinación a muchos píxeles. Esa regularidad permite difundir cada coeficiente en un registro y procesar varias posiciones vecinas a la vez. La vectorización no resuelve por sí sola los bordes, la alineación ni la selección de instrucciones disponibles; esos detalles siguen formando parte de la implementación.

```

1  // Filtro de suavizado 3x3 aplicado a imagen
2  // Con SIMD: procesar múltiples píxeles simultáneamente
3
4  void blur_filter_simd(float *input, float *output,
5                        int width, int height) {
6      const float kernel[9] = {
7          1/16.0, 2/16.0, 1/16.0,
8          2/16.0, 4/16.0, 2/16.0,
9          1/16.0, 2/16.0, 1/16.0
10     };
11
12     // Cargar kernel en registros SIMD
13     __m256 k[9];
14     for (int i = 0; i < 9; i++) {
15         k[i] = _mm256_set1_ps(kernel[i]); // Broadcast
16     }
17
18     // Procesar 8 píxeles por iteración
19     for (int y = 1; y < height - 1; y++) {
20         for (int x = 1; x + 7 < width - 1; x += 8) {
21             __m256 sum = _mm256_setzero_ps();
22
23             // Aplicar kernel (9 multiplicaciones SIMD)
24             for (int ky = -1; ky <= 1; ky++) {
25                 for (int kx = -1; kx <= 1; kx++) {
26                     int idx = (y + ky) * width + (x + kx);
27                     __m256 pixel = _mm256_loadu_ps(&input[idx]);
28                     sum = _mm256_fmadd_ps(pixel, k[(ky+1)*3+(kx+1)], sum);
29                 }
30             }
31
32             _mm256_storeu_ps(&output[y*width+x], sum);
33         }
34     }
35 }

```

Código 9.4: Aplicación de filtro SIMD

El ejemplo de convolución muestra el bloque vectorial central. Una implementación completa debe procesar los bordes y la cola escalar, y comprobar que la CPU admite FMA y AVX2.

9.4 Papel del compilador y de las funciones intrínsecas

9.4.1 Autovectorización

El compilador analiza dependencias, alineación y costo para decidir si transforma un bucle. En GCC, los niveles de optimización habilitan distintos pases y el informe de optimización permite comprobar qué bucles se vectorizaron o por qué se rechazaron [7]. La directiva `omp simd` comunica que las iteraciones pueden tratarse conforme al modelo de OpenMP, pero no corrige una afirmación falsa del programador sobre las dependencias.

```

1 // El compilador puede vectorizar automáticamente si:
2 // 1. No hay dependencias entre iteraciones
3 // 2. Los accesos a memoria son contiguos
4 // 3. No hay llamadas a funciones externas
5
6 // Compilar con: gcc -O3 -mavx2 -ftree-vectorize
7
8 void procesar_arrays(float *a, float *b, float *c, int n) {
9     #pragma omp simd // Hint para el compilador
10    for (int i = 0; i < n; i++) {
11        c[i] = sqrtf(a[i] * a[i] + b[i] * b[i]);
12    }
13 }
14
15 // Verificar vectorización:
16 // gcc -O3 -mavx2 -fopt-info-vec-all

```

Código 9.5: Código para autovectorización

9.4.2 Funciones intrínsecas y autovectorización

Las funciones intrínsecas exponen operaciones específicas de una ISA mediante una sintaxis de C. Ofrecen control sobre cargas, máscaras y operaciones, pero aumentan el costo de mantenimiento y no garantizan superar al compilador. La autovectorización conserva un código más portable; las intrínsecas se justifican cuando la medición identifica una limitación y la transformación deseada no se expresa bien en código ordinario.

Tabla 9.2: Comparación de autovectorización y funciones intrínsecas

Aspecto	Autovectorización	Funciones intrínsecas
Portabilidad	Alta (código C estándar)	Baja (específico de ISA)
Rendimiento	Variable (depende del compilador)	Control explícito, no siempre óptimo
Complejidad	Baja	Media
Mantenimiento	Fácil	Más difícil
Control	Limitado	Total

9.5 Relación con el SO

9.5.1 Detección de capacidades SIMD

Comprobar únicamente el bit AVX de `CPUID` no es suficiente. La CPU debe admitir la extensión, el sistema operativo debe habilitar la conservación del estado extendido y `XGETBV`

debe indicar que los componentes necesarios están disponibles. Este procedimiento sigue las reglas descritas por los manuales de arquitectura x86 [15].

```

1  #include <stdbool.h>
2  #include <stdint.h>
3  #include <cpuid.h>
4
5  static uint64_t xgetbv0(void) {
6      uint32_t eax, edx;
7      __asm__ volatile("xgetbv" : "=a"(eax), "=d"(edx) : "c"(0));
8      return ((uint64_t)edx << 32) | eax;
9  }
10
11 bool has_avx(void) {
12     unsigned int eax, ebx, ecx, edx;
13     if (!__get_cpuid(1, &eax, &ebx, &ecx, &edx))
14         return false;
15     // La CPU debe ofrecer AVX y OSXSAVE; el SO debe habilitar XMM/YMM.
16     if (!(ecx & bit_AVX) || !(ecx & bit_OSXSAVE))
17         return false;
18     return (xgetbv0() & 0x6) == 0x6;
19 }
20
21 bool has_avx512f(void) {
22     unsigned int eax, ebx, ecx, edx;
23     if (!has_avx() || __get_cpuid_max(0, 0) < 7)
24         return false;
25     __cpuid_count(7, 0, eax, ebx, ecx, edx);
26     // AVX-512F y estado XMM, YMM, opmask y ZMM habilitado.
27     return (ebx & bit_AVX512F) && (xgetbv0() & 0xe6) == 0xe6;
28 }

```

Código 9.6: Detección de soporte SIMD en tiempo de ejecución

9.5.2 Estado vectorial y sistema operativo

La ejecución SIMD requiere cooperación entre el hardware, el sistema operativo y la aplicación. El núcleo habilita y conserva el estado vectorial durante interrupciones y cambios de contexto; la aplicación comprueba las capacidades antes de seleccionar una rutina optimizada. Además, una carga vectorial intensa puede comportarse de manera distinta según la frecuencia, la potencia y la microarquitectura del procesador. El ancho de una instrucción no debe interpretarse como una promesa de aceleración [14, 15].

9.6 Síntesis del capítulo

SIMD agrupa operaciones iguales sobre datos independientes. Su eficacia depende de que el algoritmo exponga esa independencia y de que la memoria entregue los datos con suficiente regularidad. Las dependencias entre iteraciones, las ramas divergentes y los accesos dispersos reducen la oportunidad de vectorizar.

El compilador puede transformar bucles ordinarios y ofrece informes para verificar la decisión.

Las funciones intrínsecas proporcionan control más directo, a costa de portabilidad y mantenimiento. En ambos casos, la aplicación debe seleccionar una ruta compatible y el sistema operativo debe habilitar y conservar el estado vectorial.

9.7 Preguntas de control del capítulo 9

1. Defina paralelismo de datos y dé dos ejemplos donde aplica SIMD.
2. ¿Cuándo NO es efectivo usar SIMD? Dé dos ejemplos.
3. Compare las extensiones SSE, AVX y AVX-512 en términos de ancho.
4. Explique la diferencia entre autovectorización y funciones intrínsecas.
5. ¿Por qué el SO necesita conocer las capacidades SIMD de cada CPU?

Capítulo 10

Procesadores multinúcleo y paralelismo entre hilos

Varios núcleos aumentan la capacidad de ejecutar hilos, pero introducen comunicación, coherencia y sincronización.

Al finalizar este capítulo, el estudiante podrá explicar por qué el crecimiento del rendimiento se desplazó hacia varios núcleos, distinguir la coherencia de caché de la consistencia de memoria y describir NUMA, afinidad y SMT desde la perspectiva del sistema operativo.

Un núcleo más rápido beneficia incluso a un programa secuencial. Varios núcleos, en cambio, solo aportan rendimiento cuando existen hilos que pueden progresar en paralelo. Este cambio traslada parte de la responsabilidad al software y al sistema operativo: hay que distribuir trabajo, ubicar datos, coordinar accesos y evitar que la comunicación consuma la ganancia esperada.

10.1 Motivación para el uso de varios núcleos

La transición hacia procesadores **multinúcleo** se relaciona con el límite de potencia. El escalamiento dejó de reducir el voltaje en la proporción necesaria para aumentar indefinidamente la frecuencia, mientras la potencia dinámica conserva una dependencia aproximada $P_{\text{din}} \propto CV^2f$. La disipación térmica y la potencia estática restringieron el crecimiento basado únicamente en frecuencia, y dedicar transistores a varios núcleos se convirtió en una alternativa práctica [6, 9].

10.1.1 Evolución de los procesadores

La Figura 10.1 muestra tendencias ilustrativas, no una base de datos de productos. La frecuencia dejó de crecer al ritmo observado en etapas anteriores, mientras aumentó el número de núcleos disponibles. Esto no eliminó las mejoras de un solo hilo: las microarquitecturas siguieron avanzando, pero dentro de presupuestos de energía más estrictos.

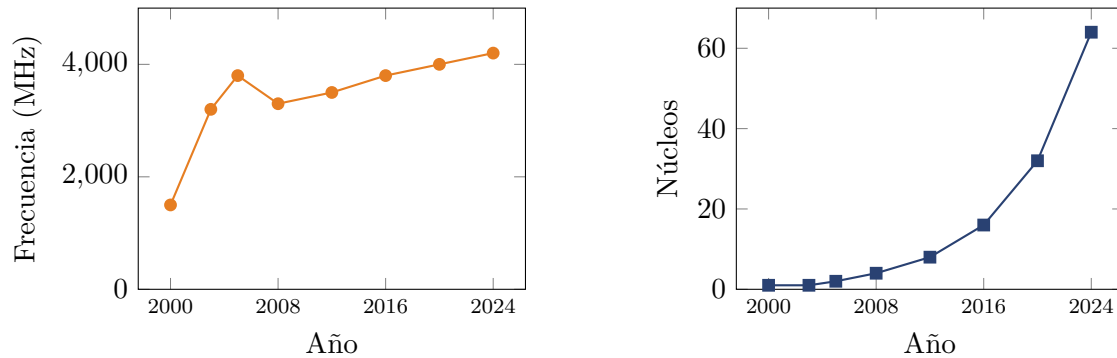


Figura 10.1: Tendencias ilustrativas de frecuencia y número de núcleos

10.2 Jerarquía de cachés privadas y compartidas

En muchos sistemas multinúcleo, las cachés cercanas a cada núcleo son privadas y algún nivel más alejado se comparte. La organización exacta depende de la microarquitectura.

Una caché privada reduce la latencia y evita que cada acceso compita con todos los núcleos. Una caché compartida facilita intercambiar datos y aprovechar la capacidad total, pero introduce contención. La jerarquía combina ambos objetivos y explica por qué la ubicación de un hilo puede cambiar su rendimiento.

10.2.1 Organización típica

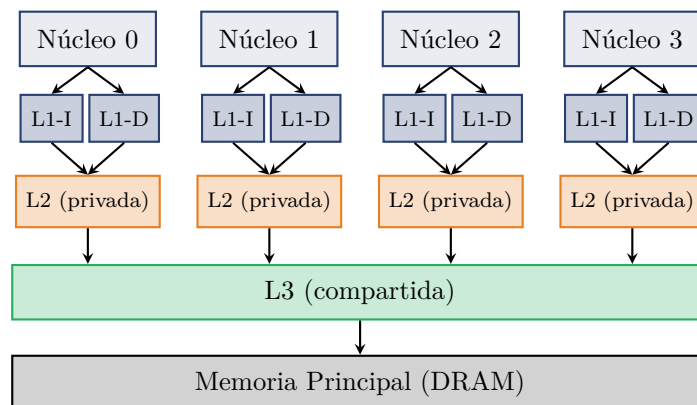


Figura 10.2: Ejemplo de jerarquía de cachés en un procesador de cuatro núcleos

10.3 Coherencia de caché

La **coherencia de caché** [23] establece cómo se propagan las escrituras cuando varios núcleos conservan copias de una misma línea. Un protocolo coherente mantiene un orden compatible para las escrituras sobre cada ubicación, pero no define por sí solo el orden de operaciones sobre ubicaciones diferentes.

10.3.1 Protocolo MESI

MESI representa una familia de protocolos de invalidación mediante cuatro estados. Una línea modificada contiene una versión distinta de la memoria; una línea exclusiva está limpia y solo aparece en una caché; una línea compartida puede coexistir en varias; y una línea inválida no puede utilizarse. Las implementaciones reales extienden o modifican este esquema, pero el modelo permite seguir una lectura y una escritura entre dos núcleos.

Tabla 10.1: Estados del protocolo MESI

Estado	Significado	Descripción
M (Modified)	Modificado	Línea modificada, solo en esta caché
E (Exclusive)	Exclusivo	Línea limpia, solo en esta caché
S (Shared)	Compartido	Línea limpia, puede estar en otras cachés
I (Invalid)	Inválido	Línea no válida, debe re-leer

10.3.2 Ejemplo de transiciones MESI

```
1 // Inicialmente: variable x = 10 en memoria
2
3 // Núcleo 0 lee x
4 // Estado en L1-C0: E (Exclusive)
5 // El núcleo 0 tiene la única copia válida
6
7 // Núcleo 1 lee x
8 // Núcleo 0 cambia: E -> S
9 // Núcleo 1 cambia: I -> S
10 // Ambos tienen copia compartida (Shared)
11
12 // Núcleo 0 escribe x = 20
13 // Núcleo 0: S -> M (Modified)
14 // Núcleo 1: S -> I (Invalid: invalidación)
15 // Memoria no actualizada todavía
16
17 // Núcleo 1 lee x
18 // Fallo de caché en el núcleo 1
19 // El dato actualizado se suministra según el protocolo
20 // Núcleo 0: M -> S
21 // Núcleo 1: I -> S
```

Código 10.1: Escenario de coherencia de caché

10.4 Consistencia de memoria

Concepto clave

El **modelo de consistencia de memoria** define qué órdenes de lecturas y escrituras pueden observar los hilos. La coherencia responde qué ocurre con copias de una misma ubicación; la consistencia establece relaciones entre operaciones, incluso sobre ubicaciones

diferentes [2, 15, 23, 26].

10.4.1 Modelos de consistencia

La consistencia secuencial sirve como modelo intuitivo: todas las operaciones parecen formar un orden único compatible con el orden de cada hilo. Las arquitecturas reales permiten ciertos reordenamientos para mejorar el rendimiento. x86-64 ofrece un modelo relativamente fuerte; Arm y RISC-V permiten más comportamientos y requieren instrucciones de ordenamiento adecuadas. Los lenguajes añaden su propio modelo, por lo que el programador debe usar primitivas de sincronización y no razonar únicamente desde la ISA.

Tabla 10.2: Modelos de consistencia de memoria

Modelo	Características	Ejemplo
Consistencia secuencial	Existe un orden total compatible con el orden de cada hilo	Modelo de referencia
TSO	Conserva varios órdenes; permite que una carga adelante a una escritura anterior distinta	x86-64
Modelos débiles	Permiten más reordenamientos observables y requieren barreras adecuadas	ARM, RISC-V
Adquirir/liberar	Ordena accesos alrededor de operaciones de sincronización	C11, C++11, Rust

10.4.2 Implicaciones para la programación

El ejemplo siguiente contiene accesos concurrentes sin la sincronización necesaria. En C o C++, una carrera de datos sobre objetos no atómicos produce comportamiento indefinido; no es válido enumerar resultados de hardware y asumir que el lenguaje los permite. Si las variables se modelan como atómicas relajadas, el caso sirve para mostrar que cada hilo puede observar un orden diferente. Un mutex o un orden atómico más fuerte establece las relaciones necesarias.

```

1 // Inicialmente: x = 0, y = 0
2
3 // Hilo 0:
4 x = 1; // Store a x
5 r1 = y; // Load de y
6
7 // Hilo 1:
8 y = 1; // Store a y
9 r2 = x; // Load de x
10
11 // Sin sincronización, este programa contiene una carrera de datos en C/C++.
12 // Si se modela con operaciones atómicas relajadas, pueden observarse:
13 // r1 = 0, r2 = 1 (esperado)
14 // r1 = 1, r2 = 0 (esperado)
15 // r1 = 1, r2 = 1 (esperado)
16 // r1 = 0, r2 = 0 (!!! posible con reordenamiento)
17
18 // Solución: usar operaciones atómicas con el orden apropiado

```



```
19 // o proteger los accesos mediante un mutex.
```

Código 10.2: Problema de consistencia de memoria

10.5 NUMA y afinidad

NUMA (*Non-Uniform Memory Access*) organiza CPU y memoria en nodos. El acceso a la memoria del nodo local suele tener menor latencia que el acceso a la memoria de otro nodo. [9, 18]

10.5.1 Arquitectura NUMA

En una máquina NUMA, cada nodo combina procesadores y memoria cercana. Todos los nodos participan en un espacio de direcciones compartido, pero la ruta hasta una página local suele ser más corta que hasta una remota. El sistema operativo decide en qué CPU ejecutar un hilo y dónde asignar físicamente sus páginas; una migración puede separar el cálculo de sus datos.

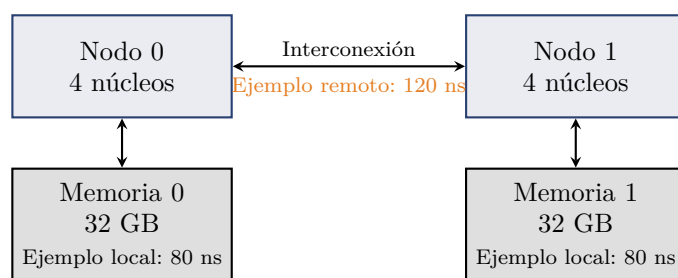


Figura 10.3: Sistema NUMA de dos nodos con latencias ilustrativas

10.5.2 Afinidad de procesos

La afinidad restringe las CPU donde puede ejecutarse un proceso o hilo. Puede preservar la localidad de caché y mantener el trabajo cerca de su memoria, pero una restricción excesiva impide balancear la carga. Debe utilizarse después de medir la topología y el comportamiento, no como una optimización automática [18].

```
# Ver topología NUMA
$ numactl --hardware
available: 2 nodes (0-1)
node 0 cpus: 0 1 2 3
node 0 size: 32768 MB
node 0 free: 28432 MB
node 1 cpus: 4 5 6 7
node 1 size: 32768 MB
node 1 free: 29120 MB

# Ejecutar proceso en nodo específico
$ numactl --cpunodebind=0 --membind=0 ./mi_programa
```

```
# Ver afinidad actual de un proceso
$ taskset -pc <PID>

# Fijar proceso a núcleos específicos
$ taskset -c 0,1,2,3 ./mi_programa
```

Código 10.3: Control de afinidad en Linux

10.6 SMT o multihilo simultáneo

SMT (*Simultaneous Multithreading*) [28], denominado Hyper-Threading por Intel, permite que un núcleo físico ejecute más de un hilo de hardware. Los hilos comparten recursos de ejecución, pero mantienen estados arquitectónicos separados.

10.6.1 Beneficios y limitaciones

SMT aprovecha ciclos en los que un hilo no puede utilizar todas las unidades. Al ofrecer instrucciones de otro hilo, puede aumentar la tasa de procesamiento del núcleo. Los dos contextos, sin embargo, comparten capacidad de ejecución y cachés; por eso la ganancia depende de que sus demandas se complementen y nunca equivale de forma general a duplicar los núcleos físicos.

Tabla 10.3: SMT: beneficios y limitaciones

Beneficio	Limitación
Mejor utilización de recursos	No equivale a añadir otro núcleo físico
Puede mejorar la tasa de procesamiento	La ganancia depende de la carga
Cambio entre hilos eficiente	Contención de recursos compartidos
Visible como CPU lógicas para el SO	Puede elevar la contención y el consumo

```
# Verificar si HT está habilitado
$ lscpu | grep "Thread(s) per core"
Thread(s) per core: 2

# Ver cores físicos vs lógicos
$ lscpu | grep -E "Core|Socket"
Core(s) per socket: 8
Socket(s): 1
# Total: 8 físicos * 2 = 16 lógicos

# Consultar el control global de SMT, si la plataforma lo ofrece
$ cat /sys/devices/system/cpu/smt/control
on
```

Código 10.4: Información sobre SMT en Linux

Modificar el control global de SMT requiere privilegios administrativos, afecta a todo el sistema y no está disponible en todas las plataformas. En equipos compartidos, el estudiante debe limitarse a consultar el estado.

10.7 Síntesis del capítulo

Los límites de potencia favorecieron el crecimiento mediante varios núcleos, pero el hardware paralelo no acelera automáticamente una aplicación. Los hilos comparten la jerarquía de memoria y necesitan protocolos que mantengan coherentes las copias de cada línea. Sobre esa base, el modelo de consistencia define qué órdenes puede observar el software y obliga a sincronizar los accesos concurrentes.

NUMA hace que la ubicación física de las páginas y de los hilos influya en la latencia. La afinidad puede preservar localidad, aunque también limitar el balanceo. SMT añade contextos arquitectónicos dentro de un núcleo y mejora su utilización cuando las cargas se complementan, pero mantiene compartidos los recursos de ejecución. El planificador debe conocer estas relaciones para tomar decisiones informadas.

10.8 Preguntas de control del capítulo 10

1. ¿Qué limitaciones físicas impulsaron la transición hacia procesadores multinúcleo?
2. Dibuje una jerarquía típica de cachés en un sistema de cuatro núcleos.
3. Explique los cuatro estados del protocolo MESI.
4. ¿Qué es NUMA y por qué es importante la afinidad de procesos?
5. Compare SMT con núcleos físicos: beneficios y limitaciones.

Capítulo 11

Hardware y sistemas operativos

El SO convierte mecanismos de hardware —privilegio, interrupciones, operaciones atómicas y DMA— en abstracciones seguras para los programas.

Al finalizar este capítulo, el estudiante podrá diferenciar interrupciones, excepciones y llamadas al sistema, describir el propósito y el costo de un cambio de contexto, y relacionar la planificación, las operaciones atómicas y DMA con mecanismos del procesador.

Los capítulos anteriores describieron mecanismos del procesador por separado. Este capítulo los reúne desde la perspectiva del núcleo. El sistema operativo necesita recuperar el control, proteger su estado, suspender una ejecución, coordinar varios núcleos y permitir que los dispositivos transfieran datos. Sus abstracciones —procesos, archivos, memoria virtual— se sostienen sobre esas capacidades de hardware.

11.1 Interrupciones, excepciones y llamadas al sistema

Las **interrupciones** y **excepciones** desvían el flujo normal hacia un manejador. Una interrupción suele provenir de un dispositivo y es asíncrona; una excepción está asociada con la instrucción en curso. Una llamada al sistema es una transición síncrona y controlada que solicita un servicio del núcleo. El evento puede ocurrir mientras la CPU ya está en modo privilegiado, por lo que no siempre implica un cambio desde usuario.

La distinción importa porque determina qué estado debe conservarse y si la instrucción puede reiniciarse. Un fallo de página, por ejemplo, es una excepción síncrona: el núcleo puede obtener la página y repetir el acceso. Una interrupción de red no está causada por la instrucción ejecutada en ese instante. Las reglas exactas de entrada y retorno pertenecen a la ISA [2, 15, 26].

11.1.1 Tipos de eventos

Tabla 11.1: Tipos de eventos y sus características

Tipo	Origen	Sincronía	Ejemplo
Interrupción	Dispositivo o temporizador	Asíncrona	Temporizador, teclado, red
Excepción	CPU	Síncrona	División por cero, fallo de página
Llamada al sistema	Software	Síncrona	Invocación de un servicio del SO

11.1.2 Manejo de interrupciones

Cuando la CPU acepta una interrupción, conserva el estado mínimo definido por la arquitectura y obtiene la dirección de un manejador. El código de entrada del núcleo guarda los registros adicionales que vaya a utilizar, atiende el evento y decide si puede regresar al contexto interrumpido. Una interrupción de temporizador también puede ofrecer al planificador la oportunidad de cambiar de hilo. El pseudocódigo siguiente separa estas responsabilidades sin imponer el mecanismo de una ISA concreta.

```

1 // 1. Ocurre una interrupción de hardware (p. ej., temporizador).
2 // 2. La CPU reconoce el evento en un punto definido por la ISA.
3 // 3. El hardware guarda el estado mínimo exigido por la arquitectura.
4 // 4. La CPU localiza el descriptor del manejador.
5 // 5. El código de entrada guarda el estado adicional necesario.
6 // 6. Se ejecuta el manejador en modo privilegiado.
7 // 7. Se restaura el estado y se retorna.
8
9 // Estructura del manejador (simplificado)
10 void timer_interrupt_handler() {
11     // Guardar registros adicionales
12     save_context();
13
14     // Actualizar contadores del planificador
15     update_scheduling_quantum();
16
17     // Decidir si hay cambio de contexto
18     if (should_reschedule()) {
19         schedule();
20     }
21
22     // Notificar fin de interrupción al controlador, si corresponde
23     send_eoi();
24
25     // Restaurar y retornar
26     restore_context();
27     return_from_interrupt();
28 }
```

Código 11.1: Flujo de manejo de interrupción

11.1.3 Llamadas al sistema

Una aplicación no puede ejecutar directamente operaciones privilegiadas. En su lugar prepara argumentos conforme a la ABI e invoca una instrucción de entrada al núcleo. El manejador valida el número de servicio y los datos procedentes del usuario antes de actuar. Esta validación es esencial: cambiar de modo no vuelve confiable la solicitud.

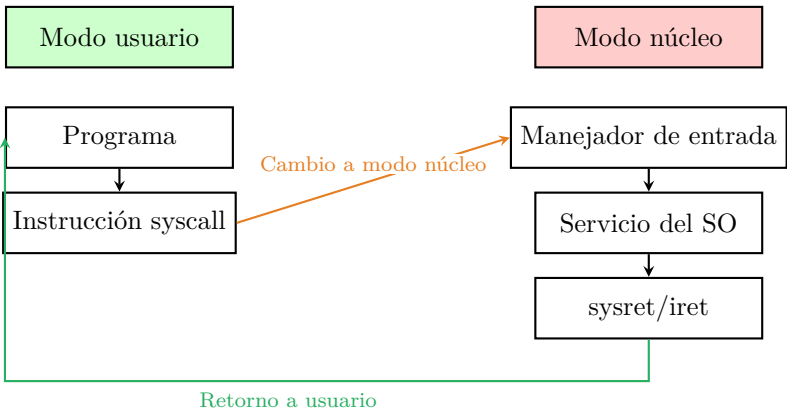


Figura 11.1: Flujo simplificado de una llamada al sistema

11.2 Modos de privilegio y protección

Los **modos de privilegio** restringen qué operaciones puede realizar el código en ejecución. El núcleo opera en el nivel más privilegiado; las aplicaciones, en uno menos privilegiado. [2, 15, 26]

La protección no depende solo de una convención de software. Si una aplicación intenta modificar las tablas de páginas o deshabilitar interrupciones, el procesador genera una excepción en lugar de ejecutar la operación. El núcleo conserva así el control de recursos que afectarían a todo el sistema.

11.2.1 Anillos de protección en x86

Tabla 11.2: Anillos de protección y su uso

Anillo	Nombre	Uso típico	Instrucciones
0	Núcleo	Sistema operativo	Todas
1	–	Poco usado en SO comunes	Restringidas
2	–	Poco usado en SO comunes	Restringidas
3	Usuario	Aplicaciones	Limitadas

11.2.2 Instrucciones privilegiadas

```
1 // Ejemplos ejecutables en el anillo 0:
2
```

```

3 // Control de interrupciones
4 CLI, STI // Su permiso depende de CPL e IOPL
5 LIDT // Cargar Interrupt Descriptor Table
6
7 // Control de memoria
8 MOV CR3, reg // Cambiar page table base
9 INVLPG // Invalidar entrada TLB
10
11 // Control de sistema
12 HLT // Detener CPU hasta interrupción
13 IN, OUT // Según IOPL y el mapa de permisos de E/S
14 RDMSR, WRMSR // Model Specific Registers
15
16 // Si una aplicación intenta una operación no autorizada:
17 // -> General Protection Fault (#GP)
18 // -> El SO decide si terminar el proceso o no

```

Código 11.2: Ejemplos de instrucciones privilegiadas

En x86, algunas instrucciones son siempre privilegiadas y otras dependen del nivel de privilegio y de permisos adicionales. Por ello, “ejecutable solo en el anillo 0” es una simplificación pedagógica, no una regla uniforme para todas las instrucciones de la lista [15].

11.3 Cambio de contexto y su costo

El **cambio de contexto** (*context switch*) guarda el estado necesario de un hilo e inicia o reanuda otro. Su costo directo y sus efectos sobre cachés, TLB y predictores dependen del procesador, el SO y la carga.

11.3.1 Estado conservado durante un cambio de contexto

El conjunto exacto depende del tipo de cambio y de la arquitectura. Para reanudar un hilo deben preservarse, como mínimo, el contador de programa, el puntero de pila, los indicadores y los registros que la convención o el núcleo consideren activos. Si cambia el proceso, puede cambiar también el espacio de direcciones. El estado de punto flotante y SIMD es mayor y algunas implementaciones pueden utilizar políticas diferentes para conservarlo. Los detalles dependen de la ISA y de la versión del núcleo [15, 17, 27].

```

1 struct context {
2     // Registros de propósito general
3     uint64_t rax, rbx, rcx, rdx;
4     uint64_t rsi, rdi, rbp, rsp;
5     uint64_t r8, r9, r10, r11;
6     uint64_t r12, r13, r14, r15;
7
8     // Registros de control
9     uint64_t rip; // Contador de programa
10    uint64_t rflags; // Indicadores de estado

```



```

11
12 // Registros de segmento
13 uint16_t cs, ds, ss, es, fs, gs;
14
15 // MMU: Tabla de páginas
16 uint64_t cr3; // Base de la tabla de páginas
17
18 // Estado de FPU/SIMD: tamaño y formato dependientes de la CPU
19 uint8_t extended_state[EXTENDED_STATE_SIZE];
20 };

```

Código 11.3: Estructura de contexto de proceso

11.3.2 Fuentes de costo

Guardar registros es solo el costo directo. Cuando el nuevo hilo utiliza otros datos y otro espacio de direcciones, puede desplazar líneas de caché, traducciones de la TLB y estado de predicción. Los identificadores de espacio de direcciones permiten conservar ciertas traducciones, pero no eliminan la pérdida de localidad. Por eso el costo observado varía con la carga y no se reduce a una constante de la CPU [8, 27].

Tabla 11.3: Fuentes de costo de un cambio de contexto

Fuente	Efecto
Estado arquitectónico	Guardar y restaurar registros y metadatos del hilo
Espacio de direcciones	Cambiar tablas de páginas; los identificadores de contexto pueden conservar entradas de TLB
Estado extendido	Conservar FPU, SIMD y otras extensiones cuando sea necesario
Efectos indirectos	Pérdida de localidad en cachés, TLB y predictores

11.4 Planificación en sistemas multinúcleo

El **planificador** del SO decide qué hilo se ejecuta en cada CPU y durante cuánto tiempo. En sistemas multinúcleo, debe considerar afinidad, balanceo de carga y contención de recursos.

11.4.1 Estrategias de planificación

En un sistema multinúcleo, elegir una CPU implica equilibrar objetivos. Mantener un hilo en el mismo núcleo favorece la localidad; moverlo puede aliviar una cola sobrecargada. El planificador también distingue núcleos físicos de hilos SMT, respeta afinidades y considera estados de energía. Ninguna estrategia es óptima para todas las cargas [18, 27].

Tabla 11.4: Estrategias de planificación en sistemas multinúcleo

Estrategia	Descripción
Afinidad de CPU	Mantener un hilo en el mismo núcleo para aprovechar la localidad
Balanceo de carga	Distribuir hilos entre núcleos disponibles
Migración	Mover hilos desde núcleos sobrecargados
Conocimiento de SMT	Distinguir hilos lógicos y núcleos físicos
Gestión energética	Reducir frecuencia o desactivar núcleos ociosos

```
# Ver afinidad de un proceso
$ taskset -p <PID>
pid <PID>'s current affinity mask: f

# La máscara 'f' (1111 binario) permite ejecutar en las CPU 0--3

# Fijar proceso a una CPU lógica específica
$ taskset -pc 2 <PID>

# Usar sched_setaffinity() en código
#define _GNU_SOURCE
#include <sched.h>

cpu_set_t mask;
CPU_ZERO(&mask);
CPU_SET(2, &mask); // Fijar a la CPU lógica 2
sched_setaffinity(0, sizeof(mask), &mask);
```

Código 11.4: Control de afinidad en Linux

11.5 Sincronización mediante primitivas atómicas

Las **primitivas atómicas** son operaciones indivisibles proporcionadas por el hardware que permiten implementar sincronización de alto nivel (mutexes, semáforos, etc.).

11.5.1 Operaciones atómicas fundamentales

Una operación atómica aparece ante los demás participantes como una transformación indivisible del objeto. Esa propiedad evita que dos hilos adquieran simultáneamente el mismo cerrojo, pero no define por sí sola el orden de todas las lecturas y escrituras. Las semánticas de adquisición y liberación del ejemplo establecen el orden necesario alrededor de la sección crítica [12, 23].

Tabla 11.5: Instrucciones atómicas comunes

Instrucción	Descripción	Uso
TAS	Probar y establecer	Cerrosos con espera activa
CAS	Comparar e intercambiar	Algoritmos sin cerrosos
LL/SC	Carga enlazada/almacenamiento condicional	Varias ISA RISC
XCHG	Intercambio	Implementación de cerrosos
FAA	Obtener y sumar	Contadores y colas

```

1  #include <stdatomic.h>
2
3  typedef atomic_flag spinlock_t;
4
5  void spinlock_acquire(spinlock_t *lock) {
6      while (atomic_flag_test_and_set_explicit(
7          lock, memory_order_acquire)) {
8          // En x86, PAUSE reduce el impacto de la espera activa.
9          __asm__ volatile("pause");
10     }
11 }
12
13 void spinlock_release(spinlock_t *lock) {
14     atomic_flag_clear_explicit(lock, memory_order_release);
15 }
16
17 spinlock_t lock = ATOMIC_FLAG_INIT;

```

Código 11.5: Cerrojo con espera activa mediante C11

11.6 E/S y DMA

El **acceso directo a memoria** (DMA, *Direct Memory Access*) permite que un dispositivo transfiera datos desde o hacia la memoria sin que la CPU copie cada elemento. El controlador configura búferes y direcciones válidas; el dispositivo realiza la transferencia y normalmente notifica su finalización. En sistemas con IOMMU, la dirección utilizada por el dispositivo puede traducirse antes de llegar a la memoria física [22, 24].

11.6.1 Comparación de E/S programada y DMA

En la E/S programada, la CPU participa repetidamente en el movimiento de datos. DMA traslada ese trabajo al dispositivo o a su controlador y resulta apropiado para bloques grandes. La CPU no desaparece del proceso: prepara descriptores, sincroniza el acceso al búfer, maneja errores y responde a la finalización. Además, la coherencia entre caché y dispositivo debe respetar las reglas de la plataforma y de la API del núcleo.

Tabla 11.6: Comparación de métodos de E/S

Aspecto	E/S Programada	DMA
Uso de CPU	Interviene en cada transferencia	Configuración y finalización
Velocidad	Limitada por CPU	Limitada por bus
Tamaño ideal	Pequeño	Grande
Ejemplo	Puerto serie, teclado	Disco, red, gráficos

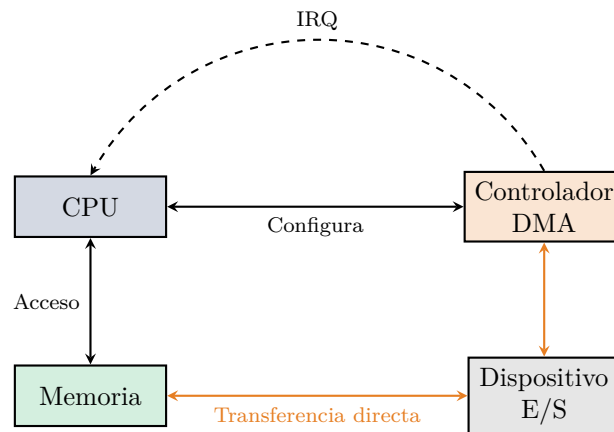


Figura 11.2: Arquitectura DMA

11.7 Síntesis del capítulo

El hardware transfiere el control al núcleo mediante interrupciones, excepciones y llamadas al sistema. Los modos de privilegio impiden que una aplicación configure directamente recursos globales, y las reglas de entrada permiten atender un evento sin perder el estado de la ejecución interrumpida. Sobre esa base, el sistema operativo puede suspender hilos y planificar otros.

Un cambio de contexto incluye el estado arquitectónico y puede perturbar cachés y traducciones. En sistemas multinúcleo, el planificador equilibra localidad y carga; las operaciones atómicas permiten coordinar estructuras compartidas; y DMA evita que la CPU copie cada dato de E/S, aunque exige configuración y sincronización. Las abstracciones del sistema dependen de la composición correcta de todos estos mecanismos.

11.8 Preguntas de control del capítulo 11

1. Compare interrupciones, excepciones y llamadas al sistema. ¿Cuándo ocurre cada una?
2. ¿Qué son los anillos de protección y qué instrucciones son privilegiadas?
3. Enumere los costos asociados con un cambio de contexto.
4. Explique tres estrategias de planificación en sistemas multinúcleo.
5. ¿Qué es DMA y por qué es importante para el rendimiento del sistema?

Capítulo 12

Talleres y prácticas sugeridas

Medir no es solo cronometrar; exige controlar la carga, repetir las pruebas y declarar qué incluye cada resultado.

Al finalizar estos talleres, el estudiante podrá diseñar mediciones reproducibles y reconocer sus fuentes de ruido, comparar localidad, vectorización y escalamiento con cargas equivalentes, y comunicar resultados con unidades, variabilidad y limitaciones.

Las prácticas de este capítulo no buscan obtener un número “oficial” para la máquina. Su propósito es aprender a formular una pregunta medible, controlar las condiciones y explicar el resultado. Antes de ejecutar cada programa, el estudiante debe anticipar qué espera observar y qué factores podrían contradecir esa expectativa. Después debe repetir la prueba, conservar los datos originales y distinguir evidencia de interpretación.

Todos los resultados dependen del equipo, del compilador, del núcleo y de la carga concurrente. Por ello, el informe debe registrar al menos el modelo de CPU, la versión del sistema y del compilador, las opciones utilizadas y el número de repeticiones. Las cifras solo son comparables cuando representan el mismo trabajo [8, 20].

12.1 Medición de llamadas al sistema y cambios de contexto

12.1.1 Objetivo

Estimar el costo de una llamada al sistema y de una transferencia entre procesos. Ambos ejercicios miden más que el mecanismo aislado; el informe debe explicitar esos costos adicionales.

La primera medición repite una llamada sencilla para que el costo del temporizador y del bucle se distribuya entre muchas observaciones. La segunda crea un intercambio mediante tuberías que obliga a coordinar dos procesos. No debe describirse su resultado como el costo puro de un cambio de contexto, porque también incluye llamadas al sistema, transferencia de un byte y decisiones del planificador [19, 27].

12.1.2 Código de referencia

```
1 #define _GNU_SOURCE
```

```

2  #include <stdio.h>
3  #include <stdlib.h>
4  #include <stdint.h>
5  #include <time.h>
6  #include <unistd.h>
7  #include <sys/syscall.h>
8
9  static uint64_t now_ns(void) {
10     struct timespec ts;
11     if (clock_gettime(CLOCK_MONOTONIC_RAW, &ts) != 0) {
12         perror("clock_gettime");
13         exit(EXIT_FAILURE);
14     }
15     return (uint64_t)ts.tv_sec * 1000000000ull + ts.tv_nsec;
16 }
17
18 int main(void) {
19     const long iterations = 1000000;
20     volatile long sink = 0;
21
22     // Calentamiento
23     for (int i = 0; i < 1000; i++) {
24         sink += syscall(SYS_getpid);
25     }
26
27     uint64_t start = now_ns();
28     for (long i = 0; i < iterations; i++) {
29         // syscall evita una posible optimización de la biblioteca.
30         sink += syscall(SYS_getpid);
31     }
32     uint64_t elapsed = now_ns() - start;
33
34     printf("Promedio: %.2f ns/llamada\n",
35           (double)elapsed / iterations);
36     return EXIT_SUCCESS;
37 }

```

Código 12.1: Tiempo promedio de una llamada al sistema

```

1  #define _GNU_SOURCE
2  #include <stdio.h>
3  #include <stdlib.h>
4  #include <stdint.h>
5  #include <sched.h>
6  #include <errno.h>
7  #include <time.h>
8  #include <unistd.h>
9  #include <sys/wait.h>
10
11 static uint64_t now_ns(void) {
12     struct timespec ts;
13     clock_gettime(CLOCK_MONOTONIC_RAW, &ts);
14     return (uint64_t)ts.tv_sec * 1000000000ull + ts.tv_nsec;
15 }
16
17 static void pin_to_one_cpu(void) {
18     cpu_set_t available, target;
19     if (sched_getaffinity(0, sizeof available, &available) != 0) {

```

```
20     perror("sched_getaffinity");
21     return;
22 }
23 int cpu;
24 for (cpu = 0; cpu < CPU_SETSIZE; cpu++)
25     if (CPU_ISSET(cpu, &available))
26         break;
27 if (cpu == CPU_SETSIZE)
28     return;
29
30 CPU_ZERO(&target);
31 CPU_SET(cpu, &target);
32 if (sched_setaffinity(0, sizeof target, &target) != 0)
33     perror("sched_setaffinity");
34 }
35
36 static void transfer(int fd, char *token, int write_op) {
37     ssize_t result;
38     do {
39         result = write_op ? write(fd, token, 1)
40                          : read(fd, token, 1);
41     } while (result < 0 && errno == EINTR);
42     if (result != 1) {
43         perror(write_op ? "write" : "read");
44         exit(EXIT_FAILURE);
45     }
46 }
47
48 int main(void) {
49     int parent_to_child[2], child_to_parent[2];
50     const int iterations = 100000;
51
52     if (pipe(parent_to_child) || pipe(child_to_parent)) {
53         perror("pipe");
54         return EXIT_FAILURE;
55     }
56
57     pid_t pid = fork();
58     if (pid < 0) {
59         perror("fork");
60         return EXIT_FAILURE;
61     }
62
63     if (pid == 0) {
64         pin_to_one_cpu();
65         close(parent_to_child[1]);
66         close(child_to_parent[0]);
67         char token;
68         for (int i = 0; i < iterations; i++) {
69             transfer(parent_to_child[0], &token, 0);
70             transfer(child_to_parent[1], &token, 1);
71         }
72         _exit(EXIT_SUCCESS);
73     }
74
75     pin_to_one_cpu();
76     close(parent_to_child[0]);
77     close(child_to_parent[1]);
```

```

78     char token = 1;
79
80     uint64_t start = now_ns();
81     for (int i = 0; i < iterations; i++) {
82         transfer(parent_to_child[1], &token, 1);
83         transfer(child_to_parent[0], &token, 0);
84     }
85     uint64_t elapsed = now_ns() - start;
86
87     waitpid(pid, NULL, 0);
88     printf("Promedio por transferencia: %.2f ns\n",
89           (double)elapsed / (2.0 * iterations));
90     return EXIT_SUCCESS;
91 }

```

Código 12.2: Transferencia entre procesos mediante tuberías

El segundo programa no aísla el cambio de contexto: también incluye operaciones sobre tuberías, entrada al núcleo y planificación. Fijar ambos procesos a una CPU lógica favorece la alternancia, pero no elimina el ruido. Ejecute cada prueba al menos diez veces y reporte mediana, mínimo y dispersión.

12.1.3 Preguntas de análisis

1. ¿Por qué el tiempo de `getpid` varía entre ejecuciones?
2. ¿Qué partes del resultado de las tuberías no corresponden al cambio de contexto?
3. ¿Cómo cambian los datos al fijar la afinidad, aumentar las repeticiones o cargar el sistema?

12.2 Efecto del paso de acceso en la caché

12.2.1 Objetivo

Observar cómo cambia el tiempo por acceso cuando aumenta el paso entre elementos de un arreglo.

El arreglo es mayor que las cachés habituales y se recorre con pasos crecientes. Con pasos pequeños, una línea traída desde memoria aporta varios elementos útiles; cuando el paso aumenta, se aprovecha una fracción menor. La forma de la curva también depende del prefetcher, del TLB y de la cantidad de accesos, por lo que el experimento no identifica automáticamente los tamaños de cada caché.

```

1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <stdint.h>
4  #include <time.h>
5
6  #define ARRAY_BYTES (64ull * 1024 * 1024)
7  #define TARGET_ACCESSES (32ull * 1024 * 1024)
8

```



```

9  static uint64_t now_ns(void) {
10     struct timespec ts;
11     clock_gettime(CLOCK_MONOTONIC_RAW, &ts);
12     return (uint64_t)ts.tv_sec * 1000000000ull + ts.tv_nsec;
13 }
14
15 int main(void) {
16     const size_t count = ARRAY_BYTES / sizeof(uint32_t);
17     uint32_t *array = malloc(ARRAY_BYTES);
18     if (!array) {
19         perror("malloc");
20         return EXIT_FAILURE;
21     }
22
23     // Inicialización por demanda: asigna físicamente las páginas.
24     for (size_t i = 0; i < count; i++)
25         array[i] = (uint32_t)i;
26
27     printf("paso_bytes,ns_por_acceso\n");
28     volatile uint64_t sum = 0;
29
30     for (size_t step = 1; step <= 8192; step *= 2) {
31         size_t accesses_per_pass = (count + step - 1) / step;
32         size_t passes = TARGET_ACCESSES / accesses_per_pass + 1;
33         uint64_t accesses = 0;
34
35         uint64_t start = now_ns();
36         for (size_t pass = 0; pass < passes; pass++) {
37             for (size_t i = 0; i < count; i += step) {
38                 sum += array[i];
39                 accesses++;
40             }
41         }
42         uint64_t elapsed = now_ns() - start;
43
44         printf("%zu,%.3f\n", step * sizeof(uint32_t),
45             (double)elapsed / accesses);
46     }
47
48     fprintf(stderr, "control=%llu\n", (unsigned long long)sum);
49     free(array);
50     return EXIT_SUCCESS;
51 }

```

Código 12.3: Experimento de paso de acceso y caché

12.2.2 Interpretación

No asigne cada cambio de la curva a L1, L2 o L3 únicamente a partir del paso. El resultado también refleja el tamaño de línea, la precarga por hardware, el TLB y el patrón de recorrido. Grafique sus propios datos y complemente la medición con:

```

perf stat -r 5 -e
cache-references,cache-misses,dTLB-loads,dTLB-load-misses ./stride

```

12.3 Comparación escalar y vectorial

12.3.1 Objetivo

Comparar una implementación escalar no vectorizada con una implementación AVX2, verificar que producen el mismo resultado y analizar por qué la aceleración observada no tiene que ser ocho.

Ambas funciones realizan el mismo número de sumas y escriben el mismo volumen de datos. El atributo de GCC evita que la referencia escalar se vectorice de forma automática; sin esa precaución, la comparación podría enfrentar dos versiones vectoriales. Aun así, el resultado puede quedar limitado por el ancho de banda de memoria, la frecuencia o el costo de las transferencias.

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <math.h>
4  #include <time.h>
5  #include <immintrin.h>
6
7  #define N (10 * 1024 * 1024)
8  #define REPEATS 7
9
10 struct stats {
11     double median, minimum, maximum;
12 };
13
14 static double now_s(void) {
15     struct timespec ts;
16     clock_gettime(CLOCK_MONOTONIC_RAW, &ts);
17     return ts.tv_sec + ts.tv_nsec * 1e-9;
18 }
19
20 __attribute__((noinline, optimize("no-tree-vectorize")))
21 void suma_escalar(const float *a, const float *b, float *c, size_t n) {
22     for (size_t i = 0; i < n; i++)
23         c[i] = a[i] + b[i];
24 }
25
26 __attribute__((noinline))
27 void suma_simd(const float *a, const float *b, float *c, size_t n) {
28     size_t i = 0;
29     for (; i + 7 < n; i += 8) {
30         __m256 va = _mm256_load_ps(&a[i]);
31         __m256 vb = _mm256_load_ps(&b[i]);
32         __m256 vc = _mm256_add_ps(va, vb);
33         _mm256_store_ps(&c[i], vc);
34     }
35     for (; i < n; i++)
36         c[i] = a[i] + b[i];
37 }
38
39 static struct stats measure(
40     void (*fn)(const float *, const float *, float *, size_t),
41     const float *a, const float *b, float *c) {
42     double samples[REPEATS];
43 }
```

```

44 // Calentamiento fuera de la región medida.
45 fn(a, b, c, N);
46
47 for (int r = 0; r < REPEATS; r++) {
48     double start = now_s();
49     fn(a, b, c, N);
50     samples[r] = now_s() - start;
51 }
52
53 // Ordenamiento por inserción: REPEATS es pequeño.
54 for (int i = 1; i < REPEATS; i++) {
55     double value = samples[i];
56     int j = i;
57     while (j > 0 && samples[j - 1] > value) {
58         samples[j] = samples[j - 1];
59         j--;
60     }
61     samples[j] = value;
62 }
63
64 struct stats result = {
65     .median = samples[REPEATS / 2],
66     .minimum = samples[0],
67     .maximum = samples[REPEATS - 1]
68 };
69 return result;
70 }
71
72 int main(void) {
73     size_t bytes = N * sizeof(float);
74     float *a = aligned_alloc(32, bytes);
75     float *b = aligned_alloc(32, bytes);
76     float *scalar = aligned_alloc(32, bytes);
77     float *simd = aligned_alloc(32, bytes);
78     if (!a || !b || !scalar || !simd) {
79         perror("aligned_alloc");
80         return EXIT_FAILURE;
81     }
82
83     for (size_t i = 0; i < N; i++) {
84         a[i] = (float)i;
85         b[i] = (float)(N - i);
86     }
87
88     struct stats scalar_stats =
89         measure(suma_escalar, a, b, scalar);
90     struct stats simd_stats =
91         measure(suma_simd, a, b, simd);
92
93     for (size_t i = 0; i < N; i++) {
94         if (fabsf(scalar[i] - simd[i]) > 1e-6f) {
95             fprintf(stderr, "Diferencia en %zu\n", i);
96             return EXIT_FAILURE;
97         }
98     }
99
100     printf("Escalar: mediana %.4f s [%.4f, %.4f]\n",
101         scalar_stats.median,

```

```

102     scalar_stats.minimum, scalar_stats.maximum);
103     printf("AVX2: mediana %.4f s [%.4f, %.4f]\n",
104           simd_stats.median,
105           simd_stats.minimum, simd_stats.maximum);
106     printf("Aceleración: %.2fx\n",
107           scalar_stats.median / simd_stats.median);
108
109     free(a); free(b); free(scalar); free(simd);
110     return EXIT_SUCCESS;
111 }

```

Código 12.4: Comparación reproducible de código escalar y AVX2

Compruebe primero que la CPU y el sistema operativo permiten utilizar AVX2, siguiendo la detección explicada en el Capítulo 9; observar una bandera en `lscpu` no sustituye esa comprobación. Compile con `gcc -O3 -mavx2 -std=gnu11 simd.c -lm -o simd`. El atributo usado para impedir la autovectorización escalar es específico de GCC. Registre la versión del compilador y examine el informe de vectorización [7, 15].

12.4 Escalamiento multinúcleo

12.4.1 Objetivo

Medir la aceleración al aumentar el número de hilos, manteniendo constante el número total de elementos procesados.

Mantener constante el tamaño del arreglo convierte la prueba en un experimento de escalamiento fuerte. Cada hilo recibe una fracción menor del mismo trabajo y el tiempo se compara con la ejecución de un solo hilo. Si el ancho de banda de memoria se satura, añadir núcleos deja de mejorar el resultado aunque todavía exista capacidad de cómputo.

```

1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <stdint.h>
4  #include <pthread.h>
5  #include <time.h>
6
7  #define N (64ull * 1024 * 1024)
8  #define MAX_THREADS 16
9
10 static uint32_t *array;
11
12 struct task {
13     size_t begin, end;
14     uint64_t sum;
15 };
16
17 static double now_s(void) {
18     struct timespec ts;
19     clock_gettime(CLOCK_MONOTONIC_RAW, &ts);
20     return ts.tv_sec + ts.tv_nsec * 1e-9;
21 }
22

```

```

23 static void *worker(void *arg) {
24     struct task *task = arg;
25     uint64_t sum = 0;
26     for (size_t i = task->begin; i < task->end; i++)
27         sum += array[i];
28     task->sum = sum;
29     return NULL;
30 }
31
32 static double run_with_threads(int num_threads, uint64_t *total) {
33     pthread_t threads[MAX_THREADS];
34     struct task tasks[MAX_THREADS];
35
36     double start = now_s();
37     for (int i = 0; i < num_threads; i++) {
38         tasks[i].begin = (N * (size_t)i) / num_threads;
39         tasks[i].end = (N * (size_t)(i + 1)) / num_threads;
40         tasks[i].sum = 0;
41         if (pthread_create(&threads[i], NULL, worker, &tasks[i])) {
42             fprintf(stderr, "No se pudo crear el hilo %d\n", i);
43             exit(EXIT_FAILURE);
44         }
45     }
46
47     *total = 0;
48     for (int i = 0; i < num_threads; i++) {
49         pthread_join(threads[i], NULL);
50         *total += tasks[i].sum;
51     }
52     return now_s() - start;
53 }
54
55 int main(void) {
56     array = malloc(N * sizeof *array);
57     if (!array) {
58         perror("malloc");
59         return EXIT_FAILURE;
60     }
61     for (size_t i = 0; i < N; i++)
62         array[i] = (uint32_t)(i % 100);
63
64     printf("hilos,tiempo_s,aceleracion,control\n");
65
66     uint64_t expected, total;
67     double time_single = run_with_threads(1, &expected);
68     printf("1,%.6f,1.00,%llu\n", time_single,
69         (unsigned long long)expected);
70
71     for (int t = 2; t <= MAX_THREADS; t *= 2) {
72         double elapsed = run_with_threads(t, &total);
73         if (total != expected) {
74             fprintf(stderr, "Resultado incorrecto con %d hilos\n", t);
75             return EXIT_FAILURE;
76         }
77         printf("%d,%.6f,%.2f,%llu\n", t, elapsed,
78             time_single / elapsed, (unsigned long long)total);
79     }
80

```

```

81     free(array);
82     return EXIT_SUCCESS;
83 }

```

Código 12.5: Experimento de escalamiento multinúcleo

La suma recorre unos 256 MiB y suele estar limitada por el ancho de banda de memoria. Por ello, la aceleración puede saturarse mucho antes de alcanzar el número máximo de núcleos. Compile con `gcc -O3 -pthread -std=gnu11 multicore.c -o multicore` y repita cada configuración.

El intervalo medido incluye `pthread_create` y `pthread_join`. Por tanto, representa el costo de la estrategia completa, no solo el recorrido paralelo del arreglo. Para aislar mejor el cómputo deben reutilizarse los hilos, sincronizarlos mediante una barrera y reportar mediana y dispersión.

12.5 Informe técnico

12.5.1 Estructura del informe

Estructura del informe técnico

El resumen, con un máximo de 200 palabras, presenta la pregunta, el resultado principal y la conclusión. La introducción formula la hipótesis y conecta el experimento con los conceptos del curso. La sección de método identifica el hardware, el software, las opciones de compilación, el procedimiento y las repeticiones; el código completo puede quedar en un apéndice.

Los resultados deben conservar unidades, mostrar la variabilidad y utilizar tablas o gráficos con ejes claramente etiquetados. El análisis separa lo observado de sus posibles explicaciones y reconoce las amenazas a la validez. La conclusión responde la hipótesis inicial, relaciona la evidencia con la teoría y propone una mejora concreta del experimento.

12.5.2 Ejemplo de tabla de resultados

Tabla 12.1: Formato de tabla de resultados

Configuración	Tiempo (s)	Desviación	Aceleración
1 hilo	10.50	0.05	1.00
2 hilos	5.40	0.08	1.94
4 hilos	2.90	0.12	3.62
8 hilos	1.80	0.15	5.83

Capítulo 13

Síntesis y cierre

13.1 Resumen integrador

A lo largo de esta guía recorrimos un camino que conecta los fundamentos de la arquitectura de computadores con sus implicaciones para el diseño y la operación de los sistemas operativos.

El punto de partida fue un procesador que obtiene y ejecuta instrucciones. A partir de ese modelo se introdujeron la ISA, la memoria y la segmentación; luego se estudió cómo una microarquitectura encuentra paralelismo entre instrucciones, datos e hilos. El recorrido terminó en los mecanismos que el núcleo utiliza para recuperar el control, proteger recursos y coordinar dispositivos. Cada nivel añade complejidad, pero también conserva una interfaz que permite razonar sin conocer todos los detalles del nivel inferior.

13.1.1 El puente entre arquitectura y SO

La Figura 13.1 organiza esa relación como una cadena de abstracciones. La ISA no sustituye al hardware, sino que define la parte que el software puede observar. El sistema operativo utiliza ese contrato y ofrece a las aplicaciones interfaces aún más estables, como procesos, archivos y espacios de direcciones.

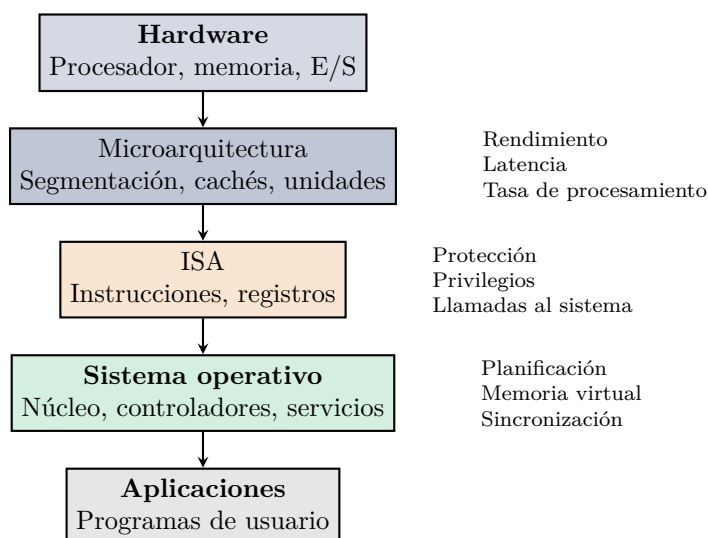


Figura 13.1: La cadena de abstracciones del computador

13.1.2 Conceptos clave por capítulo

Tabla 13.1: Resumen de conceptos clave por capítulo

Capítulo	Conceptos fundamentales
0. Guía de estudio	Ruta de lectura, prerequisites y entorno
1. Arquitectura y SO	CPU–memoria–E/S, modos de ejecución
2. Modelo de ejecución	Búsqueda, decodificación, ejecución y latencias
3. Von Neumann y Harvard	Cuello de botella y cachés separadas
4. ISA y microarquitectura	Contrato entre software y hardware
5. CISC y RISC	Filosofías de diseño y convergencia
6. Segmentación	Riesgos, reenvío y predicción
7. Métricas	CPI, IPC y ley de Amdahl
8. Escalar y superescalar	ILP, especulación y ejecución fuera de orden
9. SIMD	Paralelismo de datos, vectorización
10. Multinúcleo	Coherencia, consistencia y NUMA
11. Implicaciones para el SO	Interrupciones, cambio de contexto y DMA

13.2 Errores típicos de los estudiantes

Una métrica sin contexto conduce a conclusiones débiles. Latencia y tasa de procesamiento responden preguntas diferentes; por eso “más rápido” debe sustituirse por una medida y una carga concretas.

El cálculo no es el único costo. Optimizar instrucciones sin observar los accesos a memoria puede empeorar el tiempo total. La localidad, los fallos de caché y el ancho de banda deben formar parte de la explicación.

El paralelismo no escala de forma perfecta. Ocho núcleos no garantizan una aceleración de ocho. La fracción secuencial, la comunicación y la contención establecen límites que deben contrastarse con la ley de Amdahl y con mediciones.

Un hilo lógico no es un núcleo adicional. SMT comparte unidades y cachés dentro de un núcleo físico. Puede mejorar la utilización, pero la ganancia depende de las cargas que coinciden.

La concurrencia necesita un modelo de memoria. No es válido asumir que todos los hilos observarán las operaciones en el orden escrito. Las primitivas atómicas y los cerrojos establecen las relaciones que el programa requiere.

13.3 Ruta de profundización

13.3.1 Para estudiantes interesados en sistemas operativos

Una continuación natural es *Operating Systems: Three Easy Pieces* (OSTEP), disponible gratuitamente en el sitio de sus autores. El libro organiza el estudio alrededor de virtualización, concurrencia y persistencia [3]. Para convertir la lectura en experiencia, *xv6* ofrece un núcleo educativo pequeño que permite seguir llamadas al sistema, procesos y memoria virtual. El núcleo Linux constituye una etapa posterior: su documentación y su código muestran cómo esas ideas se adaptan a una plataforma real [17].

13.3.2 Para estudiantes interesados en arquitectura

Computer Architecture: A Quantitative Approach profundiza en memoria, paralelismo a nivel de instrucción y multiprocesadores mediante análisis cuantitativo [9]. RISC-V ofrece una ISA abierta y especificaciones públicas que permiten estudiar una arquitectura sin depender de descripciones informales [26]. Una implementación educativa o un simulador como gem5 permite experimentar con cachés y segmentación sin confundir el modelo con el comportamiento de una CPU comercial.

13.3.3 Para estudiantes interesados en rendimiento

Systems Performance, de Brendan Gregg, presenta un enfoque metódico para formular hipótesis y analizar sistemas completos [8]. En Linux, **perf** permite comenzar con contadores y perfiles; eBPF amplía la observabilidad del núcleo, y **valgrind/cachegrind** ofrece un modelo instrumentado de caché. Herramientas específicas como Intel VTune pueden añadirse cuando la pregunta justifique depender de una plataforma.

13.3.4 Recursos adicionales

Tabla 13.2: Recursos adicionales recomendados

Recurso	Tipo	URL
OSTEP	Libro gratuito	pages.cs.wisc.edu/~remzi/OSTEP
RISC-V Spec	Especificación	riscv.org
Intel SDM	Documentación	intel.com/sdm
ARM ARM	Documentación	developer.arm.com
Linux Kernel	Código fuente	kernel.org
Gem5	Simulador	gem5.org

13.4 Mensaje final

Comprender la arquitectura no consiste en memorizar nombres de registros o generaciones de procesadores. Consiste en construir un modelo que conecte una instrucción con el estado que modifica, una dirección con la jerarquía que la resuelve y un hilo con los recursos que comparte. Ese modelo permite formular mejores preguntas sobre corrección y rendimiento.

Al estudiar sistemas operativos, esta perspectiva resulta especialmente útil. Una abstracción no elimina el hardware; organiza su complejidad y establece reglas para utilizarlo con seguridad. Saber dónde termina cada contrato ayuda a diagnosticar problemas, interpretar mediciones y diseñar software que coopere con la máquina en lugar de depender de suposiciones.

— Fin de la guía —

Capítulo 14

Referencias y recursos adicionales

Las siguientes fuentes fueron consultadas para la elaboración de esta guía:

14.1 Bibliografía

- [1] Gene M. Amdahl. Validity of the single processor approach to achieving large scale computing capabilities. In *Proceedings of the April 18-20, 1967, Spring Joint Computer Conference*, AFIPS '67 (Spring), pages 483–485, New York, NY, 1967. ACM. doi: 10.1145/1465482.1465560.
- [2] ARM Limited. *ARM Architecture Reference Manual for A-profile*. ARM Limited, 2026. URL <https://developer.arm.com/documentation/ddi0487/latest/>. ARM DDI 0487; living document consulted 2026-07-28.
- [3] Remzi H. Arpaci-Dusseau and Andrea C. Arpaci-Dusseau. *Operating Systems: Three Easy Pieces*. Arpaci-Dusseau Books, 1.00 edition, 2018. ISBN 978-1985086593.
- [4] Randal E. Bryant and David R. O'Hallaron. *Computer Systems: A Programmer's Perspective*. Pearson, Boston, MA, 3rd edition, 2016. ISBN 978-0134092669.
- [5] David R. Ditzel and David A. Patterson. Retrospective on high-level language computer architecture. In *Proceedings of the 7th Annual Symposium on Computer Architecture*, pages 97–104. ACM, 1980. doi: 10.1145/800053.801914.
- [6] Hadi Esmaeilzadeh, Emily Blem, Renée St. Amant, Karthikeyan Sankaralingam, and Doug Burger. Dark silicon and the end of multicore scaling. In *Proceedings of the 38th Annual International Symposium on Computer Architecture*, pages 365–376. ACM, 2011. doi: 10.1145/2000064.2000108.
- [7] Free Software Foundation. *Using the GNU Compiler Collection: Options That Control Optimization*. GNU Project, 2026. URL <https://gcc.gnu.org/onlinedocs/gcc/Optimize-Options.html>. Online documentation; consulted 2026-07-28.
- [8] Brendan Gregg. *Systems Performance: Enterprise and the Cloud*. Addison-Wesley Professional, 2nd edition, 2020. ISBN 978-0136821656.
- [9] John L. Hennessy and David A. Patterson. *Computer Architecture: A Quantitative Approach*. Morgan Kaufmann, Cambridge, MA, 6th edition, 2019. ISBN 978-0128119051.

- [10] John L. Hennessy, Norman P. Jouppi, Steven Przybylski, Christopher Rowen, Thomas Gross, Forest Baskett, and John Gill. MIPS: A microprocessor architecture. *ACM SIGMICRO Newsletter*, 13(4):17–22, 1982. doi: 10.1145/1014194.800930.
- [11] Mark D. Hill and Michael R. Marty. Amdahl’s law in the multicore era. *Computer*, 41(7):33–38, 2008. doi: 10.1109/MC.2008.209.
- [12] David Howells, Paul E. McKenney, Will Deacon, and Peter Zijlstra. *Linux Kernel Memory Barriers*. Linux Kernel Documentation, 2026. URL <https://docs.kernel.org/core-api/wrappers/memory-barriers.html>. Consulted 2026-07-28.
- [13] Intel Corporation. *Intel Intrinsics Guide*. Intel Corporation, 2024. URL <https://www.intel.com/content/www/us/en/docs/intrinsics-guide/index.html>. Version 3.6.9; consulted 2026-07-28.
- [14] Intel Corporation. *Intel 64 and IA-32 Architectures Optimization Reference Manual*. Intel Corporation, 2026. URL <https://www.intel.com/content/www/us/en/developer/articles/technical/intel64-and-ia32-architectures-optimization.html>. Volumes 1 and 2, version 050; consulted 2026-07-28.
- [15] Intel Corporation. *Intel 64 and IA-32 Architectures Software Developer’s Manual*. Intel Corporation, 2026. URL <https://www.intel.com/content/www/us/en/developer/articles/technical/intel-sdm.html>. Combined volumes, version 092; consulted 2026-07-28.
- [16] Ruby B. Lee. Accelerating multimedia with enhanced microprocessors. *IEEE Micro*, 15(2):22–32, 1995. doi: 10.1109/40.372347.
- [17] Linux Kernel Community. Linux kernel documentation. Online documentation, 2026. URL <https://docs.kernel.org/>.
- [18] Linux man-pages project. *sched_setaffinity(2): Set and Get a Thread’s CPU Affinity Mask*. Linux man-pages project, 2026. URL https://man7.org/linux/man-pages/man2/sched_setaffinity.2.html. Consulted 2026-07-28.
- [19] Linux man-pages project. *syscall(2): Indirect System Call*. Linux man-pages project, 2026. URL <https://man7.org/linux/man-pages/man2/syscall.2.html>. Consulted 2026-07-28.
- [20] Linux perf maintainers. *perf-stat(1): Run a Command and Gather Performance Counter Statistics*. Linux perf, 2026. URL <https://www.man7.org/linux/man-pages/man1/perf-stat.1.html>. Linux manual page; consulted 2026-07-28.
- [21] David S. Miller. *Cache and TLB Flushing Under Linux*. Linux Kernel Documentation, 2026. URL <https://docs.kernel.org/core-api/cachetlb.html>. Consulted 2026-07-28.
- [22] David S. Miller, Richard Henderson, Jakub Jelinek, et al. *Dynamic DMA Mapping Guide*. Linux Kernel Documentation, 2026. URL <https://docs.kernel.org/core-api/dma-api-howto.html>. Consulted 2026-07-28.
- [23] Vijay Nagarajan, Daniel J. Sorin, Mark D. Hill, and David A. Wood. *A Primer on Memory Consistency and Cache Coherence*. Synthesis Lectures on Computer Architecture. Morgan & Claypool Publishers, 2nd edition, 2020. ISBN 978-1681737119. doi: 10.2200/S00962ED2V01Y201910CAC049.
- [24] David A. Patterson and John L. Hennessy. *Computer Organization and Design: The*

- Hardware/Software Interface*. Morgan Kaufmann, Cambridge, MA, 6th edition, 2020. ISBN 978-0128201091.
- [25] David A. Patterson and Carlo H. Sequin. A VLSI RISC. *Computer*, 15(9):8–21, 1982. doi: 10.1109/MC.1982.1654133.
- [26] RISC-V International. *RISC-V Ratified Specifications Library*. RISC-V International, 2026. URL <https://docs.riscv.org/reference/home/index.html>. Unprivileged and privileged ISA, version 20260120; consulted 2026-07-28.
- [27] Abraham Silberschatz, Peter B. Galvin, and Greg Gagne. *Operating System Concepts*. Wiley, Hoboken, NJ, 10th edition, 2018. ISBN 978-1119320913.
- [28] Dean M. Tullsen, Susan J. Eggers, and Henry M. Levy. Simultaneous multithreading: Maximizing on-chip parallelism. In *Proceedings of the 22nd Annual International Symposium on Computer Architecture*, pages 392–403. ACM, 1995. doi: 10.1145/223982.224449.
- [29] John von Neumann. First draft of a report on the edvac. Moore School of Electrical Engineering, University of Pennsylvania, 1945. URL <https://library.si.edu/digital-library/book/firstdraftofrepo00vonn>.
- [30] x86-64 psABI maintainers. *System V Application Binary Interface: AMD64 Architecture Processor Supplement*. x86 psABIs project, 2025. URL <https://gitlab.com/x86-psABIs/x86-64-ABI>. Development version, revision elce0983; consulted 2026-07-28.

14.2 Recursos en línea adicionales

Para continuar el estudio de sistemas operativos puede consultarse OSTEP (<https://pages.cs.wisc.edu/~remzi/OSTEP/>) y la documentación oficial del núcleo Linux (<https://docs.kernel.org/>). Las opciones de compilación y los informes de optimización se documentan en el manual de GCC (<https://gcc.gnu.org/onlinedocs/>).

Cuando una afirmación dependa de una ISA concreta, deben preferirse sus especificaciones. Los manuales Intel 64 e IA-32 están disponibles en <https://www.intel.com/content/www/us/en/developer/articles/technical/intel-sdm.html>; Arm publica su documentación en <https://developer.arm.com/documentation/>; y RISC-V International mantiene las especificaciones ratificadas en <https://riscv.org/specifications/ratified/>.